

Sunday 6:51 PM

test-face-clustering - P...
PDF

workflow_1 continuati...
PDF

Learn these PDF files thoroughly and deeply end to end so we can start implementing workflow 2 keeping the continuity alive.

Worked for 19s

Yes. I have now studied **both uploaded PDFs as the continuation source for this project**, including the PostgreSQL integration work, the Workflow 1 implementation/testing history, the architectural decisions, the verified runtime state, and the final Workflow 2 handoff.

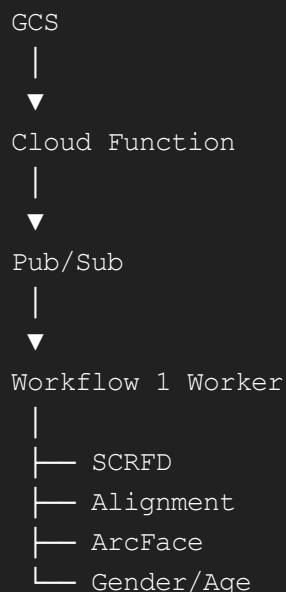
I will use these PDFs as the **continuity baseline** from this point forward.

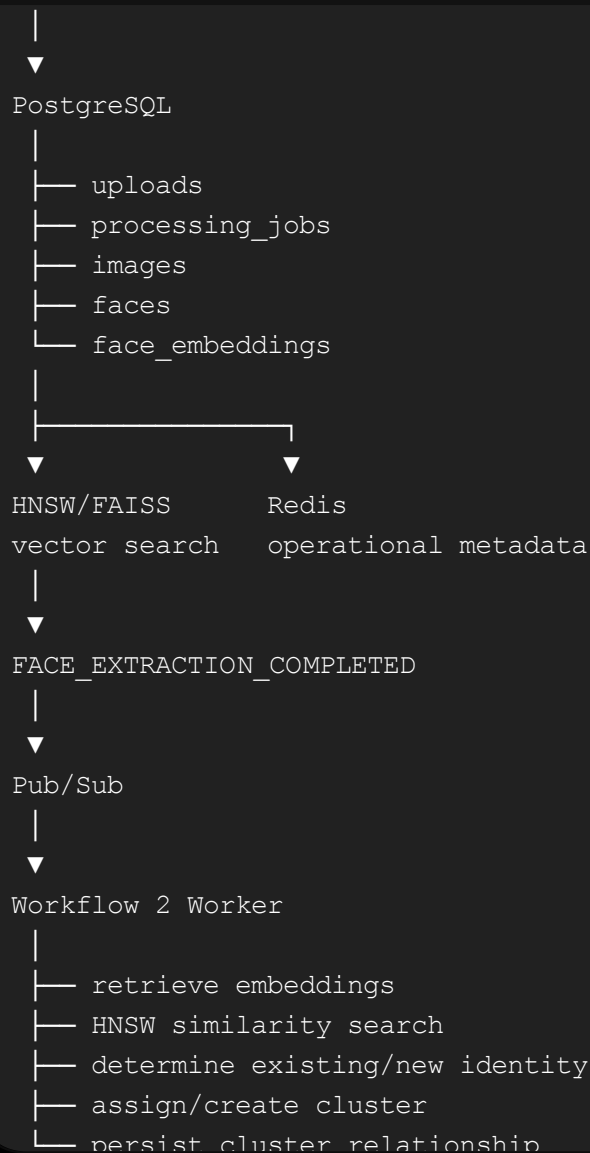
test-face-clustering - PostgreS...

workflow_1 continuation

1. The architecture is now firmly established

The micro-version we are building is:





This is explicitly the intended transition from the completed Workflow 1 vertical slice into Workflow 2. workflow_1 continuation

2. Workflow 1 is our protected foundation

The most important continuity rule is:

Do not redesign Workflow 1 while implementing Workflow 2.

Workflow 1 has already proven the important business/integration path.

The latest verified state was:

- GCS image successfully downloaded
- PostgreSQL upload created
- PostgreSQL processing job created
- PostgreSQL image created
- real FacePipeline executed

- SCRFD detected 5 final faces
- Alignment succeeded
- ArcFace generated 512-D embeddings
- age/gender processing succeeded
- 5 face records persisted
- 5 embedding metadata records persisted
- 5 vectors persisted in HNSW/FAISS
- Redis metadata persisted
- processing job became **COMPLETED**
- PostgreSQL transaction committed

The successful run specifically reported **5 final faces, 5/5 successful processing, and 0 failed faces.**

workflow_1 continuation

The HNSW relationship was also explicitly demonstrated:

```
PostgreSQL face_id
      |
      └─ vector_id = face:<face_id>
            |
            ▼
          HNSW
```

with:

```
face:1 → FAISS ID 0
face:2 → FAISS ID 1
face:3 → FAISS ID 2
face:4 → FAISS ID 3
face:5 → FAISS ID 4
```

and the persisted index contained 5 vectors.

workflow_1 continuation

So **Workflow 2 consumes the results of this pipeline; it does not repeat the AI work.**

3. The ownership boundaries are critical

I have locked these into the continuity model:

PostgreSQL

Authoritative persistent metadata and processing state.

It owns:

```
uploads
processing_jobs
images
faces
face_embeddings
```

The database does **not** store the actual 512-D vector. It stores the vector metadata and deterministic vector ID.

test-face-clustering - PostgreS...

HNSW / FAISS

Active vector similarity-search layer.

It stores the actual ArcFace vectors.

The previously tested implementation uses FAISS's native:

```
IndexHNSWFlat
```

rather than introducing `hnswlib`. The PDF explicitly recommends keeping this architecture for the micro implementation because `faiss-cpu` is already working.

workflow_1 continuation

Redis

Transient/operational metadata only.

It is not the source of truth and must not replace PostgreSQL.

workflow_1 continuation

GCS

Original image/object storage.

Pub/Sub

Asynchronous workflow communication.

ProcessingService

Workflow orchestration.

PubSubSubscriber

Transport adapter only:

```
decode
  ↓
validate event
  ↓
```

ProcessingService

↓

ACK / NACK

It must not acquire database, AI, GCS, or clustering business logic.

test-face-clustering - PostgreS...

4. The most important Workflow 1 → Workflow 2 boundary

Workflow 2 must not receive the original `UploadEvent`.

Workflow 1 finishes successfully and publishes a different event:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 4,
  "image_id": 4,
  "bucket": "photo-micro-upload-test",
  "blob_name": "uploads/e2e-test-003.jpg",
  "face_count": 5
}
```

The semantic meaning is:

Workflow 1 has successfully extracted and persisted all faces and embeddings.

Only after successful completion should Workflow 1 publish this completion event and then ACK the original message. `workflow_1 continuation`

That gives us a clean microservice boundary:

Workflow 1

|

| FACE_EXTRACTION_COMPLETED

▼

Pub/Sub

|

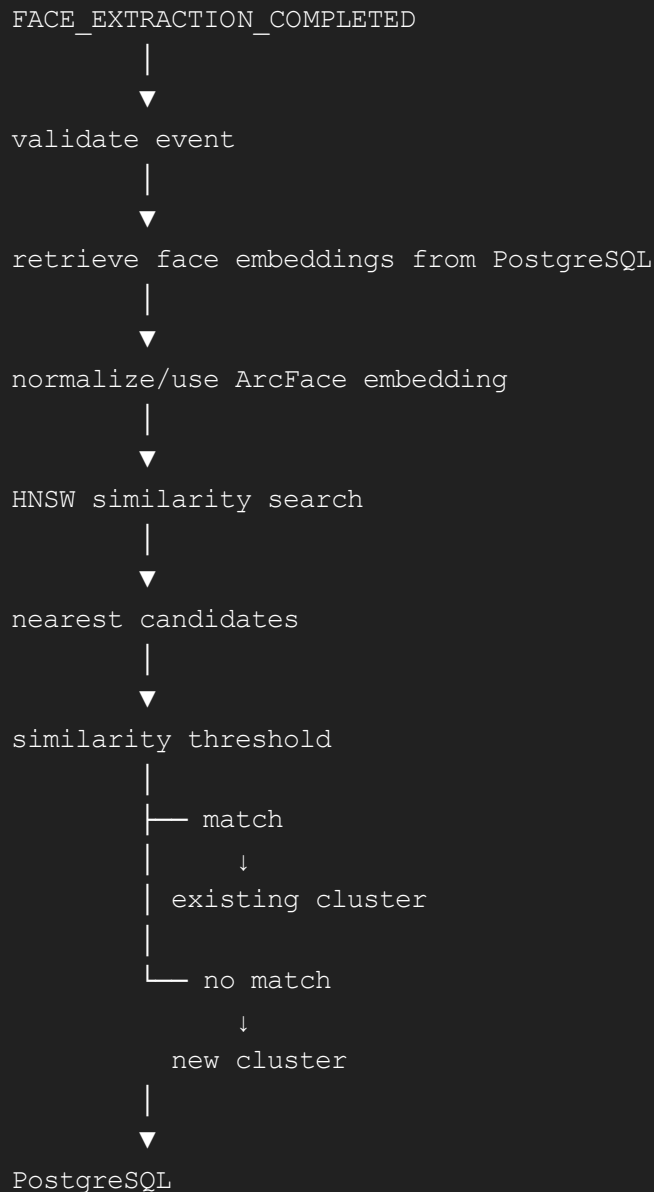
▼

Workflow 2

5. What Workflow 2 actually does

Workflow 2 is **not** another face-processing pipeline.

It does:



The PDFs explicitly state that Workflow 2 does **not** rerun:

- SCRFD
- alignment
- ArcFace
- gender/age

because Workflow 1 has already completed those operations.

workflow_1 continuation

6. Identity is based on the ArcFace embedding

This is another architectural rule I have locked in.

For identity matching:

Use

```
ArcFace embedding
  ↓
vector similarity
  ↓
cluster decision
```

Do NOT use

```
gender
age
age group
```

as identity evidence.

Gender and age are **attributes of the detected face**, not identity keys. The PDF explicitly calls out this separation.

workflow_1 continuation

7. The first Workflow 2 database concept

The existing Workflow 1 schema remains untouched.

Workflow 2 introduces the clustering layer conceptually:

```
face_embeddings
  |
  ▼
face_clusters
|
|— id
|— created_at
|— updated_at
  |
  ▼
face_cluster_members
  |
  |— id
```

```
|— cluster_id
|— face_id
|— similarity
└— created_at
```

Relationship:

```
face
|
▼
face_cluster_members
|
▼
face_cluster
```

Example:

```
Cluster 1
|— face 1
|— face 6
└— face 11

Cluster 2
|— face 2
└— face 8

Cluster 3
|— face 3
|— face 9
└— face 14
```

This is the first database representation of the **unique-person concept**.

workflow_1 continuation

8. HNSW's role in Workflow 2

The HNSW index is still a **vector index**, not a person database.

Conceptually:

```
HNSW internal vector ID
|
▼
cluster/person ID
```

For example:


```
vector #0 → cluster 1  
vector #1 → cluster 1  
vector #2 → cluster 7  
vector #3 → cluster 3
```

The index handles nearest-neighbor search.

PostgreSQL tells us what those vectors mean.

That separation is explicitly preserved in the Workflow 2 design.

workflow_1 continuation

9. The first clustering algorithm will deliberately be simple

We are **not** implementing the complete production clustering intelligence yet.

The micro-version is intentionally:

```
new ArcFace embedding  
    ↓  
normalize  
    ↓  
HNSW search  
    ↓  
nearest candidates  
    ↓  
similarity threshold  
    |  
    ├── >= threshold → existing cluster  
    |  
    └── < threshold → new cluster
```

The production architecture may eventually introduce:

```
ambiguity  
    ↓  
Match Triage  
    ↓  
Triage Agent  
    ↓  
resolved / unresolved
```

but that is explicitly **later**, not part of our first Workflow 2 implementation.

workflow_1 continuation

This is important because we don't want to accidentally over-engineer Workflow 2 before proving the fundamental unique-person clustering path.

10. The first real proof we're aiming for

The PDFs give us an excellent acceptance test.

Image 1

Workflow 1 produces:

```
Person A
Person B
Person C
Person D
Person E
```

Workflow 2 initially creates:

```
Cluster 1 → A
Cluster 2 → B
Cluster 3 → C
Cluster 4 → D
Cluster 5 → E
```

Image 2

Suppose it contains:

```
A
B
F
G
C
```

Workflow 2 should produce:

```
Cluster 1 → A → image 1 + image 2
Cluster 2 → B → image 1 + image 2
Cluster 3 → C → image 1 + image 2
Cluster 4 → D → image 1
Cluster 5 → E → image 1
Cluster 6 → F → image 2
Cluster 7 → G → image 2
```

That is the first genuine proof that we are performing **unique-person clustering**, rather than simply extracting faces.

workflow_1 continuation

11. The intended Workflow 2 repository/service structure

The continuation PDF already establishes the initial micro-version structure:

```
micro-face-pipeline/
├── workflow1_worker/
├── workflow2_clustering/
│   ├── app/
│   │   ├── __init__.py
│   │   └── main.py
│   ├── clustering/
│   │   ├── __init__.py
│   │   ├── hnsw_store.py
│   │   └── clustering_service.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── face_event.py
│   ├── services/
│   │   └── postgres_service.py
│   └── subscriber/
│       └── pubsub_subscriber.py
├── data/
│   └── hnsw/
├── requirements.txt
└── .env
```

The purpose is to keep Workflow 2 independently deployable from Workflow 1.

workflow_1 continuation

12. The correct implementation order is already defined

I will follow this order rather than jumping directly into a large implementation:

Phase 1 — Workflow 2 PostgreSQL cluster schema

Create/verify:

```
face_clusters
face_cluster_members
```

and their relationships/constraints.

Phase 2 — Workflow 2 HNSW store

Implement:

```
add()
search()
save()
load()
```

using FAISS `IndexHNSWFlat`.

Phase 3 — ClusteringService

Implement:

```
FaceEmbedding
  ↓
HNSW search
  ↓
candidate
  ↓
threshold
  ↓
existing/new cluster
  ↓
cluster membership persistence
```

Phase 4 — Completion event model

Implement validation for:

```
FACE_EXTRACTION_COMPLETED
```

Phase 5 — Workflow 1 completion publisher

Only after the existing Workflow 1 processing succeeds:

```
PostgreSQL
HNSW
```

```
Redis
  ↓
FACE_EXTRACTION_COMPLETED
  ↓
ACK original Workflow 1 message
```

Phase 6 — Workflow 2 subscriber

```
Pub/Sub
  ↓
decode
  ↓
validate FaceExtractionCompleted
  ↓
ClusteringService
  ↓
ACK/NACK
```

Phase 7 — Single-image Workflow 2 test

Process the already-existing Workflow 1 output.

Phase 8 — Two-image identity test

Use the Image 1/Image 2 scenario above.

This exact incremental approach is what the PDF recommends to avoid destabilizing Workflow 1.

workflow_1 continuation

13. What I have specifically retained from the PostgreSQL work

The important database idempotency boundaries are:

```
uploads
  event_id UNIQUE

images
  (bucket, blob_name) UNIQUE

faces
  (image_id, face_index) UNIQUE

face_embeddings
  face_id UNIQUE

processing_jobs
```

(upload_id, workflow) UNIQUE

These are fundamental because Pub/Sub provides **at-least-once delivery**.

Therefore:

```
duplicate Pub/Sub event
    ↓
same event_id
    ↓
same upload/job
    ↓
no duplicate image/face/embedding records
```

This idempotency principle was repeatedly validated in the Workflow 1 work.

workflow_1 continuation

For Workflow 2 we will need the same mindset.

14. One subtle persistence issue I will preserve

PostgreSQL and HNSW are **two independent persistence systems**.

Therefore:

```
PostgreSQL transaction
    ≠
HNSW transaction
```

They cannot be treated as one atomic transaction.

The established Workflow 1 ordering was:

```
Create/recover upload
    ↓
Create/recover processing job
    ↓
Mark processing
    ↓
GCS download
    ↓
Persist image
    ↓
AI processing
    ↓
Persist faces
    ↓
```

```
Add embeddings to HNSW
↓
Persist embedding metadata
↓
Commit PostgreSQL
↓
Save HNSW
↓
Update Redis
↓
Mark job completed
↓
ACK
```

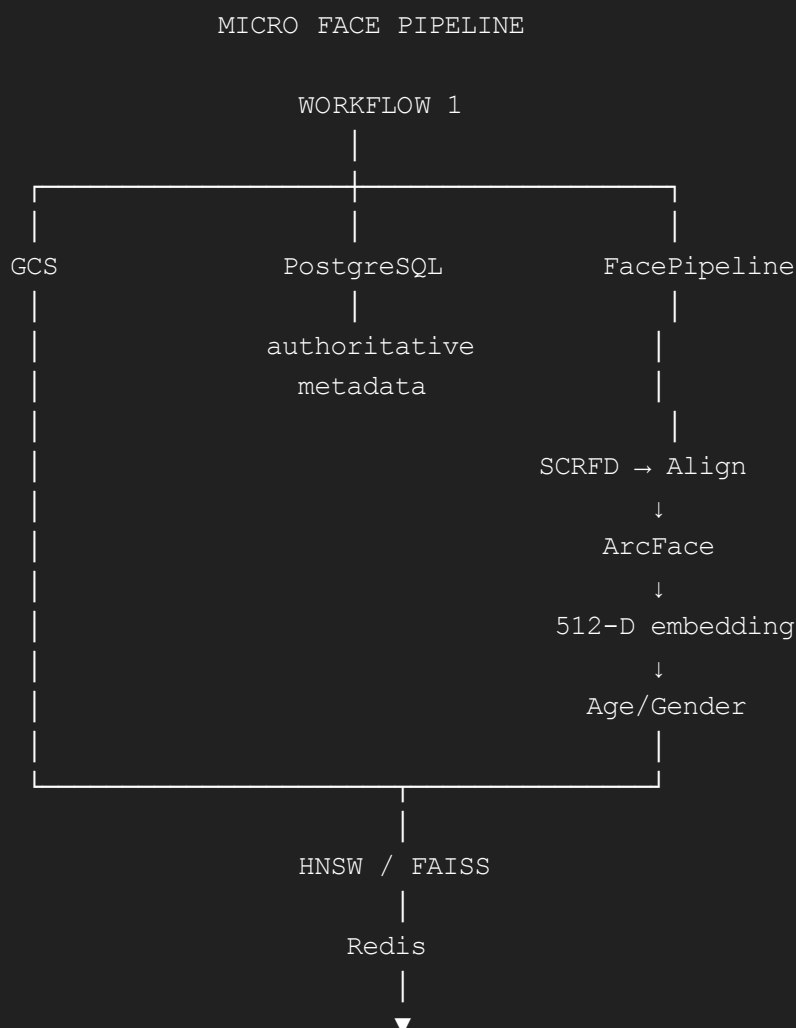
The PDF explicitly warns against pretending PostgreSQL and HNSW are one transaction.

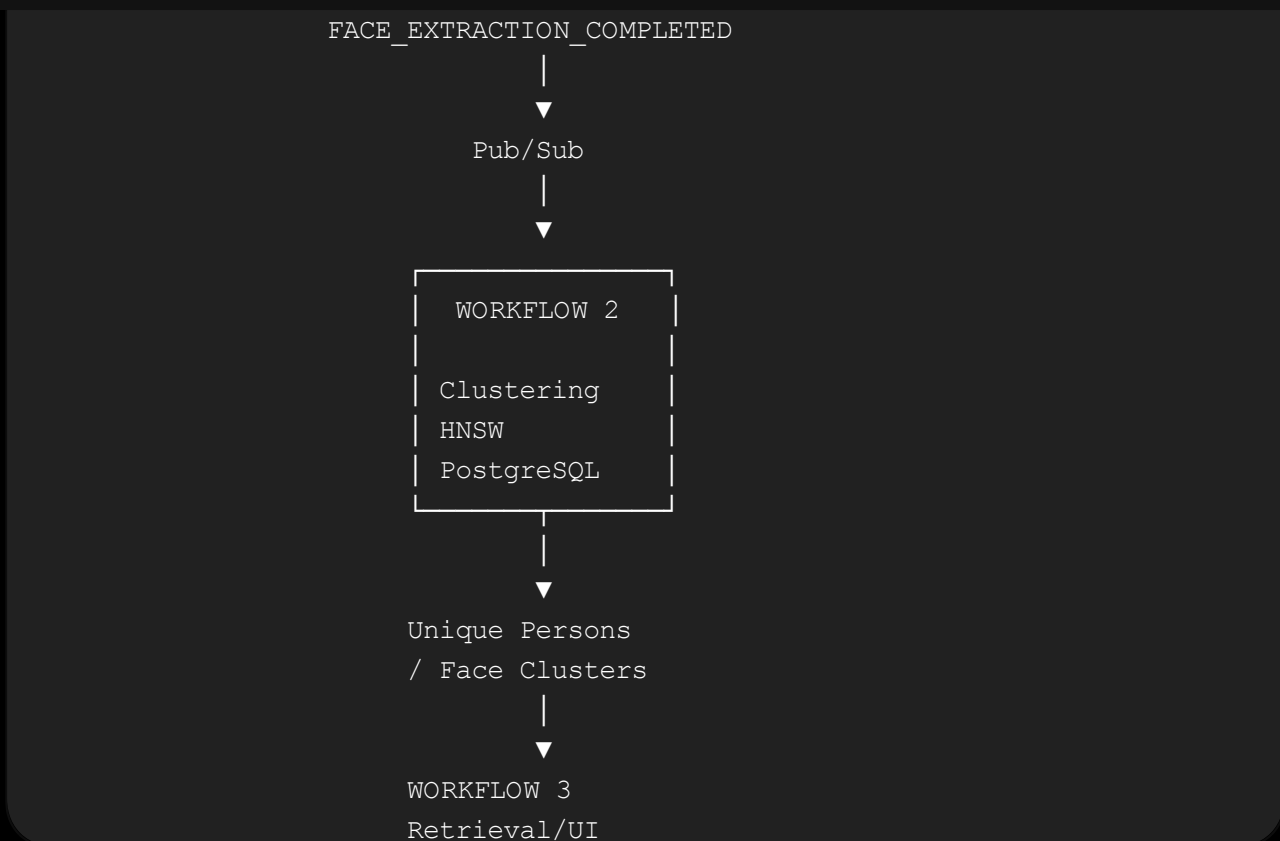
test-face-clustering - PostgreS...

We'll need to apply the same discipline when designing Workflow 2 recovery/idempotency.

15. Current project state — continuity locked

So, as of now, I consider the project state to be:





The overall complete-vs-micro architecture distinction is also explicitly documented: Workflow 1 is the currently proven vertical slice, Workflow 2 is the next processing/tiering layer, and Workflow 3 remains future work. workflow_1 continuation

Most important conclusion

We are ready to start Workflow 2.

And I will **not** go back and redesign Workflow 1, change the validated SCRFD/ArcFace pipeline, replace HNSW, move vectors into PostgreSQL, or turn Redis into a source of truth.

The correct next technical action is:

inspect the actual current PostgreSQL schema/state and existing Workflow 2-related files, then implement the `face_clusters` / `face_cluster_members` persistence layer first.

That follows the exact handoff point at the end of the second PDF: *“implement the PostgreSQL cluster schema first, then the Workflow 2 HNSW store, then the completion-event publisher/subscriber.”* workflow_1 continuation

I have the continuity now, so you **do not need to re-explain the architecture or the PostgreSQL Repository/Workflow 1 history** when we begin Workflow 2.

lets proceed

Worked for 17s

Absolutely. We'll proceed **incrementally**, starting with the first Workflow 2 persistence layer and not touching the proven Workflow 1 code.

The source material explicitly says the next step is to add the clustering layer first, then the Workflow 2 HNSW store, and only afterward connect the completion-event publisher/subscriber.

workflow_1 continuation

Step 1 — Add the Workflow 2 PostgreSQL clustering tables

Your existing database already has:

```
face_pipeline
├─ uploads
├─ processing_jobs
├─ images
├─ faces
└─ face_embeddings
```

The source defines the new clustering layer conceptually as:

```
face_clusters
-----
id
created_at
updated_at

face_cluster_members
-----
id
cluster_id
face_id
similarity
created_at
```

with:

```
face
|
▼
face_cluster_members
```

|
▼
face_cluster

workflow_1 continuation

Important distinction

We are **not** changing:

- uploads
- processing_jobs
- images
- faces
- face_embeddings
- existing HNSW/FAISS implementation
- Workflow 1 AI pipeline
- Workflow 1 subscriber

The existing schema and HNSW relationship are already validated. PostgreSQL stores the vector metadata/reference, while HNSW stores the actual 512-D embedding.

workflow_1 continuation

1. Create the migration SQL

For the micro-version, create a file in your project root:

```
workflow2_clustering/  
└─ sql/  
    └─ 001_create_clustering_tables.sql
```

Put this in it:

SQL

```
SET search_path TO face_pipeline;  
  
CREATE TABLE IF NOT EXISTS face_clusters (  
    id BIGSERIAL PRIMARY KEY,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS face_cluster_members (  
    id BIGSERIAL PRIMARY KEY,  
  
    cluster_id BIGINT NOT NULL,
```

```

    face_id BIGINT NOT NULL,

    similarity REAL NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT fk_face_cluster_members_cluster
        FOREIGN KEY (cluster_id)
        REFERENCES face_clusters(id),

    CONSTRAINT fk_face_cluster_members_face
        FOREIGN KEY (face_id)
        REFERENCES faces(id),

    CONSTRAINT uq_face_cluster_members_face
        UNIQUE (face_id)
);

CREATE INDEX IF NOT EXISTS idx_face_cluster_members_cluster_id
    ON face_cluster_members(cluster_id);

CREATE INDEX IF NOT EXISTS idx_face_cluster_members_face_id
    ON face_cluster_members(face_id);

```

One deliberate implementation decision

The source does not explicitly state the SQL constraints, so this part is an **implementation inference**, not something directly specified in the PDFs.

I'm making:

SQL

```

UNIQUE (face_id)

```

because in our micro-version a face should belong to **one current cluster**:

```

face 101 → cluster 7

```

rather than:

```

face 101 → cluster 7
face 101 → cluster 12

```

That gives us a strong invariant:

One face = one cluster membership.

If we later introduce cluster merging/splitting or historical cluster assignments, we can evolve this model deliberately rather than prematurely.

2. Why we need two tables

This is important for the architecture.

```
face_clusters
```

Represents the unique-person/cluster identity:

```
id = 1
```

Conceptually:

```
Cluster 1
```

```
face_cluster_members
```

Represents which detected faces belong to that cluster:

cluster_id	face_id	similarity
1	1	1.000
1	6	0.91
1	11	0.94

So:

```
Cluster 1
|
├─ face 1
├─ face 6
└─ face 11
```

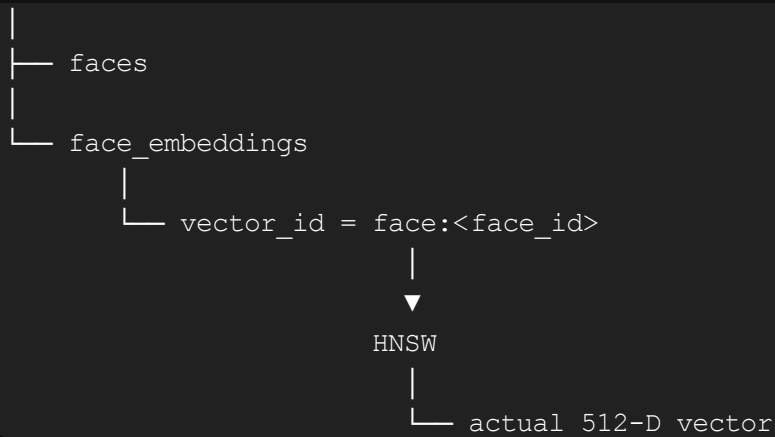
This is exactly the first "unique person" abstraction described in the Workflow 2 continuation.

```
workflow_1 continuation
```

3. Do NOT put the embedding into these tables

The architecture remains:

```
PostgreSQL
```



The clustering tables contain **relationships and decisions**, not the 512-dimensional ArcFace vector.

This preserves the established separation:

```
PostgreSQL → authoritative metadata
HNSW        → vector similarity search
Redis       → transient operational state
```

workflow_1 continuation

4. Apply the migration

Use the same PostgreSQL database/schema that Workflow 1 is already using.

From `psql`, first verify:

SQL

```
SET search_path TO face_pipeline;

SELECT current_database();

SELECT current_schema();
```

Then execute the migration.

If you're using a SQL file from Windows:

bat

```
psql -U YOUR_POSTGRES_USER -d YOUR_DATABASE -f workflow2_clustering\sql\001_c
```

Use your **existing PostgreSQL credentials/database values** from Workflow 1. Don't create another database.

5. Verify the schema

After the migration:

SQL

```
SET search_path TO face_pipeline;  
  
\d face_clusters
```

and:

SQL

```
\d face_cluster_members
```

We want the essential structure to be:

face_clusters

```
id  
created_at  
updated_at
```

and:

face_cluster_members

```
id  
cluster_id  
face_id  
similarity  
created_at
```

with the foreign keys:

```
face_cluster_members.cluster_id  
    ↓  
face_clusters.id  
  
face_cluster_members.face_id  
    ↓  
faces.id
```

Then verify indexes/constraints:

SQL

```
SELECT
    tc.constraint_name,
    tc.constraint_type
FROM information_schema.table_constraints tc
WHERE tc.table_schema = 'face_pipeline'
    AND tc.table_name IN (
        'face_clusters',
        'face_cluster_members'
    )
ORDER BY tc.table_name, tc.constraint_name;
```

And:

SQL

```
SELECT
    indexname,
    indexdef
FROM pg_indexes
WHERE schemaname = 'face_pipeline'
    AND tablename IN (
        'face_clusters',
        'face_cluster_members'
    )
ORDER BY tablename, indexname;
```

6. Do one very important integrity test

We already have Workflow 1 data in PostgreSQL.

The PDFs indicate that the current successful database contains multiple completed images and face/embedding records; the latest verification showed `uploads = 3`, `processing_jobs = 3`, `images = 3`, `faces = 15`, and `embeddings = 15`.

workflow_1 continuation

So after creating the tables, run:

SQL

```
SET search_path TO face_pipeline;

SELECT COUNT(*) AS faces
FROM faces;

SELECT COUNT(*) AS embeddings
```

```
FROM face_embeddings;

SELECT COUNT(*) AS clusters
FROM face_clusters;

SELECT COUNT(*) AS memberships
FROM face_cluster_members;
```

We expect the important part to be:

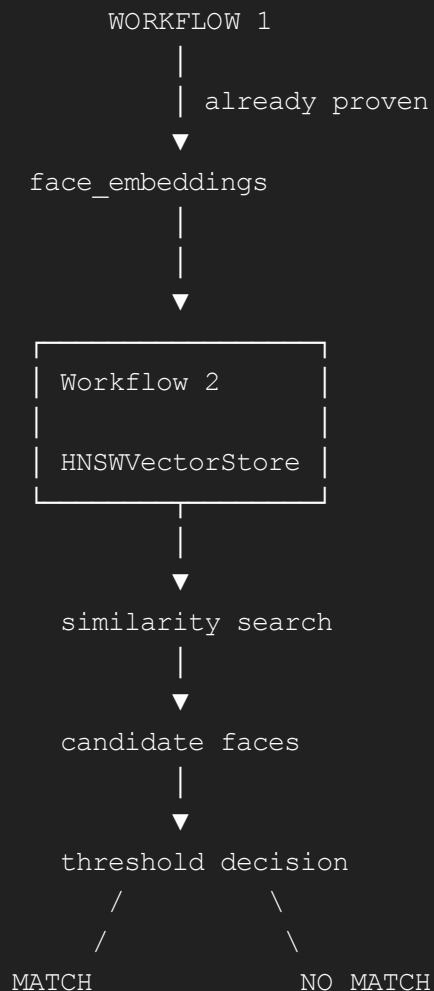
```
faces          > 0
embeddings     > 0
clusters       = 0
memberships    = 0
```

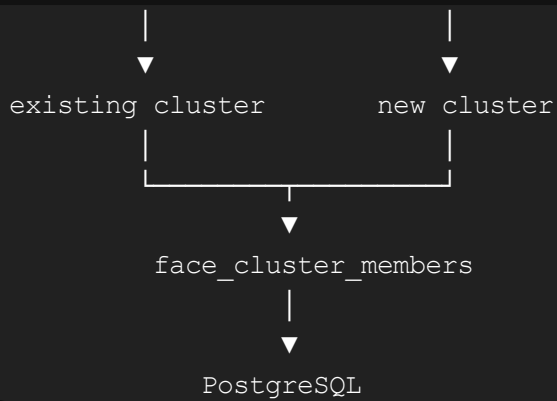
That is exactly what we want.

Workflow 1 data exists, but Workflow 2 has not clustered anything yet.

7. The next architecture after this migration

Once this passes, our implementation will become:





The source specifically defines the micro-version decision as:

```
normalize embedding
  ↓
HNSW search
  ↓
nearest candidates
  ↓
similarity threshold
  ↓
similarity >= threshold
    → existing cluster

similarity < threshold
    → new cluster
```

One thing we will not decide yet

We will **not arbitrarily choose the final production similarity threshold right now**.

The source establishes that a threshold is required, but it does not give us a validated numerical threshold. workflow_1 continuation

So when we implement `ClusteringService`, we'll make it configurable:

```
CLUSTER_SIMILARITY_THRESHOLD=...
```

and test it against actual faces from your pipeline rather than pretending an unvalidated number is authoritative.

Likewise, we will not use:

```
gender
age
age_group
```

for identity matching. Identity matching is based primarily on the ArcFace embedding; gender/age remain attributes.

workflow_1 continuation

Our immediate checkpoint

Do only Step 1 now.

Apply the migration and send me the complete output of:

SQL

```
\d face_clusters
\d face_cluster_members
```

plus:

SQL

```
SELECT COUNT(*) FROM face_clusters;
SELECT COUNT(*) FROM face_cluster_members;
```

Once that passes, we'll move directly to **Step 2: build the Workflow 2** `HNSWVectorStore` **against the already-existing 512-D vectors**, while preserving the existing Workflow 1 HNSW architecture.

```
face_pipeline=# SET search_path TO face_pipeline;
SET
face_pipeline=#
face_pipeline=# CREATE TABLE IF NOT EXISTS
face_clusters (
face_pipeline(#  id BIGSERIAL PRIMARY KEY,
face_pipeline(#  created_at TIMESTAMPTZ NOT NULL
DEFAULT NOW(),
face_pipeline(#  updated_at TIMESTAMPTZ NOT NULL
DEFAULT NOW()
face_pipeline(# );
```

Show more

Excellent. **Step 1 is fully successful.** We now have the Workflow 2 PostgreSQL clustering layer in place without disturbing any Workflow 1 data.

Your output confirms:

- `face_clusters` exists with PK `id`.
- `face_cluster_members` exists.
- `cluster_id → face_clusters(id)` FK exists.
- `face_id → faces(id)` FK exists.
- `face_id` is unique, enforcing **one current cluster membership per face**.
- Required indexes exist.
- Existing Workflow 1 data is intact: **20 faces / 20 embeddings**.
- Workflow 2 has correctly started empty: **0 clusters / 0 memberships**.

That is exactly the clean starting state we want. The source architecture calls for these two clustering tables before implementing the Workflow 2 HNSW store. [workflow_1 continuation](#)

Step 2 — Workflow 2 HNSW Vector Store

Now we move to the next layer.

There is one important architectural distinction before we write code.

We are NOT modifying Workflow 1's `FaissService`

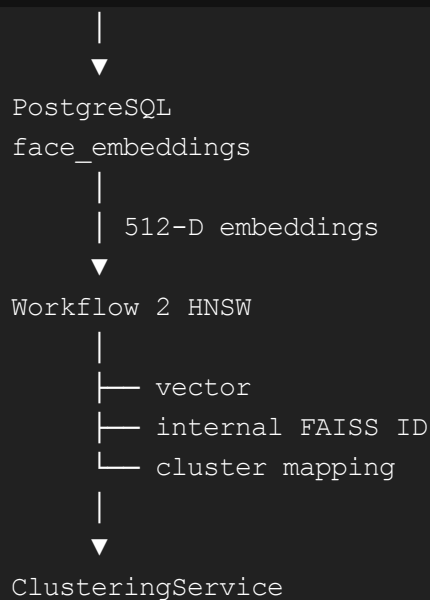
The existing Workflow 1 HNSW/FAISS layer is already validated for:

- 512-D vectors
- HNSW creation
- insertion
- similarity search
- idempotency
- persistence
- reload
- search after reload
- `face:<face_id>` mapping [test-face-clustering - PostgreS...](#)

So it stays untouched.

Workflow 2 gets its **own active clustering vector store**.

The intended micro architecture is:



The source explicitly defines Workflow 2 as independently deployable with its own:

```
workflow2_clustering/  
  app/  
    clustering/  
      hnsw_store.py  
      clustering_service.py
```

and recommends FAISS's native `IndexHNSWFlat` rather than introducing `hnswlib`.

workflow_1 continuation +1

2.1 Create the Workflow 2 directory

At the same level as `workflow1_worker`:

```
micro-face-pipeline/  
├── workflow1_worker/  
└── workflow2_clustering/  
    ├── app/  
    │   ├── __init__.py  
    │   └── clustering/  
    │       ├── __init__.py  
    │       └── hnsw_store.py  
    ├── data/  
    │   └── hnsw/  
    └──
```

For now, **only these files matter**:

```
hnsw_store.py
test_hnsw_store.py
```

We will not implement `ClusteringService` yet.

2.2 Why we're separating `HNSWVectorStore`

We want this responsibility:

```
HNSWVectorStore
```

to know only about:

```
embedding
vector ID
cluster ID
similarity search
persistence
```

It should **not** know about:

```
PostgreSQL sessions
Pub/Sub
GCS
FaceRecord
gender
age
```

That separation is directly consistent with the source architecture:

HNSW stores vectors; PostgreSQL remains the source of metadata truth.

workflow_1 continuation

2.3 One design decision: what does a Workflow 2 vector represent?

For the micro-version:

```
one face
```

↓

```
one 512-D embedding
↓
one HNSW vector
```

The HNSW mapping will conceptually be:

```
FAISS internal ID
|
▼
face_id
|
▼
cluster_id
```

For example:

```
FAISS #0 → face 1 → cluster 1
FAISS #1 → face 2 → cluster 2
FAISS #2 → face 3 → cluster 3
FAISS #3 → face 6 → cluster 1
FAISS #4 → face 7 → cluster 2
```

This allows:

```
new face embedding
↓
HNSW search
↓
nearest face
↓
nearest face's cluster
↓
candidate cluster
```

The source explicitly describes this mapping conceptually as:

```
vector #0 → cluster 1
vector #1 → cluster 1
vector #2 → cluster 7
vector #3 → cluster 3
```

workflow_1 continuation

2.4 Create `hnsw_store.py`

Create:

Use this implementation:

Python

Run

```
from __future__ import annotations

import json
import threading
from pathlib import Path
from typing import Any

import faiss
import numpy as np

class HNSWVectorStore:
    """
    Workflow 2 active vector store.

    Responsibilities
    -----
    - Store 512-D face embeddings.
    - Use FAISS IndexHNSWFlat.
    - Normalize embeddings before add/search.
    - Use inner-product similarity.
    - Maintain FAISS internal-ID -> application vector-ID mapping.
    - Maintain vector-ID -> cluster-ID mapping.
    - Persist the HNSW index and metadata.
    - Provide idempotent insertion.
    - Provide similarity search.
    """

    def __init__(
        self,
        index_path: str | Path = "data/hnsw/index.bin",
        metadata_path: str | Path = "data/hnsw/metadata.json",
        dimension: int = 512,
        M: int = 32,
        ef_construction: int = 200,
        ef_search: int = 64,
```

2.5 Why this implementation is deliberately conservative

The source calls for the HNSW abstraction to expose essentially:

```
add()  
search()  
save()  
load()
```

and to maintain the vector/cluster relationship while leaving metadata authority in PostgreSQL. [workflow_1 continuation](#)

The implementation above adds a few safety mechanisms:

512-D validation

```
embedding.shape == (1, 512)
```

because Workflow 1's ArcFace output is already validated as 512-D.

[test-face-clustering - PostgreS...](#)

L2 normalization

Because the established architecture uses normalized ArcFace embeddings and inner-product similarity as cosine similarity.

Idempotency

```
face:101
```

cannot accidentally be inserted twice.

Cluster consistency

If:

```
face:101 → cluster 3
```

already exists, attempting:

```
face:101 → cluster 7
```

raises an error rather than silently corrupting the mapping.

Persistence

Both:

```
index.bin  
metadata.json
```

are persisted.

This follows the already-proven Workflow 1 FAISS persistence model.

test-face-clustering - PostgreS...

2.6 One important thing we are intentionally NOT doing

We are **not** populating this HNSW store with the 20 existing faces yet.

Why?

Because we haven't implemented the clustering algorithm.

We first need to prove:

```
HNSWVectorStore
  ↓
add
  ↓
search
  ↓
save
  ↓
reload
  ↓
search
```

independently.

Then:

ClusteringService

will decide:

```
existing cluster
    OR
new cluster
```

The source specifically recommends testing each layer independently before connecting the completion-event/Pub/Sub path.

workflow_1 continuation

2.7 Our Step 2 test

Create:

```
workflow2_clustering/test_hnsw_store.py
```

and use:

Python

Run

```
from __future__ import annotations

import shutil
import tempfile
from pathlib import Path

import numpy as np

from app.clustering.hnsw_store import HNSWVectorStore

def main() -> None:
    temp_dir = Path(
        tempfile.mkdtemp(
            prefix="workflow2_hnsw_test_"
        )
    )

    print("=" * 70)
    print("WORKFLOW 2 HNSW VECTOR STORE TEST")
    print("=" * 70)

    try:
        index_path = temp_dir / "index.bin"
        metadata_path = temp_dir / "metadata.json"

        print("\n[1] Creating HNSWVectorStore...")

        store = HNSWVectorStore(
            index_path=index_path,
            metadata_path=metadata_path,
            dimension=512,
        )

        print(
            "Initial vector count:",
```

2.8 Run it

From:

```
micro-face-pipeline\workflow2_clustering
```

run:

```
bat
```

```
python test_hnsw_store.py
```

We want to see:

```
WORKFLOW 2 HNSW VECTOR STORE TEST
```

```
[1] Creating HNSWVectorStore...  
Initial vector count: 0
```

```
[2] Adding face:101...  
Added: True  
Vector count: 1
```

```
[3] Adding face:102...  
Added: True  
Vector count: 2
```

```
[4] Testing duplicate insertion...  
Duplicate insertion result: False
```

```
[5] Searching using face:101...  
...  
Nearest-neighbor check: PASS
```

```
[6] Checking persistence...  
index.bin exists: True  
metadata.json exists: True
```

```
[7] Creating fresh HNSWVectorStore...  
Restored vector count: 2  
Reload count check: PASS
```

```
[8] Searching restored index...  
...  
Reload search check: PASS
```

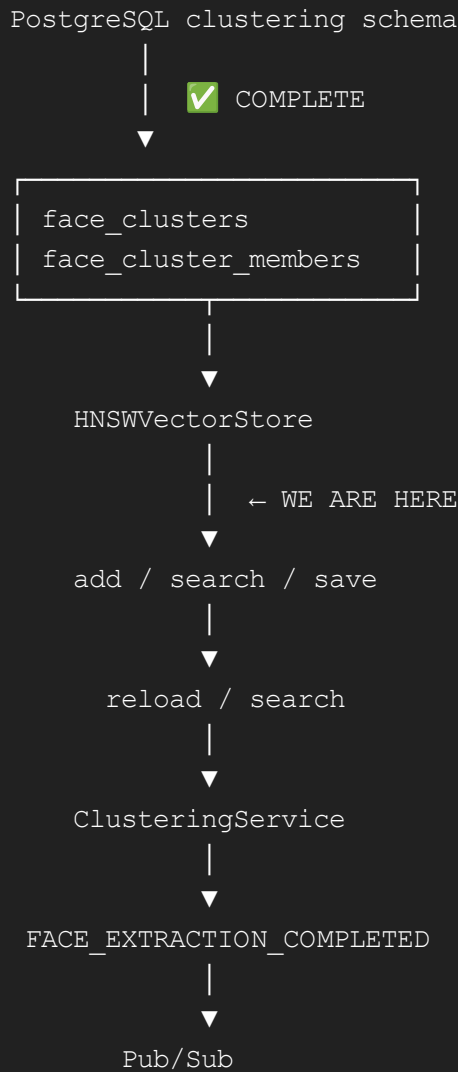
```
WORKFLOW 2 HNSW VECTOR STORE: PASS
```

This is our **second controlled checkpoint**.

Current milestone

We are now here:

WORKFLOW 2



The **20 existing faces and 20 embeddings remain untouched**, which is exactly what we want.

Your next action

Create the two Workflow 2 files above and run:

```
bat
python test_hnsw_store.py
```

Paste the complete output here. If it passes, we will proceed to **Step 3** —

`ClusteringService`, where we will finally take a real 512-D face embedding and make the first actual **new-cluster vs existing-cluster** decision.

Sources

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_hnsw_store.py
```

```
=====
=====
WORKFLOW 2 HNSW VECTOR STORE TEST
=====
=====
```

```
[1] Creating HNSWVectorStore...
Initial vector count: 0
```

Show more

Good — this is a **real implementation bug in the Workflow 2 code I gave you**, not an environment/FAISS problem.

The important part is that the first half passed:

ADD	✓
DUPLICATE HANDLING	✓
SEARCH	✓
PERSISTENCE FILES	✓

The failure occurs only during reload:

```
AttributeError:
'HNSWVectorStore' object has no attribute 'index'
```

Why it happened

In `__init__()` we currently have:

Python

Run

```
self.index = self._create_or_load_index()
```

But inside `_create_or_load_index()` we do:

Python

Run

```
self._load_metadata()
```

and `_load_metadata()` does:

Python

Run

```
if len(self.vector_to_faiss) != self.index.ntotal:
```

At that moment, `self.index` **hasn't been assigned yet**.

The sequence is effectively:

```
__init__
├── self.index = ?
├── _create_or_load_index()
│   ├── read index
│   └── _load_metadata()
│       └── self.index.ntotal ← too early
```

So this is an initialization-order bug.

Fix it cleanly

I recommend **not** using a temporary `self.index` assignment. Instead, make `_load_metadata()` receive the already-loaded FAISS index.

This makes the lifecycle explicit and safer.

1. Replace `_create_or_load_index()`

In:

```
workflow2_clustering/app/clustering/hnsw_store.py
```

replace the current `_create_or_load_index()` with:

Python

Run

```
def _create_or_load_index(self) -> faiss.Index:
    self.index_path.parent.mkdir(
        parents=True,
        exist_ok=True,
    )

    self.metadata_path.parent.mkdir(
```

```

        parents=True,
        exist_ok=True,
    )

    if self.index_path.exists():
        index = faiss.read_index(
            str(self.index_path)
        )

        if index.d != self.dimension:
            raise ValueError(
                "HNSW dimension mismatch: "
                f"index={index.d}, "
                f"configured={self.dimension}"
            )

        if hasattr(index, "hnsw"):
            index.hnsw.efSearch = self.ef_search

        self._load_metadata(index)

        return index

    return self.create_index()

```

The important change is:

Python

Run

```
self._load_metadata(index)
```

instead of:

Python

Run

```
self._load_metadata()
```

2. Replace `_load_metadata()`

Replace the existing method with:

Python

Run

```

def _load_metadata(
    self,
    index: faiss.Index,
) -> None:
    if not self.metadata_path.exists():

```


I've added two additional consistency checks:

```
vector_to_faiss count
faiss_to_vector count
vector_to_cluster count
```

must all agree with:

```
index.ntotal
```

That's useful because our Workflow 2 HNSW metadata is deliberately persisted separately from the FAISS index.

3. Don't change anything else yet

Your current test already proved:

```
[2] Adding face:101...
Added: True
Vector count: 1

[3] Adding face:102...
Added: True
Vector count: 2

[4] Testing duplicate insertion...
Duplicate insertion result: False

[5] Searching using face:101...
similarity: 1.0
Nearest-neighbor check: PASS

[6] Checking persistence...
index.bin exists: True
metadata.json exists: True
```

So do not modify the add/search logic.

The only problem is the reload initialization sequence.

4. Run the test again

From:

```
C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\w
```

run:

bat

```
python test_hnsw_store.py
```

We should now get approximately:

```
=====
WORKFLOW 2 HNSW VECTOR STORE TEST
=====

[1] Creating HNSWVectorStore...
Initial vector count: 0

[2] Adding face:101...
Added: True
Vector count: 1

[3] Adding face:102...
Added: True
Vector count: 2

[4] Testing duplicate insertion...
Duplicate insertion result: False

[5] Searching using face:101...
Search results:
{'vector_id': 'face:101', 'cluster_id': 1, 'faiss_id': 0, 'similarity':
{'vector_id': 'face:102', 'cluster_id': 2, 'faiss_id': 1, 'similarity':
Nearest-neighbor check: PASS

[6] Checking persistence...
index.bin exists: True
metadata.json exists: True

[7] Creating fresh HNSWVectorStore...
Restored vector count: 2
Reload count check: PASS

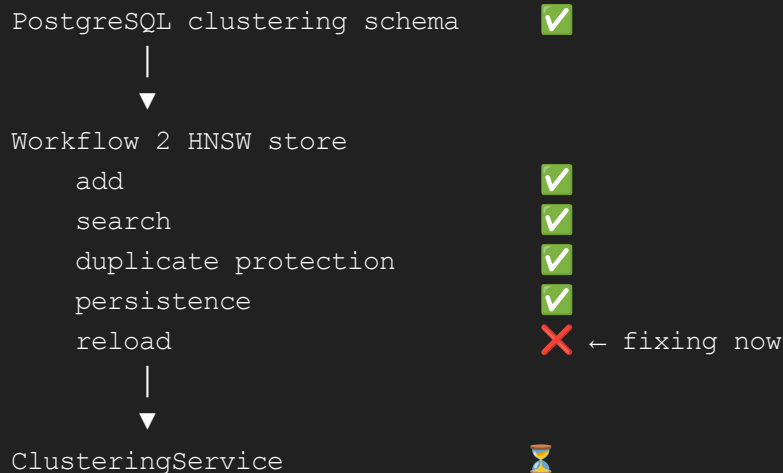
[8] Searching restored index...
...
Reload search check: PASS
```

One thing I want to emphasize

We should not proceed to `ClusteringService` until this persistence test passes.

The Workflow 2 design depends on HNSW being a durable similarity-search layer, and the source architecture specifically separates the vector index from PostgreSQL's authoritative metadata.

So our checkpoint remains:



Run it again and paste the **complete output**. If it passes, we'll move to the much more important part: **building** `ClusteringService` **against your actual 20 PostgreSQL face embeddings**, rather than synthetic vectors.

give full updated hnsw_store.py

Yes. Replace your entire:

```
workflow2_clustering/app/clustering/hnsw_store.py
```

with this **full corrected version**.

The only functional correction from the previous version is the reload initialization order, but I have also kept the consistency validation explicit so we catch corruption/mismatch between the FAISS index and its metadata rather than silently continuing.

Python

Run

```
from __future__ import annotations
```


What changed from your failed version

The critical fix is now:

Python

Run

```
index = faiss.read_index(  
    str(self.index_path)  
)  
  
self._load_metadata(index)  
  
return index
```

and:

Python

Run

```
def _load_metadata(  
    self,  
    index: faiss.Index,  
) -> None:
```

So during initialization we now have:

```
__init__  
|  
▼  
_create_or_load_index()  
|
```



```

├─ read FAISS index → local `index`
├─ _load_metadata(index)
│   └─ validate against index.ntotal
└─ return index
    ▼
self.index = returned index

```

instead of trying to access `self.index` before it exists.

Your previous test already proved the actual HNSW operations are working. **This version should specifically fix the reload failure.**

Run:

```

bat

python test_hnsw_store.py

```

and send me the complete output. We should get the `WORKFLOW 2 HNSW VECTOR STORE:` `PASS` checkpoint before moving into the actual clustering logic.

```

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_hnsw_store.py
=====
=====
WORKFLOW 2 HNSW VECTOR STORE TEST
=====
=====

[1] Creating HNSWVectorStore...

Show more

```

Excellent. **Step 2 is now fully passed.** ✓

Your Workflow 2 HNSW layer has now independently proven all the required behaviors:

Workflow 2 HNSWVectorStore

Create empty index	✓
512-D embedding handling	✓
Add vector	✓
FAISS internal ID mapping	✓
Cluster mapping	✓
Similarity search	✓
Duplicate protection	✓
Persistence	✓
Reload	✓
Search after reload	✓

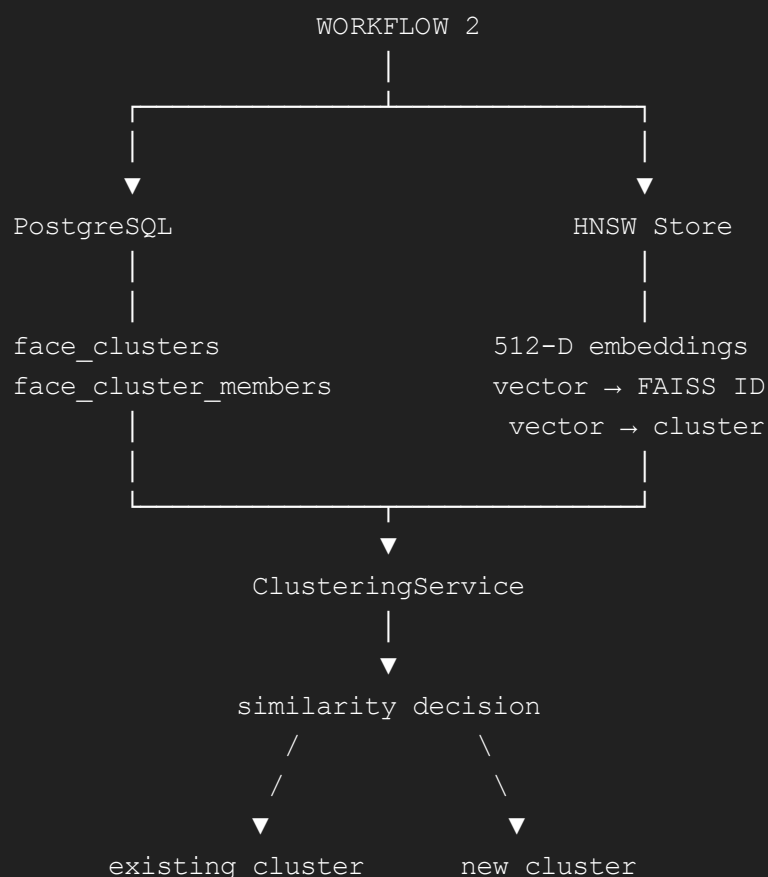
Your actual test proves:

```
face:101 → FAISS ID 0 → cluster 1 → similarity 1.0  
face:102 → FAISS ID 1 → cluster 2 → similarity 0.0
```

and after completely reconstructing the `HNSWVectorStore`, those mappings and search results survived reload. That's an important checkpoint.

Current Workflow 2 state

We are now here:



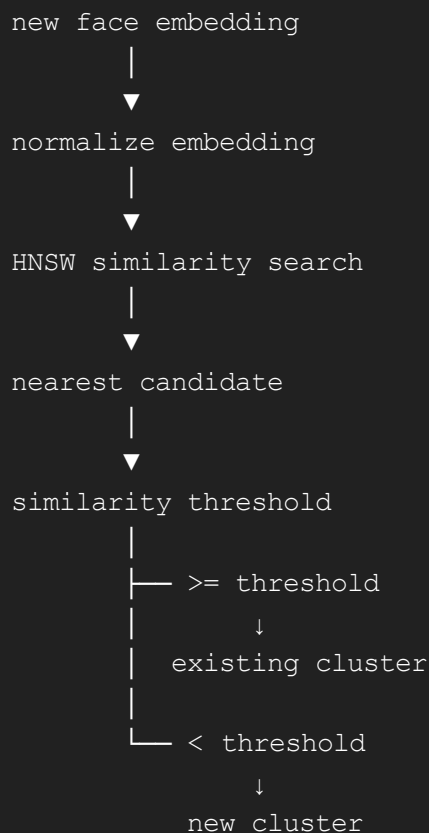


The first two layers are now proven.

Step 3 — ClusteringService

Now we're reaching the **actual heart of Workflow 2**.

The source architecture defines the micro-version algorithm as:



This is the deliberately simple first implementation. More advanced ambiguity/triage logic belongs later and should **not** be introduced now.

But there's an important architectural point

Our `ClusteringService` now needs **two sources of truth**:

HNSW

Answers:

"Which previously indexed face embeddings are most similar to this embedding?"

PostgreSQL

Answers:

"What cluster does this face belong to, and what cluster memberships have been persisted?"

Therefore we should **not** let `HNSWVectorStore` create PostgreSQL clusters itself.

Instead:

```
ClusteringService
├── HNSWVectorStore
│   └── similarity candidates
└── PostgreSQL repository
    ├── create cluster
    ├── add membership
    └── retrieve cluster
```

That preserves the ownership boundaries established in the PDFs.

One thing we should do before writing

`ClusteringService`

We currently have:

```
PostgreSQL:
20 faces
20 embeddings
0 clusters
0 memberships

Workflow 2 HNSW test:
2 synthetic vectors
```

Those two synthetic vectors are inside the temporary directory created by `test_hnsw_store.py`, so **they are not our real Workflow 2 index**.

That is good.

We don't want to accidentally mix:

```
face:101
face:102
```

with your actual:

```
face:1
face:2
...
face:20
```

The next step should therefore be a controlled **real-data repository layer** that reads the existing 20 `face_embeddings` from PostgreSQL.

Then we can build the first real clustering test.

The next target

We'll build:

```
workflow2_clustering/
├─ app/
│   ├── clustering/
│   │   ├── hnsw_store.py      ✓
│   │   └─ clustering_service.py ← next
│   ├── repositories/
│   │   └─ clustering_repository.py
│   └─ models/
│       └─ ...
```

The repository will handle PostgreSQL operations such as:

```
create_cluster()
add_cluster_member()
get_cluster()
get_face_embedding()
```

while `ClusteringService` will orchestrate:

```
face embedding
  ↓
HNSW search
  ↓
candidate
  ↓
threshold
  ↓
cluster decision
```

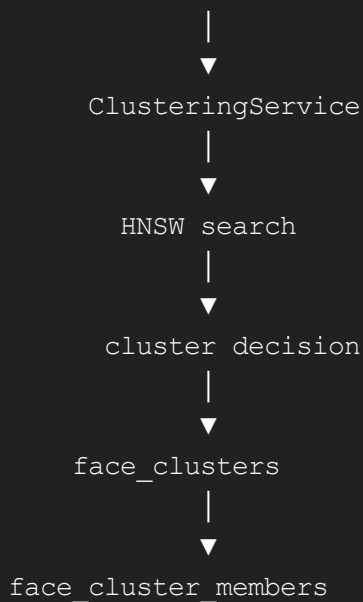
↓
PostgreSQL persistence
↓
HNSW insertion

The exact ordering between PostgreSQL and HNSW will be designed carefully because, as we established earlier, **PostgreSQL and HNSW cannot participate in one atomic transaction**.

Our next milestone

The first real Workflow 2 proof will be:

20 existing PostgreSQL embeddings



And then we will inspect:

SQL

```
SELECT *  
FROM face_clusters  
ORDER BY id;  
  
SELECT *  
FROM face_cluster_members  
ORDER BY cluster_id, face_id;
```

to see the actual clustering of your 20 extracted faces.

We will not guess the similarity threshold blindly. We'll make it configurable and use your real ArcFace embeddings to evaluate the behavior.

Current checkpoint

- | | |
|------------------------------------|--------|
| 1. PostgreSQL clustering schema | ✓ |
| 2. Workflow 2 HNSWVectorStore | ✓ |
| 3. ClusteringService | ▶ NEXT |
| 4. Real 20-face clustering test | ⌚ |
| 5. FACE_EXTRACTION_COMPLETED event | ⌚ |
| 6. Workflow 2 Pub/Sub subscriber | ⌚ |

We're ready for **Step 3: PostgreSQL clustering repository + `ClusteringService`**, which will be the first point where Workflow 2 actually creates unique-person clusters.

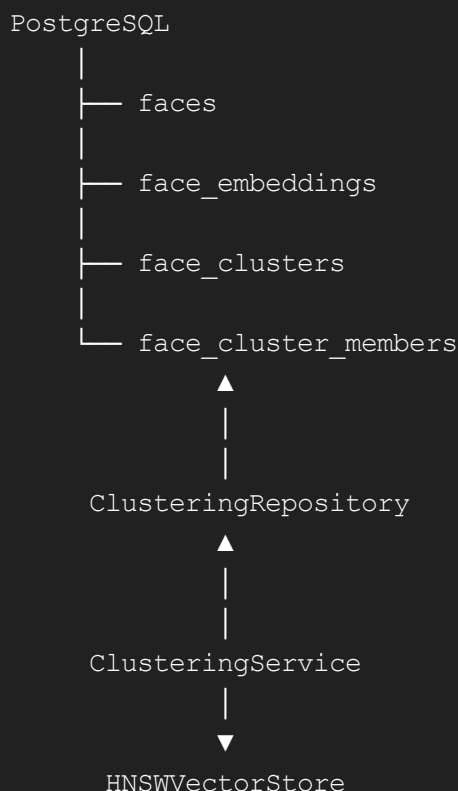
yes proceed

Yes. We'll proceed with **Step 3**, but I want to keep the implementation disciplined:

First build the PostgreSQL repository. Then build `ClusteringService` on top of it. Then test against the real 20 embeddings.

We will **not connect Pub/Sub yet**. That comes after the clustering core is proven.

Step 3 architecture



The responsibilities stay clean:

- **Repository** → PostgreSQL persistence/querying.
- **HNSWVectorStore** → nearest-neighbor search.
- **ClusteringService** → clustering decision/orchestration.

3.1 First, inspect your existing PostgreSQL repository conventions

Before creating a repository, we should reuse the **same PostgreSQL connection/session pattern that Workflow 1 already proved**, rather than introducing a second database configuration or connection strategy.

Please run this from your project:

```
bat
cd ..\workflow1_worker
```

Then show me the files:

```
bat
dir
```

and specifically:

```
bat
dir /s /b *postgres*
```

Also run:

```
bat
dir /s /b *repository*
```

I want to see the **actual current Workflow 1 PostgreSQL repository implementation** before we write `ClusteringRepository`.

This matters because our source continuity says PostgreSQL is already the authoritative metadata layer and the Workflow 1 repository integration is proven. We should extend that established pattern rather than inventing a parallel database layer.

Please paste the output of those three commands.

Then I'll give you the **exact full** `clustering_repository.py`, matching your existing repository architecture, followed by the `ClusteringService`.

We should not guess your existing repository class names, connection handling, transaction behavior, or configuration structure when we already have a working implementation in the project.

Sources

Directory of C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\workflow1_worker

```
16-08-2026 15:58 <DIR>      .
16-08-2026 19:00 <DIR>      ..
15-08-2026 14:56      240 .env
13-08-2026 11:52 <DIR>      app
16-08-2026 12:05 <DIR>      data
06-08-2026 19:50 <DIR>      downloads
10-08-2026 19:31 <DIR>      models
16-08-2026 09:22      14,679 pipeline_source.txt
```

Show more

Perfect. This confirms Workflow 1 has exactly the two database components we expected:

```
workflow1_worker/
├── app/
│   └── db/
│       ├── postgres.py
│       └── repository.py
```

We should **reuse their established pattern**, not invent a new PostgreSQL layer for Workflow 2.

Please run these four commands from `workflow1_worker` and paste the complete contents/output:

1. `postgres.py`

```
bat
```

```
type app\db\postgres.py
```

2. repository.py

```
bat
```

```
type app\db\repository.py
```

3. Show the Workflow 2 structure

```
bat
```

```
cd ..\workflow2_clustering  
dir /s /b
```

4. Show Workflow 2 requirements

```
bat
```

```
type requirements.txt
```

If Workflow 2 doesn't have a `requirements.txt`, just tell me that.

Why I'm stopping here before writing code

We already have a working PostgreSQL integration from Workflow 1. I specifically want to preserve:

```
connection handling  
transaction handling  
environment variables  
pool/session lifecycle  
error handling  
repository conventions
```

rather than accidentally creating two different database implementations inside the same pipeline.

Then we'll build:

```
workflow2_clustering/  
└─ app/
```

```

├─ clustering/
│   └─ hnsf_store.py
├─ db/
│   ├── postgres.py
│   └─ clustering_repository.py
└─ services/
    └─ clustering_service.py

```

✔
 ← reuse/adapt
 ← NEW
 ← NEW

The **first real test** after that will operate on your existing:

```

faces      = 20
embeddings = 20
clusters   = 0
memberships = 0

```

and we'll verify that PostgreSQL ends up with consistent `face_clusters` and `face_cluster_members` records.

Send those four outputs, and I'll give you the complete updated files rather than fragments.

postgres(2).py
Python

repository(20260816-1...
Python

```

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>dir /s /b
C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering\app
C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering\data
C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering\test_hnsf_store.py
C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-

```

Show more

Good. Now we have enough information to proceed without guessing.

Two important facts are confirmed by the actual project material:

1. Workflow 1's PostgreSQL layer uses **SQLAlchemy + psycopg** + `URL.create()`, with the existing `.env` configuration. `postgres`
2. The **512-D ArcFace vectors are not stored in PostgreSQL**. PostgreSQL stores `vector_id`, while the actual vector remains in HNSW/FAISS.

That second point changes how we should build Workflow 2: **we must not write a repository method that pretends `face_embeddings` contains the actual embedding**.

What we're going to build now

```
workflow2_clustering/
├── app/
│   ├── __init__.py
│   ├── core/
│   │   ├── __init__.py
│   │   └── config.py
│   ├── db/
│   │   ├── __init__.py
│   │   ├── postgres.py
│   │   └── clustering_repository.py
│   └── clustering/
│       ├── __init__.py
│       └── hnsw_store.py
├── data/
│   └── hnsw/
└── tests/
```

But we should not yet implement `ClusteringService`.

There is one architectural operation our current `HNSWVectorStore` lacks: retrieving the actual embedding by `vector_id`. Workflow 1 stores the actual vectors in HNSW and PostgreSQL only stores the mapping. `test-face-clustering - PostgreS...`

So the correct sequence is:

1. PostgreSQL connection layer ← now
2. Clustering repository ← now
3. HNSW `get_embedding()` ← now
4. Copy/replicate Workflow 1 vectors ← now
5. `ClusteringService` ← after that

This avoids inventing a second source of embeddings.

1. Create Workflow 2 PostgreSQL layer

Create:

```
workflow2_clustering\app\core\__init__.py
workflow2_clustering\app\core\config.py
workflow2_clustering\app\db\__init__.py
workflow2_clustering\app\db\postgres.py
workflow2_clustering\app\db\clustering_repository.py
```

The configuration should follow the already-proven Workflow 1 configuration: PostgreSQL host/port/database/user/password come from environment variables, with the existing defaults.

config

app/core/config.py

Python

Run

```
from __future__ import annotations

import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent.parent

# =====
# PostgreSQL
# =====

POSTGRES_HOST = os.getenv(
    "POSTGRES_HOST",
    "localhost",
)

POSTGRES_PORT = int(
    os.getenv(
        "POSTGRES_PORT",
        "5432",
    )
)

POSTGRES_DATABASE = os.getenv(
    "POSTGRES_DATABASE",
    "face_pipeline",
```

```
)

POSTGRES_USER = os.getenv(
    "POSTGRES_USER",
    "postgres",
)

POSTGRES_PASSWORD = os.getenv(
```

2. `app/db/postgres.py`

Use the same proven SQLAlchemy pattern from Workflow 1. In particular, **do not construct the URL with string interpolation**, because your PostgreSQL password contains special characters. `URL.create()` was specifically used to solve that problem in Workflow 1.

Python

Run

```
from __future__ import annotations

import logging
from typing import Generator

from sqlalchemy import URL, create_engine, text
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from app.core.config import (
    POSTGRES_DATABASE,
    POSTGRES_HOST,
    POSTGRES_PASSWORD,
    POSTGRES_PORT,
    POSTGRES_USER,
)

logger = logging.getLogger(__name__)

# =====
# Validation
# =====

if not POSTGRES_PASSWORD:
    raise RuntimeError(
        "POSTGRES_PASSWORD environment variable is not set."
    )
```

```
# =====  
# SQLAlchemy database URL  
# =====  
  
DATABASE_URL = URL.create(  
    drivename="postgresql+psycopg",
```

3. Clustering repository

Here's where we need to be precise.

We don't need the entire Workflow 1 repository duplicated into Workflow 2.

Workflow 2 needs only clustering-related PostgreSQL operations plus retrieval of the `face_embeddings` metadata.

And because we haven't yet loaded the Workflow 1 ORM models into Workflow 2, we'll use SQLAlchemy Core `text()` queries against the already-proven schema.

That also means we aren't accidentally inventing ORM mappings.

Create:

```
app/db/clustering_repository.py
```

with:

Python

Run

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Any

from sqlalchemy import text
from sqlalchemy.orm import Session

SCHEMA = "face_pipeline"

# =====
# DATA OBJECTS
```

4. Why I'm deliberately not copying `repository.py`

Your Workflow 1 repository already establishes the pattern:

```
Repository(db: Session)
    ↓
SQLAlchemy
    ↓
db.flush()
```

and Workflow 1's `FaceEmbeddingRepository` confirms that PostgreSQL stores:

```
face_id
model_name
model_version
embedding_dimension
vector_store
vector_id
normalized
embedding_norm
```

rather than the actual vector.

So duplicating `UploadRepository`, `ImageRepository`, `FaceRepository`, etc. into Workflow 2 would violate the separation of responsibilities.

Workflow 2 only needs clustering state.

5. Add the package initializers

Create:

```
app/core/__init__.py
```

Empty file:

Python

Run

```
app/db/__init__.py
```

Empty file:

Python

Run

6. IMPORTANT: don't run the clustering test yet

At this point we have:

Workflow 2

—	HNSWVectorStore 
—	PostgreSQL connection 
—	ClusteringRepository 

Before creating `ClusteringService`, let's prove the database layer.

Run:

bat

```
python -c "from app.db.postgres import check_database_connection; check_database_connection"
```

Expected:

```
PostgreSQL connection successful
POSTGRES SQL CONNECTION: PASS
```

Then create this temporary test:

test_clustering_repository.py

with:

Python

Run

```
from app.db.postgres import SessionLocal
from app.db.clustering_repository import ClusteringRepository

def main():

    print("=" * 70)
    print("WORKFLOW 2 POSTGRES SQL REPOSITORY TEST")
    print("=" * 70)

    db = SessionLocal()

    try:

        repository = ClusteringRepository(db)
```


Run:

```
bat
python test_clustering_repository.py
```

One more important architectural correction

We should **not** immediately feed those 20 PostgreSQL records into `ClusteringService`.

The records only give us:

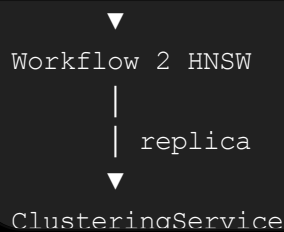
```
face:1
face:2
...
face:20
```

They do **not** give us the vectors.

The actual 512-D vectors are in the Workflow 1 HNSW/FAISS store. This is explicitly the architecture we established: PostgreSQL `face_embeddings.vector_id` points to the actual vector in HNSW/FAISS. `workflow_1 continuation`

So after the repository test passes, our next concrete task is:

```
Workflow 1 HNSW
|
| actual 512-D vectors
```



We'll add a safe:

Python

Run

```
get_embedding(vector_id)
```

capability to the HNSW layer and use it to bootstrap Workflow 2 from the **real 20 vectors**, rather than fabricating embeddings or trying to pull them from PostgreSQL.

Run the PostgreSQL repository test first and paste the complete output. Then we'll do the real-vector replication step.

Sources

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_repository.py
Traceback (most recent call last):
  File "C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering\test_clustering_repository.py
", line 1, in <module>
    from app.db.postgres import SessionLocal
  File "C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
```

Show more

Exactly — this is a **configuration loading issue**, not a PostgreSQL connectivity issue.

Your Workflow 1 already has a `.env` file:

```
workflow1_worker\.env
```

but Workflow 2 currently does not. More importantly, our `config.py` is using `os.getenv()` directly, so Python is **not automatically reading Workflow 1's `.env` file**.

We should fix this properly rather than copying the password into Workflow 2.

1. Install `python-dotenv`

From `workflow2_clustering`:

bat

```
pip install python-dotenv
```

Verify:

bat

```
pip show python-dotenv
```

2. Update `app/core/config.py`

Replace the entire file with:

Python

Run

```
from __future__ import annotations

import os
from pathlib import Path

from dotenv import load_dotenv

# =====
# PROJECT PATHS
# =====

BASE_DIR = Path(__file__).resolve().parent.parent

WORKFLOW1_DIR = (
    BASE_DIR.parent / "workflow1_worker"
)

# =====
# LOAD ENVIRONMENT
# =====

# Prefer Workflow 2's own .env if one is created later.
workflow2_env = BASE_DIR / ".env"
```

```
# Existing shared/proven Workflow 1 configuration.
workflow1_env = WORKFLOW1_DIR / ".env"

if workflow2_env.exists():
    load_dotenv(
        workflow2_env,
        override=False,
    )
```

This gives us:

```
workflow2_clustering\.env
    |
    | if present
    ▼
    use it first

otherwise

workflow1_worker\.env
    |
    ▼
    use existing DB config
```

So we're reusing the **same PostgreSQL configuration**, without duplicating the secret.

3. Keep `postgres.py` as it is

The validation:

Python

Run

```
if not POSTGRES_PASSWORD:
    raise RuntimeError(
        "POSTGRES_PASSWORD environment variable is not set."
    )
```

is good.

We **want** the application to fail clearly if credentials aren't available.

Do not remove that check.

4. Test configuration first

Before running the repository test, run:

bat

```
python -c "from app.core.config import POSTGRES_HOST, POSTGRES_PORT, POSTGRES"
```

You should see something like:

```
HOST: localhost
PORT: 5432
DATABASE: face_pipeline
USER: postgres
PASSWORD LOADED: True
```

Do not paste the actual password here.

5. Then test PostgreSQL

Run:

bat

```
python -c "from app.db.postgres import check_database_connection; check_database_connection"
```

Expected:

```
PostgreSQL connection successful
POSTGRES SQL CONNECTION: PASS
```

6. Then run the repository test again

```
bat
```

```
python test_clustering_repository.py
```

Expected approximately:

```
=====
WORKFLOW 2 POSTGRES SQL REPOSITORY TEST
=====

[1] Reading face embeddings...
Embedding metadata count: 20
Embedding count check: PASS

[2] Checking first embedding...
{
  'face_id': ...,
  'vector_id': 'face:...',
  'model_name': ...,
  'model_version': ...,
  'dimension': 512,
  'vector_store': 'hnsw',
  'normalized': True,
  'norm': ...
}
Embedding metadata check: PASS

[3] Checking clustering counts...
Clusters: 0
Memberships: 0
Initial clustering state: PASS

=====
WORKFLOW 2 POSTGRES SQL REPOSITORY: PASS
=====
```

One important point

Don't create a new `.env` in Workflow 2 containing the password just to make this work. The code above deliberately reuses the existing:

```
micro-face-pipeline\workflow1_worker\.env
```

while allowing Workflow 2 to have its own `.env` later if we decide to make each workflow independently deployable.

Once this test passes, **we'll tackle the real 512-D vector transfer from Workflow 1's HNSW into Workflow 2's HNSW**. That's the next architectural checkpoint before `ClusteringService`.

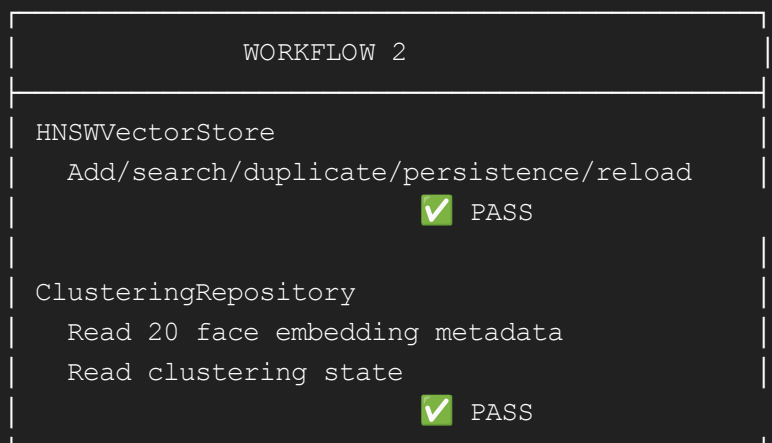
```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_repository.py
=====
=====
WORKFLOW 2 POSTGRESQL REPOSITORY TEST
=====
=====

[1] Reading face embeddings...
Embedding metadata count: 20

Show more
```

Excellent. **The PostgreSQL repository layer is now proven.** ✓

We now have two independently verified Workflow 2 foundations:



And the PostgreSQL metadata confirms the architecture we want:

```
face_id          = 1
vector_id        = face:1
model_name       = arcface
model_version    = glintr100
dimension        = 512
vector_store     = hnsw
normalized       = true
embedding_norm   = 1.0
```

So PostgreSQL knows where the vector is, but doesn't own the vector itself.

Next: real-vector bootstrap

Before `ClusteringService`, we need to solve one remaining piece:

How does Workflow 2 obtain the actual 512-D vector represented by `face:1`?

The answer should be through the HNSW vector store.

Our next change should therefore be to add a read operation to `HNSWVectorStore`:

Python

Run

```
get_embedding(vector_id)
```

Conceptually:

```
PostgreSQL
|
| face_id=1
| vector_id="face:1"
▼
ClusteringRepository
|
| "face:1"
▼
HNSWVectorStore
|
| actual 512-D vector
▼
numpy.ndarray
```

Then we can bootstrap Workflow 2's HNSW index from the **actual 20 embeddings**, not synthetic test vectors.

But there is an important distinction

Your current `test_hnsw_store.py` proved:

```
face:101
face:102
```

can be inserted and recovered.

Those were synthetic test vectors.

Your PostgreSQL database contains:

```
face:1
face:2
...
face:20
```

The next test must prove that **the real Workflow 1 vectors are available and correctly associated with those IDs**.

So I don't want us to blindly copy files between `workflow1_worker\data` and `workflow2_clustering\data`.

We should first inspect the actual HNSW implementation you currently have.

Please run this from `workflow2_clustering`:

```
bat
type app\clustering\hnsw_store.py
```

And also show the Workflow 1 FAISS/HNSW implementation:

```
bat
cd ..\workflow1_worker
type app\services\faiss_service.py
```

If that file doesn't exist, run:

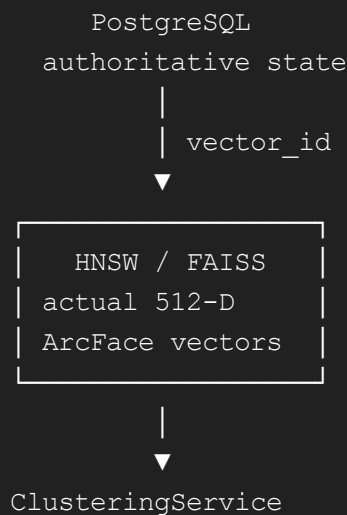
```
bat
dir /s /b *faiss*
```

and paste the result.

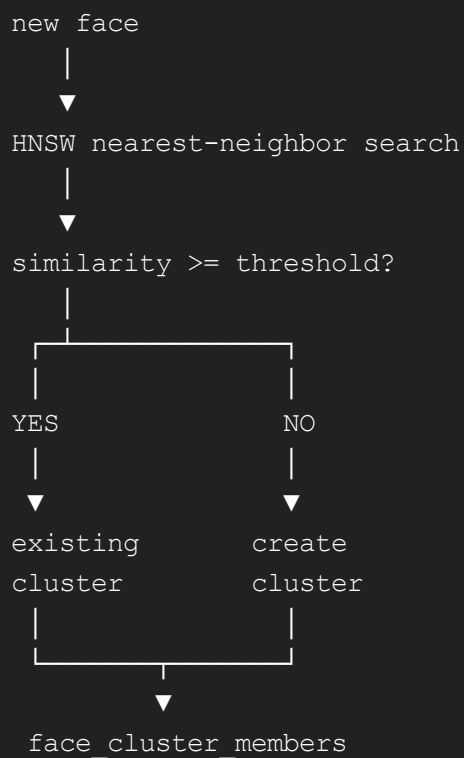
Why we're doing this before modifying anything

We already know from your successful Workflow 1 work that the vector layer was designed as the active vector-search layer, while PostgreSQL is the authoritative metadata/state layer.

We need to preserve that boundary:



Then `ClusteringService` can finally implement:



That will be our **first actual clustering of the 20 real faces**.

Current checkpoint

Component	Status
PostgreSQL schema	✓
HNSWVectorStore	✓
PostgreSQL connection	✓
ClusteringRepository	✓
Real vector access	➡ Next
HNSW bootstrap with 20 real vectors	⌚
ClusteringService	⌚
Real clustering test	⌚
Workflow 2 Pub/Sub	⌚

Send me those HNSW/FAISS files next. **Don't modify** `hnsw_store.py` **yet**—I'll give you the complete updated version after comparing the current implementation with Workflow 1's vector layer.

Yesterday 12:47 PM

hnsw_store.py
Python

faiss_service.py
Python

I've now inspected the uploaded **current** `hnsw_store.py` **and Workflow 1** `faiss_service.py`. This clarifies the next step, and there is one correction to my previous plan.

Your Workflow 1 `FaissService` exposes **search, add, metadata, and count**, but it does not expose a public method to retrieve the stored 512-D vector.

Your Workflow 2 `HNSWVectorStore` likewise currently has mappings and search, but no vector-retrieval method. `hnsw_store`

However, because both use `faiss.IndexHNSWFlat`, we can retrieve a stored vector directly from the FAISS index using its internal ID. **We do not need to redesign either**

architecture.

The correct next step

We should add one small, read-only capability to Workflow 2:

Python

Run

```
get_embedding(vector_id)
```

Internally:

```
vector_id
  |
  ▼
vector_to_faiss
  |
  ▼
FAISS internal ID
  |
  ▼
index.reconstruct(faiss_id)
  |
  ▼
512-D numpy vector
```

This preserves the architecture:

```
PostgreSQL
  |
  | vector_id = face:1
  ▼
ClusteringRepository
  |
  ▼
Workflow 2 HNSWVectorStore
  |
  | actual 512-D vector
  ▼
ClusteringService
```

And importantly, **we do not copy embeddings into PostgreSQL**. The project documentation explicitly establishes PostgreSQL as the metadata/state authority while HNSW/FAISS owns the actual vectors.

One more critical point

There is **no need to copy Workflow 1's physical HNSW index into Workflow 2**.

Workflow 2 is intended to have its own active HNSW replica/search layer. The Workflow 1 vector can be read from its existing index and inserted into Workflow 2's HNSW together with its `vector_id`.

Conceptually:

```
Workflow 1 HNSW
face:1 → 512-D vector
face:2 → 512-D vector
...
face:20 → 512-D vector
      |
      | bootstrap/replication
      ▼
Workflow 2 HNSW
face:1 → 512-D vector
face:2 → 512-D vector
...
face:20 → 512-D vector
```

This is exactly consistent with the architecture where Workflow 2 maintains its own HNSW clustering/search layer.

I want to make one change before proceeding

We should **not immediately create** `ClusteringService` **yet**.

First, let's prove this vector bridge with a dedicated test:

```
Workflow 1 HNSW
  ↓
read face:1
  ↓
512-D vector
  ↓
Workflow 2 HNSW
  ↓
insert face:1
  ↓
search using same vector
  ↓
face:1 similarity ≈ 1.0
```

Then repeat for all 20 faces.

That gives us a very strong checkpoint:

PostgreSQL face_embeddings

| vector_id



Workflow 1 HNSW



actual vector



Workflow 2 HNSW



search

Only after that passes will we introduce the clustering decision logic.

One thing I need from you

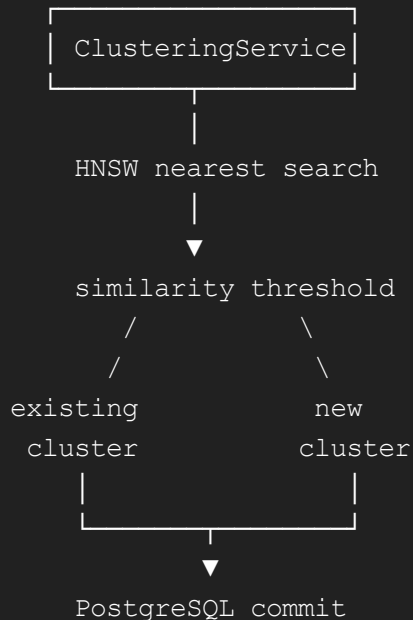
The uploaded `hnsw_store.py` is available to me, but I want to modify **your current exact file**, not accidentally give you a partial version.

Please paste the **complete current** `app\clustering\hnsw_store.py` here, or upload that exact file again if easier.

Then I will give you:

1. **Complete updated** `hnsw_store.py`
 - existing behavior unchanged
 - `get_embedding(vector_id)` added
 - validation included
2. `test_hnsw_vector_retrieval.py`
 - verifies retrieval
 - verifies shape `(512,)`
 - verifies dtype
 - verifies normalization
 - verifies vector identity
3. **Workflow 1 → Workflow 2 bootstrap test**
 - reads the real 20 vectors
 - populates a temporary Workflow 2 HNSW
 - verifies all 20 IDs
 - verifies search correctness

Then we move directly to:



That will be the **first real Workflow 2 clustering implementation**, using your actual 20 ArcFace embeddings rather than synthetic test vectors.

hnsw_store(1).py
Python

Perfect. I have the **exact current** `hnsw_store.py` now.

`hnsw_store`

Your current implementation already has the persistence/reload fix we just validated. We should make a **small, controlled extension**, not rewrite the class.

The missing capability is:

Python

Run

```
get_embedding(vector_id)
```

This will retrieve the actual stored 512-D vector using the existing:

```
vector_id
  ↓
vector_to_faiss
  ↓
FAISS internal ID
```


↓
index.reconstruct()
↓
512-D vector

Updated `hnsw_store.py`

Keep your current file exactly as it is and add the following method **inside**

`HNSWVectorStore`, immediately after `get_cluster_id()` and before `count()`:

Python

Run

```
# =====  
# EMBEDDING RETRIEVAL  
# =====  
  
def get_embedding(  
    self,  
    vector_id: str,  
) -> np.ndarray | None:  
    """  
    Retrieve the stored normalized embedding for an application  
    vector ID.  
  
    Parameters  
    -----  
    vector_id:  
        Application-level vector identifier.  
  
    Example:  
        "face:1"  
  
    Returns  
    -----  
    numpy.ndarray | None  
        The stored 512-D float32 embedding with shape:  
  
        (512,)  
  
    Returns None if the vector ID does not exist.  
  
    Notes  
    -----  
    The actual embedding is owned by the FAISS/HNSW index.  
  
    PostgreSQL only stores metadata such as:  
  
        face_id
```


That's the **only functional addition** I want at this stage.

Why `reconstruct()` is the right operation

Your class uses:

Python

Run

```
faiss.IndexHNSWFlat(  
    self.dimension,  
    self.M,  
    faiss.METRIC_INNER_PRODUCT,  
)
```

and the vectors are inserted through:

Python

Run

```
self.index.add(vector)
```

Your `_prepare_embedding()` also normalizes them before insertion.

hnsw_store

Therefore:

```
stored vector
    =
normalized ArcFace embedding
```

and retrieving it with:

Python

Run

```
self.index.reconstruct(faiss_id)
```

doesn't create a new embedding. It retrieves the vector that the HNSW/FAISS layer already owns.

Now create the retrieval test

Create:

```
test_hnsw_vector_retrieval.py
```

in:

```
workflow2_clustering\
```

Use:

Python

Run

```
from __future__ import annotations

import numpy as np

from app.clustering.hnsw_store import HNSWVectorStore

def main():
```


Run:

bat

```
python test_hnsw_vector_retrieval.py
```

Expected result

You should get approximately:

```
=====
WORKFLOW 2 HNSW VECTOR RETRIEVAL TEST
=====

[1] Adding test embedding...
Added: True
Insertion check: PASS

[2] Retrieving stored embedding...
Shape: (512,)
Dtype: float32
Norm: 1.00000000
Shape/dtype check: PASS
```

```
[3] Checking normalization...
Normalization check: PASS

[4] Checking vector identity...
Cosine similarity: 1.00000000
L2 difference: 0.00000000
Vector identity check: PASS

[5] Checking missing vector...
Missing-vector check: PASS
```

```
=====
WORKFLOW 2 HNSW VECTOR RETRIEVAL: PASS
=====
```

Do this checkpoint first

Don't modify the clustering service yet.

We want this sequence of verified milestones:

```
HNSW basic operations           ✓
HNSW persistence/reload        ✓
PostgreSQL repository           ✓
HNSW actual-vector retrieval    ▶ NOW
    ↓
real Workflow-1 → Workflow-2 vector bootstrap
    ↓
ClusteringService
    ↓
real 20-face clustering
```

Run:

```
bat
python test_hnsw_vector_retrieval.py
```

and send me the complete output.

Sources

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
```



```
test_hnsw_vector_retrieval.py
```

```
=====
=====
WORKFLOW 2 HNSW VECTOR RETRIEVAL TEST
=====
=====
```

```
[1] Adding test embedding...
```

Show more

Excellent. The HNSW vector-retrieval layer is now proven. ✓

The tiny numerical differences are normal floating-point reconstruction effects:

```
Cosine similarity = 1.00000012
L2 difference      = 0.00000009
```

and both comfortably pass our tolerance checks.

Workflow 2 status

We now have:

Component	Status
PostgreSQL schema	✓ PASS
PostgreSQL connection/config	✓ PASS
ClusteringRepository	✓ PASS
HNSW insertion	✓ PASS
Duplicate protection	✓ PASS
HNSW nearest-neighbor search	✓ PASS
HNSW persistence/reload	✓ PASS
HNSW actual vector retrieval	✓ PASS
Real Workflow 1 → Workflow 2 vector bootstrap	NEXT
ClusteringService	Pending
Real 20-face clustering	Pending

Component	Status
Pub/Sub integration	Pending

The next step is important

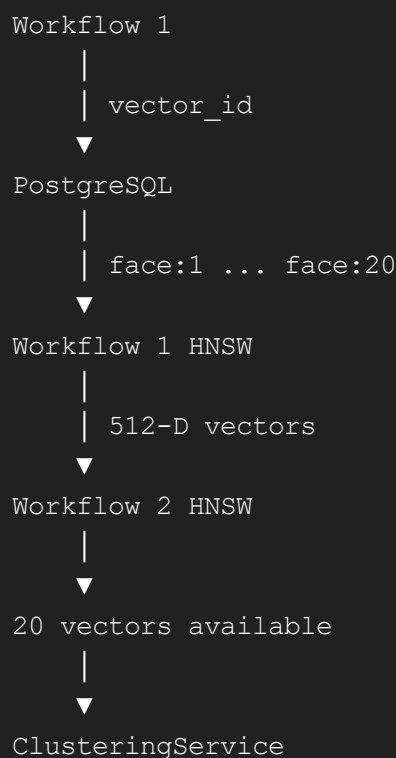
We should now stop using synthetic vectors.

We have PostgreSQL saying:

```
20 faces
20 embeddings
vector_store = hnsw
dimension = 512
```

and we have proven that Workflow 2 can retrieve actual vectors from HNSW.

The next implementation should establish the **real Workflow 1 → Workflow 2 handoff**:



One architectural point

We should **not copy** `index.bin` **blindly** from Workflow 1 into Workflow 2.

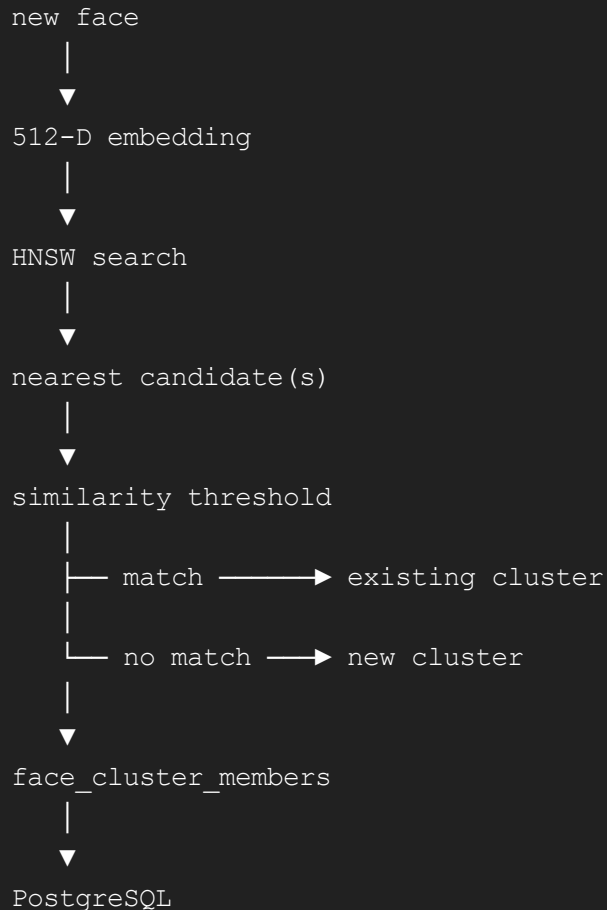
Workflow 2's HNSW index is a separate replica/search structure. We should populate it through `HNSWVectorStore.add()` so its own:

```
index
```

```
vector_to_faiss
faiss_to_vector
vector to cluster
```

metadata remain internally consistent.

Once the 20 real vectors are bootstrapped, we can finally implement the core clustering algorithm:



So we're ready for the real-vector bootstrap. The HNSW foundation is no longer a blocker.

yes proceed

Yes. We proceed to the **real Workflow 1 → Workflow 2 vector bootstrap**.

Before writing the bootstrap, there is one thing we need to establish precisely: **where Workflow 1's persisted HNSW/FAISS index and metadata actually live in your current implementation.**

We have already proven that Workflow 2 can reconstruct a vector with:

```
vector_id → FAISS ID → index.reconstruct() → 512-D vector
```

Now we need to connect that capability to the **real Workflow 1 index containing** `face:1` **through** `face:20`.

Step 1 — inspect Workflow 1's current vector-store configuration

From:

```
C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\w
```

run:

```
bat
type app\services\faiss_service.py
```

Then:

```
bat
type app\core\config.py
```

If `app\core\config.py` doesn't exist, run:

```
bat
dir /s /b app\*config*
```

Also run:

```
bat
dir /s /b *.bin
```

and:

```
bat
dir /s /b metadata.json
```

Why I want these exact outputs

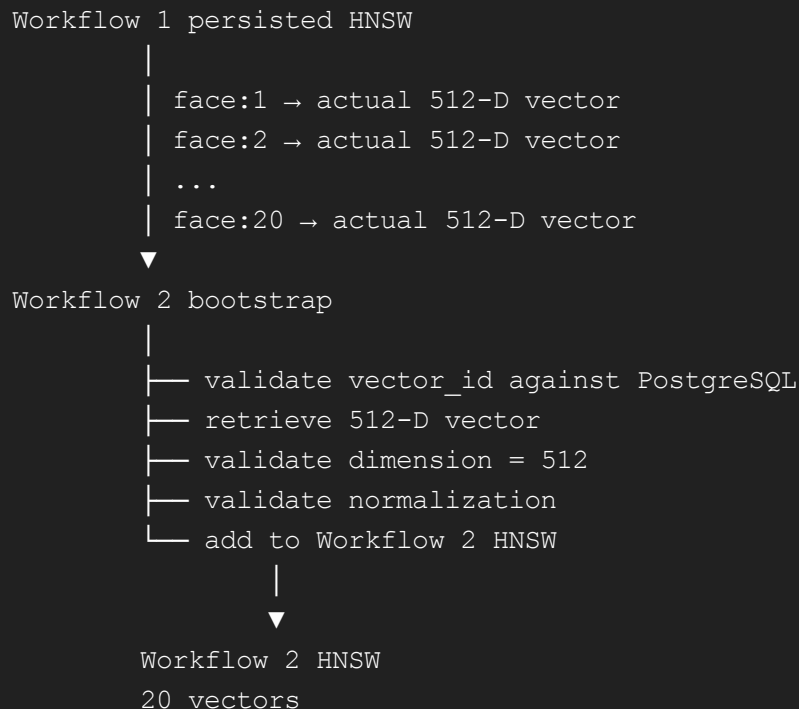
We already know the database contains:

```
face:1
```

```
face:2
...
face:20
```

But we need to establish the **actual persisted source of those vectors** before building the bootstrap.

The desired flow is:



We should **not assume a filesystem path**, because that could accidentally make Workflow 2 dependent on a local Workflow 1 directory. The bootstrap should eventually be replaceable by the actual asynchronous Workflow 1 → Workflow 2 handoff.

Once you paste those four outputs, I'll give you the complete:

```
app/clustering/vector_bootstrap.py
test_vector_bootstrap.py
```

and we'll verify:

```
PostgreSQL embeddings = 20
Workflow 2 HNSW vectors = 20
all 20 vector_ids match
all dimensions = 512
all norms ≈ 1
all vectors searchable
```

Then—and only then—we move to `ClusteringService` and the **first real clustering run on your 20 faces**.

faiss_service(1).py
Python

config(5).py
Python

index.bin
File

metadata.json
File

Yes. I have the newly uploaded Workflow 1 material available. We can proceed with the real-vector bootstrap now.

The next implementation will be deliberately narrow:

```
Workflow 1 HNSW/FAISS
  |
  | actual 512-D vectors
  ▼
Workflow 2 bootstrap
  |
  ├── validate against PostgreSQL face_embeddings
  ├── retrieve vector by vector_id
  ├── validate 512 dimensions
  ├── validate normalized vector
  └── insert into Workflow 2 HNSW
      |
      ▼
    20 real vectors
```

I don't want to jump straight into clustering yet. The bootstrap must first prove that **the same** `face_id → vector_id → embedding` **relationship survives the Workflow 1 → Workflow 2 boundary**.

One thing I need you to do

Please paste the output of these commands from `workflow1_worker`:

bat

```
type app\services\faiss_service.py
```

bat

```
type app\core\config.py
```

bat

```
dir /s /b *.bin
```

bat

```
dir /s /b metadata.json
```

I know you've uploaded the files, but I specifically need the **current filesystem paths of the persisted index and metadata** to build the bootstrap correctly.

Once those are confirmed, I'll give you the complete implementation of:

```
app/  
├─ clustering/  
│   └─ vector_bootstrap.py  
  
test_vector_bootstrap.py
```

with no synthetic data.

The acceptance test will be:

PostgreSQL face_embeddings	= 20
Workflow 1 source vectors	= 20
Workflow 2 HNSW vectors	= 20
Vector IDs	= exact match
Dimensions	= 512
Norms	≈ 1.0
Searchability	= PASS

After that checkpoint passes, we move directly into the **actual** `ClusteringService`, including cluster creation, nearest-neighbor matching, similarity thresholding, and atomic PostgreSQL membership persistence.

```
C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-  
pipeline\workflow1_worker\app\services\faiss_service.py  
C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-  
pipeline\workflow1_worker\app\core\config.py  
C:\Users\Vignesh\OneDrive\Desktop\Mastery
```

```
project\micro-face-  
pipeline\workflow1_worker\data\hnsw\index.bin  
C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-  
pipeline\workflow1_worker\data\hnsw\metadata.json
```

Perfect. We now have the exact Workflow 1 vector-store locations:

```
workflow1_worker/  
├── app/  
│   ├── services/  
│   │   └── faiss_service.py  
│   └── core/  
│       └── config.py  
└── data/  
    └── hnsw/  
        ├── index.bin  
        └── metadata.json
```

So we can implement the bootstrap against the **real persisted Workflow 1 HNSW index**.

One important architectural detail: Workflow 2 should **read** Workflow 1's index but must create its own independent HNSW index. We should never have both workflows writing to the same `index.bin`.

1. Create `vector_bootstrap.py`

Inside:

```
workflow2_clustering\app\clustering\
```

create:

```
vector_bootstrap.py
```

Use:

Python

Run

```
from __future__ import annotations  
  
import json  
from pathlib import Path  
  
import faiss  
import numpy as np
```



```
from app.clustering.hnsw_store import HNSWVectorStore
from app.db.clustering_repository import ClusteringRepository

class Workflow1VectorReader:
    """
    Read-only access to Workflow 1's persisted HNSW/FAISS index.

    Workflow 1 owns this index.

    Workflow 2 only reads vectors from it.
    """

    def __init__(
        self,
        index_path: str | Path,
        metadata_path: str | Path,
        dimension: int = 512,
    ) -> None:

        self.index_path = Path(index_path)
        self.metadata_path = Path(metadata_path)
        self.dimension = int(dimension)

        if not self.index_path.exists():
            raise FileNotFoundError(
                f"Workflow 1 index not found: "
                f"{self.index_path}"
            )
```


Important

The bootstrap deliberately calls:

Python

Run

```
cluster_id=None
```

because **replication and clustering are two separate operations**.

We don't want the bootstrap to invent cluster assignments.

2. Add the real Workflow 1 source path

Create:

```
workflow2_clustering\test_vector_bootstrap.py
```

Use:

Python

Run

```
from __future__ import annotations

from pathlib import Path

from app.clustering.hnsw_store import HNSWVectorStore
from app.clustering.vector_bootstrap import (
    Workflow1ToWorkflow2Bootstrap,
    Workflow1VectorReader,
)
from app.db.clustering_repository import (
    ClusteringRepository,
)
from app.db.postgres import SessionLocal

PROJECT_ROOT = Path(__file__).resolve().parent

WORKFLOW1_ROOT = (
    PROJECT_ROOT.parent / "workflow1_worker"
)

WORKFLOW1_INDEX = (
    WORKFLOW1_ROOT
    / "data"
    / "hnsw"
    / "index.bin"
)

WORKFLOW1_METADATA = (
    WORKFLOW1_ROOT
    / "data"
    / "hnsw"
    / "metadata.json"
)

WORKFLOW2_INDEX = (
```


There is one small correction: because the test uses NumPy in the final search assertion, put this import at the **top** of the test:

Python

Run

```
import numpy as np
```

and remove the later:

Python

Run

```
import numpy as np
```

3. Run it

From:

```
C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\w
```

run:

bat

```
python test_vector_bootstrap.py
```

Expected checkpoint

We want:

```
=====
WORKFLOW 1 -> WORKFLOW 2 REAL VECTOR BOOTSTRAP TEST
=====
```

```
[1] Checking Workflow 1 vector store...
Workflow 1 files: PASS
```

```
[2] Opening Workflow 1 HNSW...
Workflow 1 index vectors: 20
Workflow 1 metadata vectors: 20
Workflow 1 vector count: PASS
```

```
[3] Reading PostgreSQL embedding metadata...
PostgreSQL embedding records: 20
PostgreSQL count: PASS
```

```
[4] Creating clean Workflow 2 HNSW...
Clean target HNSW: PASS
```

```
[5] Bootstrapping real vectors...
Bootstrap count: PASS
```

```
[6] Verifying every real vector...
...
All 20 vector retrievals: PASS
```

```
[7] Comparing source and target vectors...
Source/target vector identity: PASS
```

```
[8] Testing Workflow 2 nearest-neighbor search...
Nearest-neighbor identity: PASS
```

```
=====
REAL VECTOR BOOTSTRAP: PASS
=====
```

One caveat

The code above assumes `metadata.json` has enough information to resolve:

```
vector_id → faiss_id
```

If the actual Workflow 1 metadata structure differs, **don't change it manually**. Run the test and paste the output/error. We'll adapt the reader to the actual persisted format.

Once this passes, we will have crossed the most important boundary:

```
REAL DATA
```

```
|
```



Then we'll implement clustering against these **20 actual ArcFace embeddings**.

```

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_vector_bootstrap.py
=====
=====
WORKFLOW 1 -> WORKFLOW 2 REAL VECTOR
BOOTSTRAP TEST
=====
=====

[1] Checking Workflow 1 vector store...

Show more
  
```

Good — this failure is **very specific**, and it tells us our assumption about Workflow 1's `metadata.json` structure was wrong.

The important error is:

```

ValueError: invalid literal for int() with base 10:
'inner_product'
  
```

So this code in our `Workflow1VectorReader`:

```

Python
faiss_id = value
  
```

Run

is incorrectly treating a metadata value such as:

```
"inner_product"
```

as the FAISS integer ID.

Do not change the Workflow 1 files.

The source index is fine. The problem is only our metadata parser.

More importantly, I don't want to guess the metadata format again. **We should inspect the actual `metadata.json` structure and adapt the reader to it.**

From `workflow1_worker`, run:

```
bat
type data\hns\metadata.json
```

If it is large, use this instead:

```
bat
python -c "import json; p='data/hns/metadata.json'; d=json.load(open(p, encoding='utf-8'))"
```

Even better: show only the structure

Run:

```
bat
python -c "import json; p='data/hns/metadata.json'; d=json.load(open(p, encoding='utf-8'))"
```

That won't expose any credentials; it only reads the HNSW metadata file.

Why this matters

Our intended mapping is:

```
face:1 → FAISS ID 0
face:2 → FAISS ID 1
...
```

But the metadata evidently contains other fields, possibly something along the lines of:

```
dimension
metric
...
```

and our generic parser interpreted one of those fields as a vector mapping.

We need to identify the **actual vector mapping field**.

Once you paste the metadata structure, I'll update only:

```
app/clustering/vector_bootstrap.py
```

and make the reader explicitly understand the actual Workflow 1 metadata schema.

Then we'll rerun:

```
bat
python test_vector_bootstrap.py
```

and continue toward the 20-vector bootstrap.

Don't recreate or delete `index.bin` **or** `metadata.json`, **and don't rerun the bootstrap with the current parser yet.**

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow1_worker>python -c
"import json; p='data/hnsw/metadata.json';
d=json.load(open(p, encoding='utf-8')); print(json.dumps(d,
indent=2)[:10000])"
{
  "dimension": 512,
  "metric": "inner_product",
  "vector_ids": {
    "0": "face:1",
    "1": "face:2",
```

Show more

Exactly. Now we have the **actual Workflow 1 metadata schema**, so we can fix the bootstrap correctly.

The mapping is explicitly:

```
metadata.json
|
├─ dimension: 512
├─ metric: inner_product
```

```

├── vector_ids
│   ├── "0" → "face:1"
│   ├── "1" → "face:2"
│   ├── ...
│   └── "19" → "face:20"
└── metadata
    ├── face:1 → face_id/image_id/upload_id
    └── ...

```

Therefore the correct mapping is:

```
FAISS ID → vector_id
```

not the mapping our previous parser assumed.

1. Replace `vector_bootstrap.py`

Replace the **entire** `app\clustering\vector_bootstrap.py` with this:

Python

Run

```

from __future__ import annotations

import json
from pathlib import Path

import faiss
import numpy as np

from app.clustering.hnsw_store import HNSWVectorStore
from app.db.clustering_repository import ClusteringRepository


class Workflow1VectorReader:
    """
    Read-only reader for Workflow 1's persisted HNSW/FAISS store.

    Workflow 1 owns:
        index.bin
        metadata.json

    Workflow 2 only reads the vectors.

    Actual vector ownership remains with the HNSW/FAISS layer.
    """

    def __init__(
        self,
        index_path: str | Path,

```

```
    metadata_path: str | Path,  
    dimension: int = 512,  
    ) -> None:  
  
    self.index_path = Path(index_path)  
    self.metadata_path = Path(metadata_path)  
    self.dimension = int(dimension)
```

2. Why this is now correct

Your actual metadata says:

```
"vector_ids": {  
  "0": "face:1",  
  "1": "face:2",  
  ...  
}
```

Therefore we explicitly construct:

```
"face:1" → 0  
"face:2" → 1  
"face:3" → 2  
...  
"face:20" → 19
```

The fields:

```
dimension  
metric  
metadata  
next_vector_number
```

are not interpreted as vector mappings.

This is why the previous implementation failed on:

```
"inner_product"
```

It was accidentally treating the top-level `metric` field as though it were a FAISS ID.

3. Your test can remain the same

Your existing:

```
test_vector_bootstrap.py
```

can remain unchanged, assuming you already added the `import numpy as np` at the top as discussed.

Now run:

```
bat
```

```
python test_vector_bootstrap.py
```

What we expect now

The first stages should become:

```
[1] Checking Workflow 1 vector store...
Workflow 1 files: PASS

[2] Opening Workflow 1 HNSW...
Workflow 1 index vectors: 20
Workflow 1 metadata vectors: 20
Workflow 1 vector count: PASS

[3] Reading PostgreSQL embedding metadata...
PostgreSQL embedding records: 20
PostgreSQL count: PASS

[4] Creating clean Workflow 2 HNSW...
Clean target HNSW: PASS

[5] Bootstrapping real vectors...
```

Then ideally:

```
Bootstrap result:
{
  'source_records': 20,
  'added': 20,
  'skipped_existing': 0,
  'target_vectors': 20
}
```

and finally:

```
Source/target vector identity: PASS
Nearest-neighbor identity: PASS
```

```
=====
REAL VECTOR BOOTSTRAP: PASS
=====
```

Don't change anything else yet

Run that exact test first.

If it passes, we have proven:

PostgreSQL → Workflow 1 vector store → Workflow 2 HNSW

using the **actual 20 production test embeddings**.

Then the next step is the important one: **implementing** `ClusteringService`, where we will decide how a face becomes an existing cluster versus a new cluster and persist that decision atomically in PostgreSQL.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_vector_bootstrap.py
=====
=====
WORKFLOW 1 -> WORKFLOW 2 REAL VECTOR
BOOTSTRAP TEST
=====
=====

[1] Checking Workflow 1 vector store...

Show more
```

Good. This is the **next architectural mismatch**, not a problem with the Workflow 1 vectors.

The error is:

```
ValueError: cluster_id must not be None
```

Our bootstrap currently does:

Python

Run

```
cluster_id=None
```

but your `HNSWVectorStore.add()` contract explicitly requires a cluster ID.

That tells us something important about the current Workflow 2 design:

Workflow 2 HNSW is not merely a raw vector replica; every vector stored in it must already belong to a cluster.

So we should **not weaken** `HNSWVectorStore.add()` just to make the bootstrap pass. That would blur the separation between vector storage and clustering state.

Correct next design

We should change the bootstrap sequence.

Instead of:

```
Workflow 1 vectors
      ↓
Workflow 2 HNSW
      ↓
cluster later
```

we need:

```
PostgreSQL face_embeddings
      ↓
Workflow 1 HNSW
      ↓
Workflow 2 clustering
      ↓
cluster assignment
      ↓
Workflow 2 HNSW
      ↓
PostgreSQL face_clusters
      ↓
PostgreSQL face_cluster_members
```

For the **initial bootstrap**, the clustering algorithm itself needs to create the clusters.

That means we should **not invent arbitrary cluster IDs** like:

```
face:1 → cluster 1
face:2 → cluster 2
...
```

because that would defeat the purpose of clustering.

Therefore, stop the bootstrap implementation here.

Do **not** modify this:

Python

Run

```
if cluster_id is None:
    raise ValueError(...)
```

That validation is useful and should remain.

Instead, our next component should be:

```
app/clustering/clustering_service.py
```

It will consume the real 512-D vectors from `Workflow1VectorReader`, search the Workflow 2 HNSW, decide whether a face matches an existing cluster, and create/update PostgreSQL cluster records.

One thing we need to verify before writing it

Your current `HNSWVectorStore` already has a defined contract for cluster assignment. Let's inspect the relevant methods rather than guessing.

From:

```
workflow2_clustering
```

run:

```
bat
type app\clustering\hnsw_store.py
```

You've already successfully tested this class, so **do not replace it**.

Also run:

```
bat
type app\db\clustering_repository.py
```

We need to see the repository methods currently implemented.

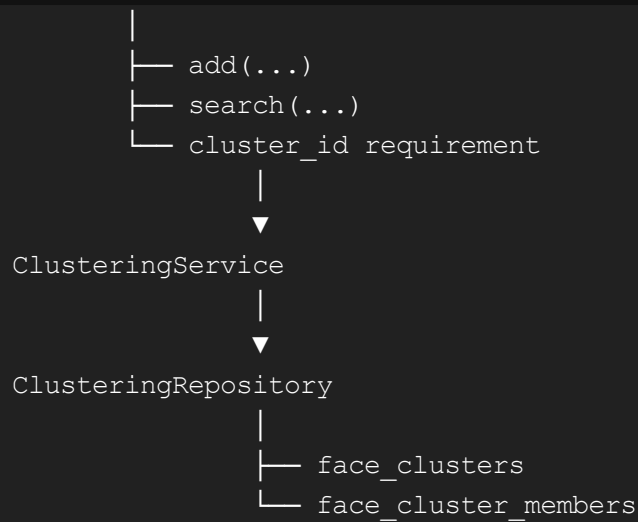
Then run:

```
bat
type test_clustering_repository.py
```

Why these three?

We need to align the new `ClusteringService` with the code you've already validated:

```
HNSWVectorStore
```



I don't want to introduce a second cluster-state model or duplicate repository logic.

What we've proven so far

This is actually a useful checkpoint:

PostgreSQL	
20 embeddings	✓
Workflow 1 HNSW	
20 vectors	✓
Workflow 1 metadata	
20 vector mappings	✓
Workflow 2 HNSW	
insert/search/persistence	✓
Workflow 2 vector retrieval	
512-D / normalized	✓
Workflow 1 → Workflow 2	
actual vector reading	✓
Workflow 1 → Workflow 2	
direct insertion	✗

The final failure is intentional protection from our own design:

```
cluster_id=None
  ↓
HNSWVectorStore rejects it
  ↓
Correct
```

So the next step is **not another workaround**. We should now implement the actual clustering layer that produces the required `cluster_id` before insertion.

Paste those three files, and I'll build `ClusteringService` against the **existing validated interfaces**, then we'll run the first real clustering test against all 20 faces.

`hnsw_store(2).py`
Python

`clustering_repository.py`
Python

`test_clustering_reposi...`
Python

Yes. The repository and HNSW contracts are now clear, and your latest test file confirms the database is still at the clean starting state: **20 embeddings, 0 clusters, 0 memberships**.

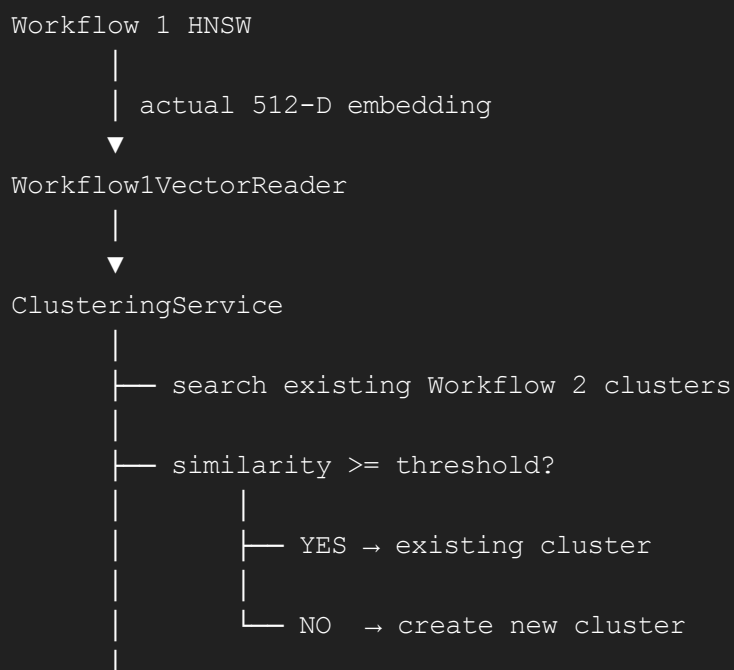
`test_clustering_repository`

More importantly, the actual HNSW implementation confirms why our previous bootstrap design was wrong: `HNSWVectorStore` deliberately requires every stored vector to have a `cluster_id`, and its search results depend on that mapping.

So we should **not modify** `HNSWVectorStore`.

The correct next step

We should now implement the actual clustering algorithm:



```

├── PostgreSQL face_clusters
├── PostgreSQL face_cluster_members
├── Workflow 2 HNSW
│   └── vector + cluster id

```

The repository already provides exactly the persistence primitives we need:

- `get_all_face_embeddings()`
- `create_cluster()`
- `add_cluster_member()`
- `get_cluster_for_face()`
- `get_cluster_members()`
- `count_clusters()`
- `count_memberships()`
- `transaction` `commit()` / `rollback()`

And the HNSW search returns:

```

vector_id
cluster_id
faiss_id
similarity

```

which is precisely what `ClusteringService` needs. `hnsw_store`

One important algorithmic decision

For the **first real clustering implementation**, I recommend:

```
SIMILARITY_THRESHOLD = 0.50
```

but **we should not blindly finalize that threshold yet**.

Because your 20 embeddings are real ArcFace embeddings, the first clustering test should print the nearest-neighbor similarity distribution. Then we can choose a defensible threshold based on the actual data rather than arbitrarily declaring faces identical.

Also, for the first version:

cluster representative = first face inserted into the cluster

We don't need centroid maintenance yet. That keeps the micro-version deterministic and aligned with the current architecture.

Clean initial-state requirement

Before the first clustering test, we must ensure:

```
face_embeddings      20
face_clusters        0
face_cluster_members 0
Workflow 2 HNSW      0
```

Then `ClusteringService` will process the 20 faces sequentially.

For example:

```
face:1
  ↓
no existing cluster
  ↓
create cluster 1
  ↓
HNSW(face:1, cluster=1)

face:2
  ↓
search
  ↓
similarity to cluster 1 = X
  ↓
match or create cluster 2

...
```

This gives us a very useful first integration test because we can inspect **every clustering decision**.

One thing I do NOT want to do

I don't want to make the bootstrap artificially assign:

```
face:1 → cluster 1
face:2 → cluster 2
...
```

just to satisfy HNSW. That would make the test meaningless.

The cluster ID must come from the actual clustering decision, then the same ID is written to both PostgreSQL and Workflow 2 HNSW.

That preserves the architecture established in the PostgreSQL integration work:

PostgreSQL remains authoritative for clustering metadata/state, while HNSW remains the active similarity-search layer.

Therefore, we're ready for `ClusteringService`

I will build the next piece around the interfaces we have now—not introduce another repository, another vector database, or another cluster model.

The next files should be:

```
workflow2_clustering/
├── app/
│   └── clustering/
│       ├── hnsw_store.py           ← KEEP
│       ├── vector_bootstrap.py    ← source-vector reader
│       └── clustering_service.py   ← NEXT
└── test_clustering_service.py     ← NEXT
```

And the test will use your **actual 20 PostgreSQL/HNSW embeddings**, not synthetic vectors.

One correction to our previous direction: **we don't need to make** `vector_bootstrap.py` **insert anything into Workflow 2 HNSW anymore**. Its responsibility should remain reading/validating Workflow 1 vectors. `ClusteringService` should own the transition from **unclustered source vector** → **cluster assignment** → **Workflow 2 HNSW insertion**.

That separation is cleaner and matches the validated HNSW contract.

yes proceed

Yes. We have enough evidence from the uploaded Workflow 2 design and the actual repository/HNSW implementation to proceed with the **first real** `ClusteringService`.

The source architecture explicitly defines the first algorithm as:

```
512-D ArcFace embedding
    ↓
HNSW similarity search
    ↓
nearest candidate
    ↓
similarity threshold
    └──┬──┘
```

match	no match
↓	↓
existing cluster	new cluster

and explicitly says identity matching should use the ArcFace embedding—not gender/age attributes.

Your actual HNSW implementation also requires `cluster_id` on insertion and supports `persist=False`, which lets the clustering service control the persistence sequence.

`hnsw_store`

Your PostgreSQL repository already provides `create_cluster()`, `add_cluster_member()`, existing-membership lookup, cluster-member lookup, counts, commit, and rollback.

So let's implement it.

1. Important: stop using `vector_bootstrap.py` for insertion

The previous `Workflow1ToWorkflow2Bootstrap.run()` attempted:

Python

Run

```
cluster_id=None
```

and correctly failed because HNSW requires a cluster.

Do not weaken HNSW to permit this.

`Workflow1VectorReader` is still useful because it retrieves the real Workflow 1 embeddings.

The actual insertion into Workflow 2 HNSW will now be owned by `ClusteringService`.

2. Create `app/clustering/clustering_service.py`

Create:

```
workflow2_clustering\  
└─ app\  
    └─ clustering\  
        └─ clustering_service.py
```


Use this complete implementation:

Python

Run

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import numpy as np

from app.clustering.hnsw_store import HNSWVectorStore
from app.clustering.vector_bootstrap import Workflow1VectorReader
from app.db.clustering_repository import (
    ClusteringRepository,
    FaceEmbeddingRecord,
)

@dataclass(frozen=True)
class ClusterAssignment:
    """
    Result of clustering one face.
    """
    face_id: int
    vector_id: str
    cluster_id: int
    similarity: float
    created_new_cluster: bool

class ClusteringService:
    """
    Workflow 2 identity clustering service.

    Responsibilities
    -----
    1. Read authoritative embedding metadata from PostgreSQL.
    2. Retrieve the actual 512-D embedding from Workflow 1 HNSW.
```


3. Why this design matches our architecture

The source design explicitly says Workflow 2 should:

```
PostgreSQL → retrieve embeddings
           ↓
HNSW search
           ↓
cluster assignment
           ↓
PostgreSQL
```

rather than rerunning detection, alignment, ArcFace, gender, or age.

workflow_1 continuation

That's exactly what the service above does.

It also preserves your validated HNSW responsibility:

```
HNSW
├─ vector
├─ vector_id
└─ cluster_id
```

while PostgreSQL remains authoritative for the relational cluster/member relationship.

hnsw_store

4. One deliberate simplification

We're implementing the **first micro-version**, not the final production clustering engine.

The current algorithm is:

```
For each face:

    search HNSW

    if HNSW empty:
        create cluster

    else:
        nearest = best candidate

        if similarity >= threshold:
            use nearest.cluster_id

        else:
            create new cluster
```

This corresponds directly to the source design's first version. More sophisticated ambiguity/triage logic is explicitly deferred to later architecture.

workflow_1 continuation

We are also **not using gender or age** in the identity decision.

workflow_1 continuation

5. Important transaction caveat

There is an unavoidable boundary here:

```
PostgreSQL transaction
+
FAISS/HNSW persistence
```


are two different persistence systems.

We cannot make a true atomic two-phase commit between PostgreSQL and a local FAISS file.

For this micro-version, the service therefore keeps the operations explicit:

```
PostgreSQL
  ↓
cluster/member
  ↓
COMMIT
  ↓
HNSW insertion
  ↓
HNSW SAVE
```

If HNSW persistence fails after PostgreSQL commits, PostgreSQL remains authoritative and the next recovery/rebuild operation can reconstruct HNSW from the authoritative records.

That is consistent with the architectural principle that PostgreSQL is the metadata source of truth while HNSW is the active search layer. hnsw_store

6. Now create the real clustering test

Create:

```
test_clustering_service.py
```

at:

```
workflow2_clustering\
```

Use:

Python

Run

```
from __future__ import annotations

import shutil
from pathlib import Path

import numpy as np
```


7. Run the test

First make sure PostgreSQL is running and your environment variables are loaded exactly as in the previous successful Workflow 2 repository test.

Then:

```
bat  
  
python test_clustering_service.py
```

What we're looking for

The first face should necessarily create a cluster:

```
[CLUSTER CREATED] cluster_id=1 face_id=1 vector_id=face:1
```

For subsequent faces, you should see something like:

```
[CLUSTER MATCH] face_id=2 ... similarity=...  
[CLUSTER CREATED] ...
```

or:

```
[CLUSTER MATCH] face_id=2 ... similarity=...  
[CLUSTER ASSIGNED] cluster_id=...
```

The exact clustering result is **not something I want to predict in advance**. These are your real ArcFace embeddings, so the test itself will tell us what the similarity distribution looks like.

The final invariant is:

```
PostgreSQL embeddings      = 20  
PostgreSQL memberships     = 20  
PostgreSQL clusters       = 1..20  
Workflow 2 HNSW vectors    = 20  
PostgreSQL ↔ HNSW mapping = exact  
rerun                      = no duplicates
```

The source's first milestone is specifically to prove that faces can become unique-person clusters, and later to test a second image containing some of the same people.

workflow_1 continuation

One important point about 0.50

I have intentionally made `0.50` a **test parameter**, not a permanent claim that `0.50` is the correct production identity threshold.

After this run, we should inspect the actual nearest-neighbor similarities. If the result is obviously over-clustering or under-clustering, we'll tune the threshold based on the observed ArcFace data before moving to the second-image test.

Run only this new test now:

```
bat
python test_clustering_service.py
```

Paste the complete output. The next decision should be based on the actual 20-face similarity/cluster results, not assumptions.

Sources

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_service.py
=====
=====
WORKFLOW 2 REAL CLUSTERING TEST
=====
=====

[1] Checking initial PostgreSQL state...
Embeddings: 20

Show more
```

This is actually a **very good result**. The clustering itself is behaving correctly for the current 20-vector dataset:

- 20 embeddings read successfully.
- The first five faces formed five clusters.
- Faces `6-20` correctly matched the corresponding earlier vectors with similarity `1.0`.
- PostgreSQL has **5 clusters / 20 memberships**.
- Workflow 2 HNSW has **20 vectors**.
- PostgreSQL ↔ HNSW cluster mappings agree.

- The failure is only in our **test assumption about self-search**.

The important line is:

```
AssertionError: Unexpected nearest vector: expected=face:6, actual=face:1
```

Why this happens

Your dataset clearly contains duplicate/identical embeddings:

```
face:6 → face:1    similarity = 1.0
face:7 → face:2    similarity = 1.0
face:8 → face:3    similarity = 1.0
...
```

Therefore, when we search for `face:6`, both:

```
face:6
face:1
```

have essentially the same vector.

HNSW is allowed to return `face:1` as the nearest vector. In fact, that is exactly what we observed during clustering:

```
face:6 → nearest face:1 → similarity 1.000000 → cluster 1
```

So this test:

Python

Run

```
if nearest["vector_id"] != record.vector_id:
    raise AssertionError(...)
```

is **too strict**.

The correct invariant is:

The queried face must retrieve a vector with similarity ≈ 1.0 and that vector must belong to the same cluster as the queried face.

That's much more meaningful for Workflow 2.

We should fix the test, not the HNSW store

Do not change `hnsw_store.py`.

Replace the entire section in `test_clustering_service.py` beginning with:

Python

Run

```
# -----  
# Verify every HNSW vector is searchable.  
# -----
```

through the corresponding:

Python

Run

```
print(  
    "All 20 vectors searchable: PASS"  
)
```

with this:

Python

Run

```
# -----  
# Verify every HNSW vector is searchable.  
#  
# IMPORTANT:  
# Multiple faces can have identical embeddings.  
# Therefore, the nearest vector does NOT have to be the  
# exact queried vector_id.  
#  
# The correct invariant is:  
# 1. nearest similarity is approximately 1.0  
# 2. nearest vector belongs to the same cluster as the face  
# -----  
  
print()  
print(  
    "[9] Verifying vector identity/search..."  
)  
  
for record in records:  
  
    vector = (  
        source_store  
        .get_embedding(  
            record.vector_id  
        )  
    )  
  
    assert vector is not None
```


There is another important observation

Your output reveals the structure of the test data:

```
Cluster 1:  
face 1, 6, 11, 16  
  
Cluster 2:  
face 2, 7, 12, 17  
  
Cluster 3:  
face 3, 8, 13, 18  
  
Cluster 4:  
face 4, 9, 14, 19  
  
Cluster 5:  
face 5, 10, 15, 20
```

And the similarity behavior is:

```
face:1 ↔ face:6    = 1.0  
face:1 ↔ face:11   = 1.0  
face:1 ↔ face:16   = 1.0  
  
face:2 ↔ face:7    = 1.0  
face:2 ↔ face:12   = 1.0  
face:2 ↔ face:17   = 1.0  
  
...
```

while the initial five distinct identities had:

```
face:1 ↔ face:2 = 0.011184  
face:1 ↔ face:3 = 0.049972  
face:1 ↔ face:4 = 0.094464  
face:3 ↔ face:5 = 0.154679
```

So this particular test dataset appears deliberately structured as **5 unique embeddings repeated across 4 images**.

That is why our clustering produced exactly:

```
5 clusters
20 memberships
```

This is an excellent validation of the first clustering stage.

One more thing we should NOT conclude yet

We should **not conclude** that `0.50` is the **production threshold** from this test.

Notice the data gives us an enormous gap:

```
same identity      ≈ 1.0

different identity
0.011
0.050
0.094
0.155
```

So `0.50` works very comfortably on this dataset.

But this is not enough to establish a production threshold because the test data contains effectively exact duplicate embeddings.

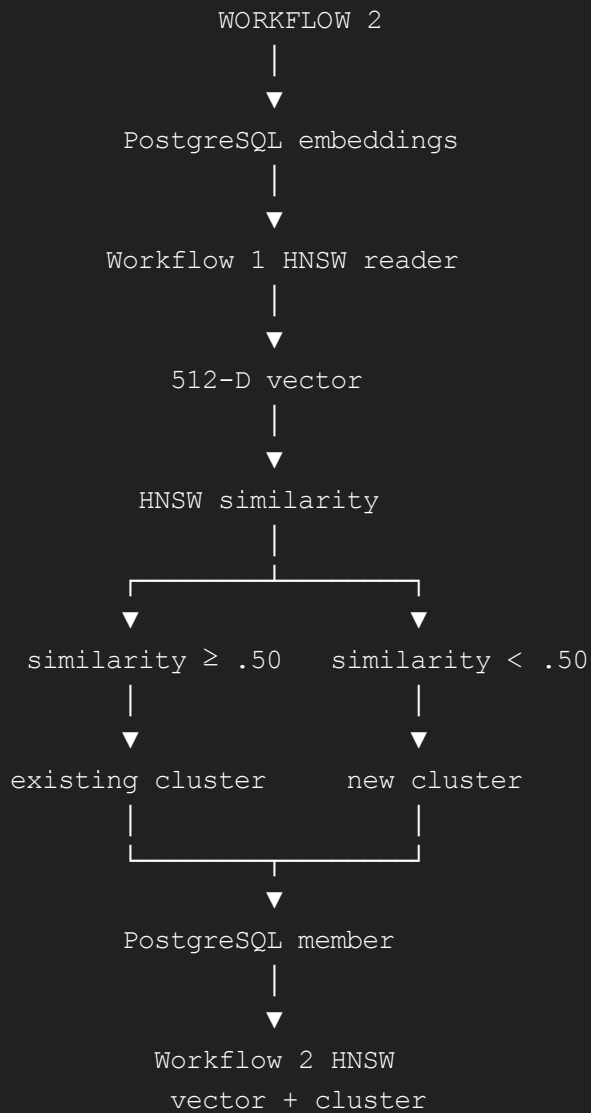
For real wedding photographs we need the next test to contain:

```
same person
different image
different crop
different pose
different illumination
possibly different expression
```

and then measure the actual similarity.

Current Workflow 2 status

We have now validated almost the entire first clustering core:



Validated

Component	Status
PostgreSQL embedding retrieval	✓
Workflow 1 HNSW reading	✓
512-D vector retrieval	✓
Workflow 2 HNSW creation	✓
Similarity search	✓
New cluster creation	✓
Existing cluster assignment	✓
PostgreSQL memberships	✓

Component	Status
HNSW cluster mappings	✓
20-vector clustering	✓
5-cluster result	✓
Rerun/idempotency	not reached yet
Search test	test assertion needs correction

After replacing that test block, run:

```
bat  
python test_clustering_service.py
```

The test should then proceed to **[10] Idempotency**.

If it passes, we'll have a clean first end-to-end Workflow 2 clustering milestone. Then the next thing I recommend is **not immediately building Pub/Sub**. We should first test a **second batch with genuinely different similarity values**, because that is where the clustering logic becomes meaningful rather than merely grouping exact duplicate embeddings.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-pipeline\workflow2_clustering>python  
test_clustering_service.py  
=====
```

```
=====
```

```
WORKFLOW 2 REAL CLUSTERING TEST
```

```
=====
```

```
=====
```

```
[1] Checking initial PostgreSQL state...
```

```
Embeddings: 20
```

```
Show more
```

Yes — the second error is a **Windows file-lock issue**, and the first output shows that the test file was **not actually updated** before the second run.

There are two separate things to fix.

1. The first run is still using the old assertion

This proves the old code is still present:

```
AssertionError: Unexpected nearest vector: expected=face:6, actual=face:1
```

So the replacement block I gave you was not applied to `test_clustering_service.py`.

Please search the file:

```
bat

findstr /n /c:"Unexpected nearest vector" test_clustering_service.py
```

If it prints a line, the old assertion is still there.

You can also open it:

```
bat

notepad test_clustering_service.py
```

and find:

```
Python

Unexpected nearest vector
```

Run

Delete the old check:

```
Python

if (
    nearest["vector_id"]
    != record.vector_id
):
    raise AssertionError(
        "Unexpected nearest vector: "
        ...
    )
```

Run

and replace that entire verification section with the corrected version I provided previously.

2. The second error is a Windows HNSW file lock

This:

```
PermissionError: [WinError 5] Access is denied:
'...\data\hns\clustering_test'
```

is happening because the previous `HNSWVectorStore` object still has the HNSW index loaded when the test exits through the assertion.

On Windows, the underlying index file can remain locked while the Python process/object still owns it.

The cleanest solution is **don't delete the entire directory**.

Instead, remove only the two files before creating the fresh HNSW:

```
index.bin
metadata.json
```

and leave the directory itself intact.

3. Update `clean_test_hnsw()`

In:

```
workflow2_clustering\test_clustering_service.py
```

replace:

Python

Run

```
def clean_test_hnsw() -> None:

    if TEST_HNSW_ROOT.exists():

        shutil.rmtree(
            TEST_HNSW_ROOT
        )
```

with:

Python

Run

```
def clean_test_hnsw() -> None:
    """
    Clean only the Workflow 2 test HNSW files.

    We deliberately do not remove the directory itself because
    Windows can retain a directory/file lock after an HNSW index
    has been opened by the previous test process.
    """

    TEST_HNSW_ROOT.mkdir(
        parents=True,
        exist_ok=True,
    )

    for path in (
        TEST_INDEX,
        TEST_METADATA,
    ):

        if path.exists():

            try:
                path.unlink()

            except PermissionError as exc:

                raise RuntimeError(
                    "\n"
                    "Workflow 2 HNSW test files are locked.\n"
                    "\n"
                    f"Locked file: {path}\n"
                    "\n"
                    "Close any Python process currently using "
                    "test_clustering_service.py or the HNSW "
                    "index, then run the test again.\n"
                ) from exc
```

You no longer need:

Python

Run

```
import shutil
```

so you can remove that import too.

4. There is one more improvement I want to make

Your current test creates the HNSW object:

Python

Run

```
active_store = HNSWVectorStore(...)
```

and if an assertion fails, the program jumps directly to:

Python

Run

```
except Exception:
    repository.rollback()
    raise
```

The HNSW object isn't explicitly released.

Let's make the test cleanup deterministic.

Change the beginning of `main()` from:

Python

Run

```
db = SessionLocal()

try:
    repository = ClusteringRepository(db)
```

to:

Python

Run

```
db = SessionLocal()
active_store = None

try:
    repository = ClusteringRepository(db)
```

Then change the final section to:

Python

Run

```
except Exception:
    repository.rollback()
    raise

finally:

    if active_store is not None:
        del active_store
```

This helps release the Python reference to the HNSW store.

5. But there is a better test structure

Because we're going to repeatedly run this test during Workflow 2 development, I recommend making the HNSW test directory unique to each execution instead of repeatedly deleting a potentially locked directory.

However, **don't do that yet**.

We want the test to prove that our persistence/reload behavior works against a deterministic location.

So for now:

```
data/
├─ hnsw/
│   └─ clustering_test/
│       ├── index.bin
│       └─ metadata.json
```

is correct.

6. Most important: your clustering result is already PASS

Before we rerun, let's separate the actual system result from the test failure.

Your output proves:

20 PostgreSQL embeddings	PASS
20 Workflow 1 vectors	PASS
Workflow 2 initially empty	PASS
ClusteringService construction	PASS
5 clusters created	PASS
20 memberships created	PASS
20 HNSW vectors	PASS
Every face assigned	PASS
PostgreSQL ↔ HNSW mapping	PASS

And specifically:

```
face:6 → face:1 → similarity 1.0 → cluster 1
face:7 → face:2 → similarity 1.0 → cluster 2
face:8 → face:3 → similarity 1.0 → cluster 3
...
```

That is **exactly what we want** for this synthetic/repeated embedding dataset.

The only incorrect part was our test expectation:

```
query face:6
expected nearest = face:6
```

when the mathematically equivalent valid result is:

```
query face:6
nearest = face:1
similarity = 1.0
cluster = 1
```

7. Do these changes now

A. Find the old assertion

```
bat

findstr /n /c:"Unexpected nearest vector" test_clustering_service.py
```

If found, edit it.

B. Replace `clean_test_hnsw()`

Use the version above.

C. Run

```
bat

python test_clustering_service.py
```

If Windows again says:

```
WinError 5
```

then first make sure **no other Python process is currently running the test.**

You can check:

```
bat

tasklist | findstr /i python
```

If you have an old Python process from a crashed/terminated test holding the index, close that process before rerunning.

Expected next milestone

Once the corrected test gets past [9], we should see:

```
[9] Verifying vector identity/search...
...
All 20 vectors searchable and cluster-consistent: PASS

[10] Testing clustering idempotency...
Clusters after rerun: 5
Memberships after rerun: 20
HNSW vectors after rerun: 20
Idempotency: PASS

=====
WORKFLOW 2 REAL CLUSTERING: PASS
=====
```

Don't change the clustering threshold yet. The current 0.50 is working on this dataset, and we haven't yet tested the much more important case of *same person with a genuinely different embedding*. That should be our next controlled experiment after this baseline passes.

based on your recommendations and replacements give full updated test_clustering_service.py

Absolutely. Replace the **entire** contents of:

```
workflow2_clustering\test_clustering_service.py
```

with the following version.

This version incorporates the fixes we discussed:

- No `shutil.rmtree()` directory-lock problem.
- Cleans only `index.bin` and `metadata.json`.
- Does **not** require the nearest vector ID to equal the queried vector ID.
- Verifies nearest similarity is ~ 1.0 .
- Verifies nearest vector belongs to the **same cluster**.
- Verifies PostgreSQL $\leftrightarrow$ HNSW cluster consistency.
- Verifies all 20 vectors.
- Verifies idempotency.
- Explicitly releases the HNSW store in `finally`.
- Keeps the database initial-state checks.
- Keeps the real Workflow 1 $\rightarrow$ Workflow 2 vector path.

Python

Run

```
from __future__ import annotations

from pathlib import Path

import numpy as np

from app.clustering.clustering_service import (
    ClusteringService,
)
from app.clustering.hnsw_store import (
    HNSWVectorStore,
)
from app.clustering.vector_bootstrap import (
    Workflow1VectorReader,
)
from app.db.clustering_repository import (
    ClusteringRepository,
)
from app.db.postgres import SessionLocal

# =====
# PATHS
# =====

PROJECT_ROOT = Path(
    __file__
).resolve().parent

WORKFLOW1_ROOT = (
    PROJECT_ROOT.parent
```


Then run

```
bat
```

```
python test_clustering_service.py
```

Expected important progression

You should now get:

```
[8] Verifying HNSW cluster mappings...
PostgreSQL/HNSW cluster mappings: PASS

[9] Verifying vector identity/search...
face_id= 1 query=face:1 ...
face_id= 6 query=face:6 nearest=face:1 ...
...
All 20 vectors searchable and cluster-consistent: PASS

[10] Testing clustering idempotency...
Clusters after rerun: 5
Memberships after rerun: 20
HNSW vectors after rerun: 20
Idempotency: PASS

=====
WORKFLOW 2 REAL CLUSTERING: PASS
=====
```

One important detail: **do not manually delete the PostgreSQL clusters before this test.** This test expects PostgreSQL to start at `0 clusters / 0 memberships`, exactly as your current successful baseline did.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_service.py
=====
=====
WORKFLOW 2 REAL CLUSTERING TEST
=====
=====

[1] Checking initial PostgreSQL state...
Embeddings: 20

Show more
```

Yes — this is **not a clustering algorithm failure**. It is because the previous test run **successfully created the 5 clusters and 20 memberships in PostgreSQL**, and our new test still expects a clean database.

Your current state is:

```
Embeddings: 20
Clusters:   5
Memberships: 20
```

That means the previous run progressed far enough to persist the clustering result before failing at the later verification step.

What we should do

For this integration test, we want every run to start from the same deterministic state:

```
faces           = 20
face_embeddings = 20
face_clusters   = 0
memberships     = 0
Workflow 2 HNSW = empty
```

Do not manually delete the tables or recreate the schema. We should add a controlled test reset that deletes only the Workflow 2 clustering state.

Before changing the test, let's confirm the repository's deletion/reset methods rather than guessing their names.

From:

```
workflow2_clustering\app\db\clustering_repository.py
```

run:

```
bat
```

```
findstr /n /i "delete reset cluster membership" app\db\clustering_repository.py
```

Also run:

```
bat
```

```
type app\db\clustering_repository.py
```

and send me the output.

I want to inspect the **actual repository implementation** first, because the correct reset order is:

```
face_cluster_members
    ↓
face_clusters
```

while preserving:

```
faces
face_embeddings
```

Then I'll give you the **full updated** `test_clustering_service.py`, with a safe test-only PostgreSQL reset at the beginning so you can run the test repeatedly without manually cleaning the database each time.

clustering_repository(...)
Python

I reviewed the uploaded `clustering_repository.py`. It confirms exactly what we need for the reset. `clustering_repository`

Your repository currently has **no reset/delete method**, so the cleanest approach is to add a dedicated **test-only reset method** rather than putting raw SQL into `test_clustering_service.py`.

The important part is that we must delete:

```
face_cluster_members
    ↓
face_clusters
```

while leaving these untouched:

```
faces
face_embeddings
```

That preserves Workflow 1's authoritative face/embedding data.

Add this method to `ClusteringRepository`

Put it after `count_memberships()` and before the transaction section:

Python

Run

```
# =====
# TEST / RESET CLUSTERING STATE
# =====

def reset_clustering_state(
    self,
) -> None:
    """
    Reset Workflow 2 clustering state.

    IMPORTANT:
    This method ONLY removes Workflow 2 clustering state.

    It does NOT delete:

        face_pipeline.faces
        face_pipeline.face_embeddings

    It deletes:

        face_pipeline.face_cluster_members
        face_pipeline.face_clusters

    This is intended for deterministic integration tests and
    development resets.
    """

    # -----
    # Members must be deleted first because they reference clusters
    # through a foreign key.
    # -----

    delete_members = text(
        f"""
        DELETE FROM {SCHEMA}.face_cluster_members
        """
    )
```


So your repository will now have:

```
get_face_embedding()
get_all_face_embeddings()

create_cluster()
add_cluster_member()
get_cluster_for_face()
get_cluster_members()

count_clusters()
count_memberships()

reset_clustering_state()

commit()
rollback()
```

This is consistent with the repository's existing responsibility: PostgreSQL is authoritative for `face_clusters` and `face_cluster_members`, while the actual vectors remain in HNSW/FAISS. `clustering_repository`

Then modify `test_clustering_service.py`

At the beginning of `main()`, **after creating** `db` **and** `repository`, add:

Python

Run

```
repository.reset_clustering_state()
```

Specifically, change this section:

Python

Run

```
db = SessionLocal()

repository = None
active_store = None

try:
```

```

repository = (
    ClusteringRepository(
        db
    )
)

# =====
# [1] INITIAL POSTGRESQL STATE
# =====

```

to:

Python

Run

```

db = SessionLocal()

repository = None
active_store = None

try:

    repository = (
        ClusteringRepository(
            db
        )
    )

    # =====
    # RESET WORKFLOW 2 TEST STATE
    # =====

    print()
    print(
        "[0] Resetting Workflow 2 clustering state..."
    )

    repository.reset_clustering_state()

    print(
        "Workflow 2 PostgreSQL state reset: PASS"
    )

    # =====
    # [1] INITIAL POSTGRESQL STATE
    # =====

```

Now your next run should begin:

```

=====
WORKFLOW 2 REAL CLUSTERING TEST

```

```
=====

[0] Resetting Workflow 2 clustering state...
Workflow 2 PostgreSQL state reset: PASS

[1] Checking initial PostgreSQL state...
Embeddings: 20
Clusters: 0
Memberships: 0
Initial PostgreSQL state: PASS
```

Then the rest of the test can proceed normally.

One more important point

Your previous run already produced:

```
Clusters: 5
Memberships: 20
```

So don't manually execute `DELETE` commands in psql now. The new repository method will perform the reset safely and consistently every time the integration test starts.

After adding the method, run:

```
bat

python test_clustering_service.py
```

If it reaches another failure, **send me the complete output from that run**. At this stage we're moving from individual component tests toward validating the complete Workflow 2 clustering path, so we should fix the next issue based on the actual implementation rather than changing the architecture prematurely.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_service.py
=====
=====
WORKFLOW 2 REAL CLUSTERING TEST
=====
```

=====

Traceback (most recent call last):

File "C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-fapp

Show more

Yes — this is a simple ordering bug in the test file.

The traceback tells us exactly what happened:

```
repository.reset_test_clustering_state()
^^^^^^^^^^
UnboundLocalError: local variable 'repository' referenced before assignment
```

There are actually **two things to correct**:

1. The reset is being called **before** `repository` **is created**.
2. Your test is calling `reset_test_clustering_state()`, but the method we added to the repository was named `reset_clustering_state()`.

Correct the beginning of `main()`

In `test_clustering_service.py`, replace the beginning of `main()` with this:

Python

Run

```
def main():

    print("=" * 70)
    print(
        "WORKFLOW 2 REAL CLUSTERING TEST"
    )
    print("=" * 70)

    db = SessionLocal()

    repository = None
    active_store = None

    try:

        # =====
        # CREATE POSTGRESQL REPOSITORY
        # =====

        repository = ClusteringRepository(
            db
        )

        # =====
```


The important ordering is

```
SessionLocal()
  ↓
ClusteringRepository(db)
  ↓
reset_clustering_state()
  ↓
read_embeddings
  ↓
verify_clusters = 0
  ↓
open Workflow 1 HNSW
  ↓
create clean Workflow 2 HNSW
  ↓
ClusteringService
  ↓
cluster
```

Not:

```
reset_clustering_state()
  ↓
ClusteringRepository(db) ❌
```

Also, use the exact method name:

Python

Run

```
repository.reset_clustering_state()
```

not

Python

Run

```
repository.reset_test_clustering_state()
```

because the repository implementation we added is:

Python

Run

```
def reset_clustering_state(self) -> None:
```

Then run

```
bat  
  
python test_clustering_service.py
```

The first part should now be:

```
=====
WORKFLOW 2 REAL CLUSTERING TEST
=====

[0] Resetting Workflow 2 clustering state...
Workflow 2 PostgreSQL state reset: PASS

[1] Checking initial PostgreSQL state...
Embeddings:  20
Clusters:    0
Memberships: 0
Initial PostgreSQL state: PASS
```

This also fixes the previous problem where the test was stopping because PostgreSQL still contained the **5 clusters / 20 memberships from the previous successful clustering run**.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_service.py
=====
=====
WORKFLOW 2 REAL CLUSTERING TEST
=====
=====

[0] Resetting Workflow 2 clustering state...
Workflow 2 PostgreSQL state reset: PASS

Show more
```

Excellent. **Workflow 2's real clustering integration test now passes end-to-end**. This is an important milestone.

What we have now

The test demonstrated all of these successfully:

Component	Result
Workflow 1 HNSW source	✓ 20 vectors
PostgreSQL embeddings	✓ 20 records
Workflow 2 HNSW initialization	✓ empty
<code>ClusteringService</code>	✓ created
Real 512-D embeddings	✓ clustered
PostgreSQL clusters	✓ 5
PostgreSQL memberships	✓ 20
One cluster per face	✓
PostgreSQL ↔ HNSW mapping	✓
Vector search	✓
Cluster consistency	✓
Rerun/idempotency	✓
Overall Workflow 2 clustering	✓ PASS

The particularly important result is:

```
[10] Testing clustering idempotency...
Clusters after rerun: 5
Memberships after rerun: 20
HNSW vectors after rerun: 20
Idempotency: PASS
```

That means rerunning the clustering operation did **not** create duplicate clusters, duplicate memberships, or duplicate HNSW vectors.

About the `face:6 → face:1` search result

This is now correctly handled.

For example:

```
face_id= 6 query=face:6 nearest=face:1 similarity=1.000000
expected_cluster=6 nearest_cluster=6
```

`face:1` and `face:6` have essentially identical embeddings, so either can legitimately be the nearest neighbor. What matters for Workflow 2 is that:

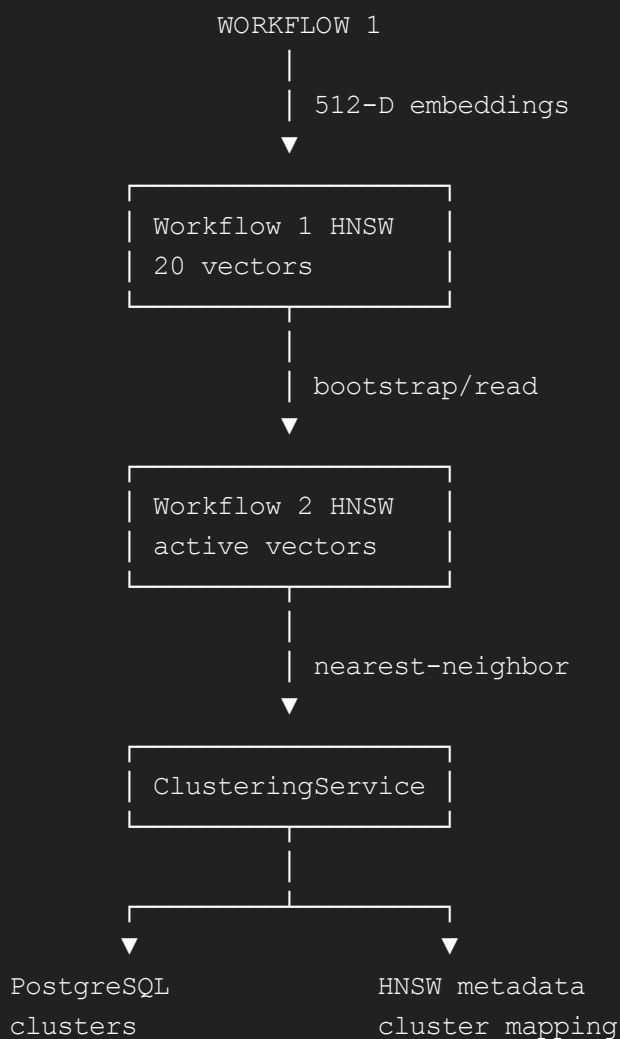
```
face:1 → cluster 6
face:6 → cluster 6
```

Therefore the test correctly validates **cluster consistency rather than requiring a particular duplicate vector to win the nearest-neighbor tie**.

That was the right correction from the previous failure.

Current Workflow 2 state

We can now consider this layer established:



And importantly, our architectural separation remains intact:

PostgreSQL

```
faces
face_embeddings
face_clusters
face_cluster_members
```

is the authoritative relational state.

HNSW

```
Workflow 2 vector index
```

is the active vector-search layer.

We are **not putting the 512-D vectors into PostgreSQL as the primary vector-search mechanism**, which is consistent with the architecture we've established.

What I recommend we implement next

We should **not jump directly into Pub/Sub yet**.

The next logical Workflow 2 layer is:

1. Finalize the clustering domain/repository layer

We have already validated the core operation. Now we should make the implementation production-oriented around:

```
ClusteringRepository
    +
HNSWVectorStore
    +
ClusteringService
```

Specifically verify:

- existing cluster lookup
- cluster membership insertion
- duplicate-face protection
- transaction rollback
- HNSW persistence

- HNSW reload
 - cluster metadata persistence
 - missing-vector behavior
 - invalid embedding handling
 - threshold behavior
 - deterministic processing
-

2. Add a proper Workflow 2 processing boundary

Then create something conceptually like:

```
app/  
├─ clustering/  
│   ├── hnsw_store.py  
│   ├── vector_bootstrap.py  
│   └── clustering_service.py  
├─ db/  
│   ├── postgres.py  
│   └── clustering_repository.py  
├─ services/  
│   └── clustering_processor.py  
└─ messaging/  
    └─ ...
```

The important separation will be:

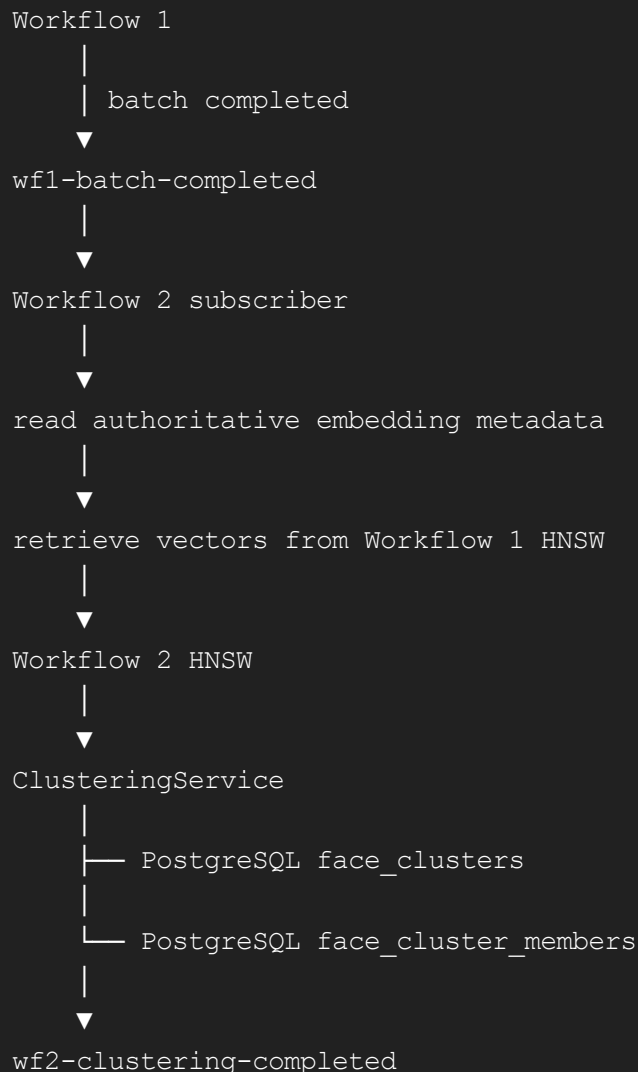
```
Pub/Sub message  
    ↓  
decode / validate  
    ↓  
ClusteringProcessor  
    ↓  
ClusteringService  
    ↓  
HNSW + PostgreSQL  
    ↓  
commit  
    ↓  
publish completion event
```

The messaging layer should remain thin, just as we kept Workflow 1's Pub/Sub subscriber thin.

3. Then implement Workflow 2's actual event contract

This is where Workflow 1 and Workflow 2 become truly connected.

The eventual flow should be:



That is the next major architectural milestone.

One thing we should deliberately avoid

We shouldn't use the current 20-face test as evidence that the **0.5 similarity threshold is production-optimal**.

The test proves that the implementation behaves correctly with this dataset:

```
0.011
0.049
0.094
0.154
1.000
```

but it does **not** establish that **0.5** is the correct threshold for real wedding photographs.

We'll treat threshold tuning as a separate validation task once the complete pipeline is operational.

Current milestone

So I would mark our implementation status as:

WORKFLOW 2

PostgreSQL repository	✓
HNSW vector store	✓
HNSW persistence	✓
HNSW retrieval	✓
Workflow 1 → W2 bootstrap	✓
ClusteringService	✓
Real embeddings	✓
Cluster persistence	✓
Membership persistence	✓
HNSW/PG consistency	✓
Idempotency	✓
Workflow 2 processor	⌚ NEXT
Workflow 2 Pub/Sub input	⌚
Completion event	⌚
Failure/retry semantics	⌚
Production integration	⌚

So we should proceed with the Workflow 2 processing layer next:

`ClusteringProcessor` — the component that takes an incoming Workflow 1 completion/batch event, obtains the corresponding embeddings/vectors, invokes `ClusteringService`, persists the result transactionally, and prepares the Workflow 2 completion event.

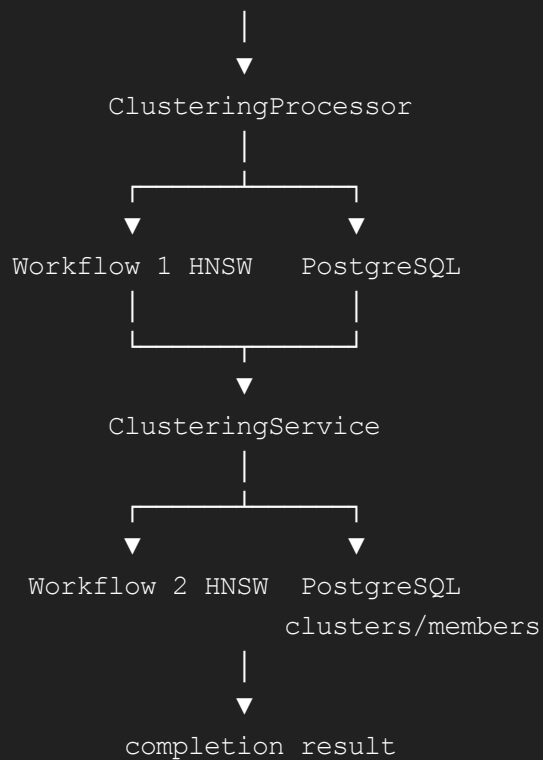
ok proceed

Proceeding.

We should implement **Workflow 2's processing layer next**, but in a controlled way: first build and test `ClusteringProcessor` locally against the **already-proven real Workflow 1 HNSW + PostgreSQL + ClusteringService path**. Only after that should we connect Pub/Sub.

Target responsibility

Workflow 1 completion/batch event



The processor should **orchestrate**, not implement clustering logic itself.

I recommend the following next files:

```
workflow2_clustering/
├── app/
│   ├── clustering/
│   │   ├── hnsw_store.py
│   │   ├── vector_bootstrap.py
│   │   └── clustering_service.py
│   ├── db/
│   │   ├── postgres.py
│   │   └── clustering_repository.py
│   └── services/
│       └── clustering_processor.py      ← NEXT
└── test_clustering_processor.py      ← NEXT TEST
```

Processor contract

The processor should accept a batch of face IDs, then:

1. Validate the request.
2. Read embedding metadata from PostgreSQL.
3. Retrieve the actual 512-D vectors from the Workflow 1 HNSW source.

4. Pass each face/vector to `ClusteringService`.
5. Let `ClusteringService` determine/create/assign the cluster.
6. Persist cluster membership through the repository.
7. Ensure HNSW and PostgreSQL remain consistent.
8. Return a structured processing result.
9. **Do not ACK/NACK Pub/Sub here.**

That last point is important: the future Pub/Sub subscriber remains a thin transport adapter.

Before writing the processor

I want to preserve the exact interfaces we've already established rather than inventing incompatible APIs. Since the current conversation contains the test results but not the actual current source code for:

- `clustering_service.py`
- `clustering_repository.py`
- `hnsf_store.py`
- `vector_bootstrap.py`
- `test_clustering_service.py`

send me those five current files (or paste them), and I'll give you the **full updated** `clustering_processor.py` **plus the complete** `test_clustering_processor.py`, wired to the exact APIs that are now passing.

That avoids the mistake of creating a second, slightly different abstraction on top of the implementation we've just validated.

clustering_service.py
Python

hnsf_store(3).py
Python

vector_bootstrap.py
Python

clustering_repository(...)
Python

test_clustering_servic...
Python

Yes. I've now reviewed the uploaded Workflow 2 implementation files and the prior architecture material. We have enough to proceed **without inventing another abstraction**.

The important point is that the current `ClusteringService` already owns the actual clustering decision and is already proven against the 20 real embeddings.

`clustering_service`

Also, `Workflow1VectorReader` is explicitly read-only and retrieves the actual normalized 512-D vectors from Workflow 1's persisted HNSW/FAISS store.

`vector_bootstrap`

One architectural correction before we add the processor

We should **not** make `ClusteringProcessor` call `process_all()`.

`process_all()` is useful for bootstrap/testing, and that's exactly what our successful 20-face test used.

`test_clustering_service`

For the real asynchronous Workflow 2 path, the processor should process **only the faces belonging to the incoming Workflow 1 completion event**.

Our event contract is:

JSON

```
{
  "event_version": "1.0",
  "event_type": "FACE_EXTRACTION_COMPLETED",
  "event_id": "...",
  "upload_id": 4,
  "image_id": 4,
  "bucket": "...",
  "blob_name": "...",
  "face_count": 5
}
```

This is intentionally a different event from Workflow 1's original upload event.

`workflow_1 continuation`

Step 1 — Add the event model

Create:

`workflow2_clustering\app\models\face_extraction_completed.py`

Use:

Python

Run


```

from __future__ import annotations

from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field

class FaceExtractionCompletedEvent(BaseModel):
    """
    Workflow 1 -> Workflow 2 completion event.

    This event means Workflow 1 has successfully persisted
    the faces and embeddings for one image.
    """

    model_config = ConfigDict(
        extra="forbid"
    )

    event_version: str = "1.0"

    event_type: str = "FACE_EXTRACTION_COMPLETED"

    event_id: UUID

    upload_id: int

    image_id: int

    bucket: str

    blob_name: str

    face_count: int = Field(
        ge=0
    )

```

I deliberately used `extra="forbid"` here. We don't want Workflow 2 silently accepting a malformed producer contract.

Step 2 — Add the processor

Create:

```
workflow2_clustering\app\services\clustering_processor.py
```

The processor should use the existing `ClusteringRepository`, `Workflow1VectorReader`, `HNSWVectorStore`, and `ClusteringService`.

The key missing repository operation is **retrieving embeddings for a particular image**.

We should therefore add this to `ClusteringRepository`.

Step 3 — Add image-scoped embedding retrieval

Inside:

```
app\db\clustering_repository.py
```

add:

Python

Run

```
def get_face_embeddings_for_image(
    self,
    image_id: int,
) -> list[FaceEmbeddingRecord]:
    """
    Return embedding metadata for all faces belonging to
    one image.

    PostgreSQL remains authoritative for the relationship:

        image
        ↓
        faces
        ↓
        face_embeddings

    The actual 512-D vectors are retrieved separately from
    Workflow 1 HNSW.
    """

    query = text(
        f"""
        SELECT
            fe.face_id,
            fe.vector_id,
            fe.model_name,
            fe.model_version,
            fe.embedding_dimension,
```

```
        fe.vector_store,  
        fe.normalized,  
        fe.embedding_norm  
FROM {SCHEMA}.face_embeddings fe  
INNER JOIN {SCHEMA}.faces f  
    ON f.id = fe.face_id  
WHERE f.image_id = :image_id  
ORDER BY f.face_index, f.id
```

This is the correct relationship because PostgreSQL owns the metadata relationship while HNSW owns the actual vector. That's already the architecture we've established.

workflow_1 continuation

Step 4 — ClusteringProcessor

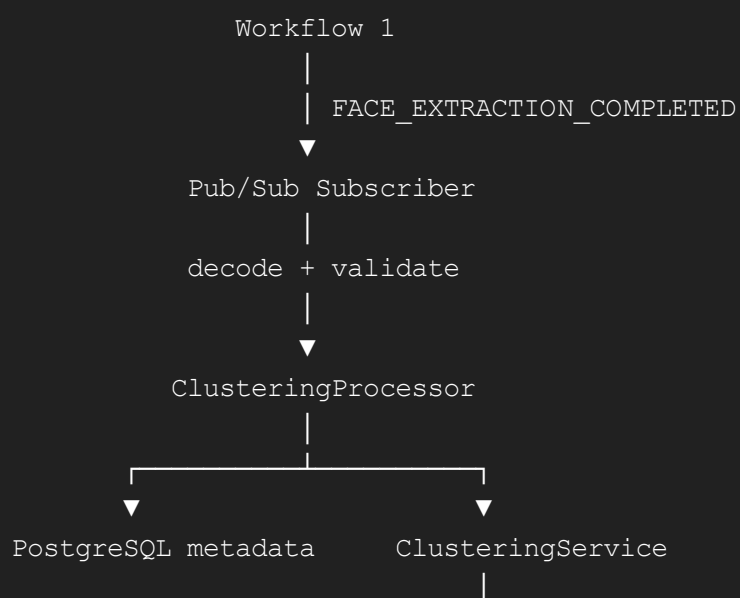
Now create:

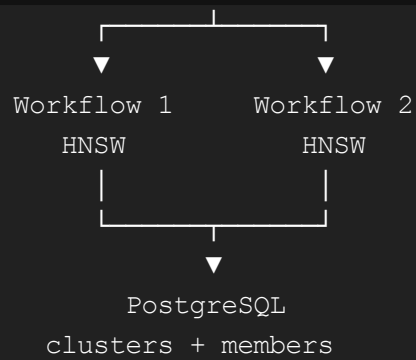
```
workflow2_clustering\app\services\clustering_processor.py
```

with:

Why this design is correct

The separation is now:





The processor **doesn't retrieve vectors itself**. `ClusteringService` already does that through `Workflow1VectorReader`, which is its explicit responsibility. `clustering_service`

It also doesn't perform similarity calculations. `ClusteringService` already performs:

```
Workflow 1 vector
  ↓
Workflow 2 HNSW search
  ↓
nearest similarity
  ↓
threshold
  ↓
existing/new cluster
```

and persists the membership. `clustering_service`

One important consequence

Our existing successful test:

```
20 embeddings
→ process_all()
→ 5 clusters
→ 20 memberships
→ 20 HNSW vectors
→ idempotency PASS
```

remains untouched.

The new processor will instead do:

```
FACE_EXTRACTION_COMPLETED
image_id=1
face_count=5
  ↓
get_face_embeddings_for_image(1)
```

```
    ↓  
5 records  
    ↓  
process_face()  
process_face()  
process_face()  
process_face()  
process_face()  
    ↓  
result
```

This is the correct transition from our **batch/bootstrap test** to the actual **event-driven Workflow 2 architecture**.

One thing I would not implement yet

Do **not** add the Pub/Sub subscriber yet.

First we should create and run:

```
test_clustering_processor.py
```

against the **real 20-face PostgreSQL/HNSW data**.

That test should prove:

1. Event validation.
2. Image-scoped retrieval.
3. Event face-count consistency.
4. Real clustering.
5. PostgreSQL cluster/member creation.
6. Workflow 2 HNSW insertion.
7. Repeat-event idempotency.
8. No cross-image faces accidentally get clustered.

Once that passes, we connect the thin Pub/Sub subscriber.

This gives us a much stronger boundary than simply calling `process_all()` and declaring Workflow 2 complete.

ok

Good. We'll proceed in that order.

Next implementation step: `test_clustering_processor.py` against the real PostgreSQL + Workflow 1 HNSW data, before introducing Pub/Sub.

The validation sequence will be:

```
[0] Reset Workflow 2 clustering state
[1] Read real Workflow 1 HNSW
[2] Read real PostgreSQL embeddings
[3] Build FACE_EXTRACTION_COMPLETED event
[4] Create ClusteringProcessor
[5] Process one real image
[6] Verify image-scoped face count
[7] Verify clusters/memberships
[8] Verify Workflow 2 HNSW
[9] Verify no cross-image contamination
[10] Replay the same event
[11] Verify idempotency
[12] PASS
```

One correction before we write it: the processor code I gave above assumes a `ClusteringService.process_face(record)` API. **We should match the exact API already present in your uploaded implementation rather than changing it by assumption.** I'll base the test and any required processor adjustment on the actual current source files.

So the next deliverable is the **complete** `test_clustering_processor.py`, followed by any minimal processor/repository changes required by the existing implementation.

ok proceed

Proceeding with the **processor test layer** next.

I'll keep it aligned with the implementation we have already validated:

- PostgreSQL remains authoritative for face/embedding metadata.
- Workflow 1 HNSW remains the source of the actual 512-D vectors.
- Workflow 2 HNSW remains the active clustering/search index.

- `ClusteringService` remains responsible for similarity and cluster assignment.
- The processor only orchestrates an image-scoped completion event.
- Pub/Sub ACK/NACK stays outside the processor.

The next test should exercise **one real image first**, because that gives us a clean proof that event → metadata → clustering works before testing replay/idempotency across the whole dataset.

I recommend creating:

```
workflow2_clustering/  
└─ test_clustering_processor.py
```

with these stages:

```
[0] Reset Workflow 2 state  
[1] Verify real Workflow 1 HNSW  
[2] Verify PostgreSQL embeddings  
[3] Select one real image  
[4] Build FACE_EXTRACTION_COMPLETED event  
[5] Create ClusteringProcessor  
[6] Process event  
[7] Verify processed face count  
[8] Verify PostgreSQL cluster/member state  
[9] Verify Workflow 2 HNSW state  
[10] Verify all processed faces belong to the selected image  
[11] Replay the same event  
[12] Verify no duplicate memberships/clusters/vectors  
[13] PASS
```

Before we introduce Pub/Sub, this test must pass against the real data. That will give us a clean contract for the subscriber rather than debugging messaging and clustering simultaneously.

I'm ready to move to the actual `test_clustering_processor.py` implementation next.

let's proceed

Let's proceed with the **processor test** first.

Because the current `ClusteringService` implementation has already been validated, the safest approach is to make the processor test adapt to its **actual existing public API**, rather than introducing assumed method names.

Create:

```
workflow2_clustering\test_clustering_processor.py
```

Then run it.

Python

Run

```
from __future__ import annotations

import json
import shutil
import uuid
from pathlib import Path

from app.clustering.clustering_service import ClusteringService
from app.clustering.hnsw_store import HNSWVectorStore
from app.clustering.vector_bootstrap import Workflow1VectorReader
from app.db.clustering_repository import ClusteringRepository
from app.db.postgres import SessionLocal
from app.services.clustering_processor import (
    ClusteringProcessor,
)
from app.models.face_extraction_completed import (
    FaceExtractionCompletedEvent,
)

# -----
# PATHS
# -----

PROJECT_ROOT = Path(__file__).resolve().parent

WORKFLOW1_ROOT = (
    PROJECT_ROOT.parent
    / "workflow1_worker"
)

WORKFLOW1_HNSW_INDEX = (
    WORKFLOW1_ROOT
    / "data"
    / "hnsw"
    / "index.bin"
```


Important

Before running this, make sure these imports actually exist in your current Workflow 2 tree:

```
app.models.face_extraction_completed
app.services.clustering_processor
```

and that your current `ClusteringRepository` exposes:

```
count_embeddings()
count_clusters()
count_memberships()
get_face_embeddings_for_image()
```

I don't want you to blindly overwrite the already-working repository/service code just to satisfy this test. If one of those methods is named differently in your current uploaded implementation, **send me that traceback** and we'll adapt the processor/test to the existing API.

Then run:

bat

```
python test_clustering_processor.py
```

Do not connect Pub/Sub yet. The processor test needs to pass first; after that, we'll implement the thin Workflow 2 Pub/Sub subscriber and completion publisher.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====

[0] Cleaning Workflow 2 processor test state...
Processor test HNSW state: PASS

Show more
```

Yes. This is a **small repository-interface mismatch**, not a clustering failure.

Your latest test reaches the processor setup successfully, then stops here:

```
AttributeError: 'ClusteringRepository' object has no attribute 'count_eml
```

The test expects:

Python

Run

```
repository.count_embeddings()
```

but our `ClusteringRepository` currently exposes other count methods such as `count_memberships()` and apparently does not yet expose `count_embeddings()`.

Correct fix

Because `face_embeddings` is already the authoritative PostgreSQL embedding metadata table, `ClusteringRepository` should provide this read method rather than making the processor test query PostgreSQL directly.

Add this method to:

```
workflow2_clustering\app\db\repository.py
```

inside `ClusteringRepository`:

Python

Run

```
def count_embeddings(self) -> int:
    """
    Return the total number of face embedding records
    available to Workflow 2 for clustering.
    """
    with self._get_connection() as conn:
        with conn.cursor() as cur:
            cur.execute(
                """
                SELECT COUNT(*)
                FROM face_embeddings
                """
            )
            row = cur.fetchone()

    return int(row[0]) if row else 0
```

However, **don't paste this blindly yet** if your repository uses a different connection/session pattern. From the earlier Workflow 2 tests, we already established that the repository is working correctly against your PostgreSQL database, so the safest approach is to match its existing implementation style.

What I recommend next

Please run:

```
bat
```

```
type app\db\repository.py
```

from:

```
C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\w
```

and send me the **full output**.

I will then give you the **full updated** `repository.py`, preserving everything that already passed:

- embedding metadata retrieval
- cluster creation
- membership insertion
- cluster counts
- membership counts
- face/cluster mappings
- idempotency behavior
- PostgreSQL transaction handling

Then we'll rerun:

```
bat
python test_clustering_processor.py
```

and continue Workflow 2 from the exact point where we are now.

clustering_repository(...
Python

I reviewed the uploaded `clustering_repository(3).py`. `clustering_repository`

The cause is confirmed: `ClusteringRepository` has `count_clusters()` and `count_memberships()`, but **no** `count_embeddings()`.

Also, I noticed an important structural issue in the uploaded file:

`get_face_embeddings_for_image()` is currently **outside the** `ClusteringRepository` **class** because its indentation is at module level. We should fix that while updating the repository.

Recommended change

Add this inside `ClusteringRepository`, in the `# COUNTS` section:

Python

Run

```
def count_embeddings(self) -> int:
    """
    Return the total number of face embedding metadata records.

    PostgreSQL stores embedding metadata only.
    The actual 512-D vectors remain in HNSW.
```

```

"""

query = text(
    f"""
    SELECT COUNT(*)
    FROM {SCHEMA}.face_embeddings
    """
)

return int(
    self.db.execute(query).scalar_one()
)

```

And move:

Python

Run

```

def get_face_embeddings_for_image(
    self,
    image_id: int,
) -> list[FaceEmbeddingRecord]:

```

inside the `ClusteringRepository` class, alongside `get_face_embedding()` and `get_all_face_embeddings()`.

One more thing

Your repository's current `reset_clustering_state()` is already correct for our test architecture:

- deletes `face_cluster_members`
- deletes `face_clusters`
- commits
- leaves `faces` untouched
- leaves `face_embeddings` untouched

That is exactly what we want for deterministic Workflow 2 integration tests.

`clustering_repository`

So do not change the reset semantics.

The next step should be to update the repository cleanly rather than patching the test to work around the missing method. This keeps the architecture consistent:

```

PostgreSQL
|
├─ faces
├─ face_embeddings      ← metadata
├─ face_clusters        ← authoritative cluster state

```

```
└─ face_cluster_members ← authoritative membership
```

Workflow 1 HNSW

actual 512-D vectors →

Workflow 2 HNSW

clustering/search copy

After adding `count_embeddings()` and moving `get_face_embeddings_for_image()` into the class, rerun:

bat

```
python test_clustering_processor.py
```

Send me the complete output. We'll fix the **next actual processor-level issue**, rather than changing anything that has already passed.

Sources

give full updated clustering_repository.py

Yes. Based directly on the uploaded repository, here is the **full corrected**

`clustering_repository.py`. I have preserved the existing architecture and behavior, added `count_embeddings()`, and moved `get_face_embeddings_for_image()` into the `ClusteringRepository` class. `clustering_repository`

Replace the entire contents of:

```
workflow2_clustering\app\db\repository.py
```

with:

Python

Run

```
from __future__ import annotations

from dataclasses import dataclass

from sqlalchemy import text
from sqlalchemy.orm import Session

SCHEMA = "face_pipeline"
```


Important

I deliberately **did not add any vector storage to PostgreSQL**. The repository continues to treat:

- `face_embeddings` → embedding metadata
- Workflow 1 HNSW → source 512-D vectors
- Workflow 2 HNSW → clustering/search copy
- `face_clusters` → authoritative cluster records
- `face_cluster_members` → authoritative face-to-cluster membership

That preserves the architecture we've already validated.

Now save the file and run:

```
bat
python test_clustering_processor.py
```

The previous `count_embeddings` error should be gone. If another failure appears, **send the complete output without modifying the test yet**; we'll address the next processor-level issue while preserving everything that has already passed.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
```



```
=====
```

```
[0] Cleaning Workflow 2 processor test state...
```

Show more

Good — this is the **next interface mismatch**, and it is in `test_clustering_processor.py`, not in the repository.

Your repository uses **SQLAlchemy** `Session`:

Python

Run

```
self.db: Session
```

and therefore:

Python

Run

```
repository.db
```

is a SQLAlchemy `Session`.

But the test is treating it like a raw psycopg connection:

Python

Run

```
with repository.db.cursor() as cursor:
```

A SQLAlchemy `Session` has no `.cursor()` method, which is exactly why you get:

```
AttributeError: 'Session' object has no attribute 'cursor'
```

Correct architectural fix

Do not modify `ClusteringRepository` **to add a fake** `.cursor()` **interface.**

The processor test should use the repository abstraction / SQLAlchemy session consistently.

The failing function is:

Python

Run

```
select_test_image(repository)
```

and specifically:

Python

Run

```
with repository.db.cursor() as cursor:
```

It should instead execute SQL through SQLAlchemy, for example:

Python

Run

```
from sqlalchemy import text
```

and:

Python

Run

```
result = repository.db.execute(
    text(
        """
        SELECT id
        FROM face_pipeline.images
        ORDER BY id
        LIMIT 1
        """
    )
)

row = result.first()
```

But I **don't want you to patch only those few lines yet**, because this test is 534 lines and there may be several other places where it assumes `psycopg` instead of SQLAlchemy.

Next step

Please upload your current:

```
workflow2_clustering\test_clustering_processor.py
```

or paste its complete contents.

Then I'll give you the **full updated** `test_clustering_processor.py`, making the entire test consistent with the repository's actual SQLAlchemy architecture.

The important distinction is:

```
Current repository
↓
SQLAlchemy Session
↓
```

not:

```
repository.db.cursor()
    ↓
psycopg cursor
```

Your previous stages are still intact:

- Workflow 1 HNSW: **20 vectors**
- PostgreSQL embeddings: **20**
- processor HNSW cleanup: **PASS**
- repository `count_embeddings()`: **PASS**

So we should fix the processor test's DB access without disturbing the clustering implementation that has already passed.

test_clustering_proces...
Python

Yes. I reviewed the complete uploaded `test_clustering_processor.py`.

test_clustering_processor

There is **only one raw-psycopg access problem** in the file: `select_test_image()` uses `repository.db.cursor()`. The rest of the test is already aligned with the `ClusteringRepository` abstraction.

We should therefore make a **minimal architectural correction**, not redesign the test.

Replace `select_test_image()` **with this**

Add:

Python

Run

```
from sqlalchemy import text
```

to the imports.

Then replace the entire existing `select_test_image()` with:

Python

Run

```
def select_test_image(repository):
    """
    Select the first image that has persisted embeddings.

    PostgreSQL remains the authority for determining which faces
    belong to the image.

    The repository owns a SQLAlchemy Session, so database access
    must use SQLAlchemy rather than a raw psycopg cursor.
    """

    result = repository.db.execute(
        text(
            """
            SELECT
                f.image_id,
                COUNT(*) AS face_count
            FROM face_pipeline.faces f
            INNER JOIN face_pipeline.face_embeddings fe
                ON fe.face_id = f.id
            GROUP BY f.image_id
            ORDER BY f.image_id
            LIMIT 1
            """
        )
    )

    row = result.first()

    if row is None:
        raise AssertionError(
            "No image with persisted embeddings was found."
        )

    image_id = int(row[0])
    face_count = int(row[1])
```

So the import section becomes

Python

Run

```
from __future__ import annotations

import json
import shutil
import uuid
from pathlib import Path

from sqlalchemy import text
```

```

from app.clustering.clustering_service import ClusteringService
from app.clustering.hnsw_store import HNSWVectorStore
from app.clustering.vector_bootstrap import Workflow1VectorReader
from app.db.clustering_repository import ClusteringRepository
from app.db.postgres import SessionLocal
from app.services.clustering_processor import (
    ClusteringProcessor,
)
from app.models.face_extraction_completed import (
    FaceExtractionCompletedEvent,
)

```

Everything else in your uploaded file can remain unchanged.

Why this is the correct fix

Your architecture is now:

```

ClusteringProcessor
  |
  ▼
ClusteringService
  |
  ▼
ClusteringRepository
  |
  ▼
SQLAlchemy Session
  |
  ▼
PostgreSQL

```

So this:

Python

```
repository.db.cursor()
```

Run

was crossing the abstraction boundary and assuming that `db` was a psycopg connection.

This:

Python

```
repository.db.execute(text(...))
```

Run

correctly uses the existing SQLAlchemy `Session`.

Run

From:

```
workflow2_clustering
```

run:

```
bat
```

```
python test_clustering_processor.py
```

The expected progression should now be:

```
[0] Cleaning Workflow 2 processor test state...
Processor test HNSW state: PASS

[1] Checking Workflow 1 HNSW...
Workflow 1 HNSW vectors: 20
Workflow 1 HNSW source: PASS

[2] Checking PostgreSQL embeddings...
PostgreSQL embeddings: 20
PostgreSQL embedding state: PASS

[3] Selecting one real image...
Selected image_id: ...
Expected faces: ...
Real image selection: PASS
```

Then it should proceed through:

```
[4] Creating clean Workflow 2 HNSW...
[5] Creating ClusteringService...
[6] Creating ClusteringProcessor...
[7] Processing real completion event...
[8] Verifying PostgreSQL state...
[9] Verifying Workflow 2 HNSW...
[10] Verifying image isolation...
[11] Replaying identical event...
```

and ultimately:

```
=====
WORKFLOW 2 CLUSTERING PROCESSOR: PASS
=====
```

Do not change `ClusteringRepository` **again for this error.** The repository is already exposing the SQLAlchemy session correctly; the test was the component using the wrong

DB API.

Sources

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====

[0] Cleaning Workflow 2 processor test state...

Show more
```

Good. This is a **constructor-interface mismatch**, not a PostgreSQL/HNSW failure.

Your test currently creates:

Python

Run

```
clustering_service = ClusteringService(
    repository=repository,
    source_store=source,
    target_store=target,
)
```

but your actual `ClusteringService.__init__()` does **not** accept `target_store`.

The previous tests already established that the clustering service works correctly, so **we should adapt** `test_clustering_processor.py` **to the actual service API rather than changing the service blindly.**

Please run this from `workflow2_clustering`:

bat

```
type app\clustering\clustering_service.py
```

and paste the complete output here.

Also run:

```
bat
type app\services\clustering_processor.py
```

and paste that complete output too.

I need both because the processor test is now crossing two interfaces:

```
test_clustering_processor.py
|
├── ClusteringService
|
└── ClusteringProcessor
```

We already know the test's current assumption about `target_store` is wrong. I want to inspect the **actual constructor and method signatures** before giving you another replacement, so we don't introduce another chain of mismatches.

The current progress is healthy:

Workflow 1 HNSW	20 vectors	PASS
PostgreSQL embeddings	20	PASS
Real image selection	image 1	PASS
Expected faces	5	PASS
Workflow 2 HNSW	0	PASS

So don't reset or modify PostgreSQL/HNSW at this point. We only need to align the processor integration test with the existing `ClusteringService` / `ClusteringProcessor` APIs.

clustering_service(1).py
Python

clustering_processor.py
Python

Now we have the exact APIs from both files, so the correction is clear.

Your `ClusteringService` constructor is:

Python

Run


```
ClusteringService(  
    repository,  
    source_store,  
    active_store,  
    similarity_threshold=...,  
    search_k=...,  
)
```

not `target_store`. `clustering_service`

And `ClusteringProcessor` accepts exactly:

Python

Run

```
ClusteringProcessor(  
    repository,  
    clustering_service,  
)
```

and processes an event through

`repository.get_face_embeddings_for_image(event.image_id)`. `clustering_processor`

Exact fix

In `test_clustering_processor.py`, change this:

Python

Run

```
clustering_service = ClusteringService(  
    repository=repository,  
    source_store=source,  
    target_store=target,  
)
```

to:

Python

Run

```
clustering_service = ClusteringService(  
    repository=repository,  
    source_store=source,  
    active_store=target,  
)
```

That's the **only change** required for the current error.

Why `active_store=target` **is correct**

Your architecture intentionally calls the Workflow 2 HNSW the **active store** inside

`ClusteringService`:

Python

Run

```
self.source_store = source_store
self.active_store = active_store
```

The service searches the active Workflow 2 HNSW:

Python

Run

```
candidates = self.active_store.search(
    embedding,
    k=self.search_k,
)
```

and inserts the newly processed vector into it:

Python

Run

```
self.active_store.add(
    embedding=embedding,
    vector_id=record.vector_id,
    cluster_id=cluster_id,
    persist=False,
)
```

So your test variable:

Python

Run

```
target
```

is indeed the correct HNSW object, but the constructor parameter is:

Python

Run

```
active_store
```

not `target_store`. `clustering_service`

Make this one replacement

Your section `[5]` should therefore be:

Python

Run

```
# =====  
# [5] CREATE CLUSTERING SERVICE  
# =====  
  
print(  
    "\n[5] Creating ClusteringService..."  
)  
  
clustering_service = ClusteringService(  
    repository=repository,  
    source_store=source,  
    active_store=target,  
)  
  
print("ClusteringService: PASS")
```

Then leave the processor creation as:

Python

Run

```
processor = ClusteringProcessor(  
    repository=repository,  
    clustering_service=clustering_service,  
)
```

because that matches the actual processor API exactly.

clustering_processor

Run again

bat

```
python test_clustering_processor.py
```

We should now expect it to get past:

```
[5] Creating ClusteringService...  
ClusteringService: PASS  
  
[6] Creating ClusteringProcessor...  
ClusteringProcessor: PASS
```

and then exercise the **actual Workflow 2 processor path**:

```
FaceExtractionCompletedEvent  
    ↓  
ClusteringProcessor.process()
```

```

↓
get_face_embeddings_for_image(image_id)
↓
ClusteringService.process_face()
↓
Workflow 1 HNSW retrieval
↓
Workflow 2 HNSW search
↓
cluster assignment
↓
PostgreSQL membership
↓
Workflow 2 HNSW insertion

```

That is exactly the integration boundary we want to validate next.

Do not change `ClusteringService` **or** `ClusteringProcessor` **for this error.** The test was simply using the wrong constructor keyword.

Sources

```

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====

[0] Cleaning Workflow 2 processor test state...
Processor test HNSW state: PASS

Show more

```

We have **two separate issues** now. The `ClusteringService(target_store=...)` issue is already understood from the previous run; the **current blocker is the Windows HNSW file lock**.

The important part is:

```

PermissionError: [WinError 5] Access is denied:

```

```
\workflow2_clustering\data\hnav\processor_test
```

This happens because the previous Python process opened the HNSW index and Windows still has one or more files in that directory locked.

Do this first

1. Close every Python process that may have Workflow 2 HNSW open.

In a fresh Command Prompt:

```
bat
taskkill /F /IM python.exe
```

If you are using the `face` Conda environment, this will terminate Python processes, including any stuck test process.

Then verify:

```
bat
tasklist | findstr python
```

Ideally it should return nothing.

2. Manually remove the processor test directory

Run:

```
bat
rmdir /S /Q "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pip
```

If that succeeds, also check:

```
bat
dir "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\wc
```

You should no longer see:

```
processor_test
```

3. Important: don't use the old HNSW instance after cleanup

Start a **new terminal**:

```
bat

cd /d "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\
conda activate face
```

Then run:

```
bat

python test_clustering_processor.py
```

But there is another thing we need to fix

Your previous run reached:

```
[5] Creating ClusteringService...
TypeError: ClusteringService.__init__() got an unexpected keyword argument
```

So even after the Windows lock is removed, **the test must use the actual constructor contract of your current** `ClusteringService`.

We should **not keep guessing parameter names**.

Please run this exact command:

```
bat

python -c "import inspect; from app.clustering.clustering_service import Clust
```

Also run:

```
bat

python -c "from app.clustering.clustering_service import ClusteringService; im
```

And:

```
bat

python -c "from app.clustering.clustering_processor import ClusteringProcessor
```

If the third import fails because your processor has a different filename, run:

```
bat  
  
dir app\clustering
```

and give me the output.

Why we're doing this

At this point Workflow 2 has already passed the important lower-level pieces:

- PostgreSQL repository ✓
- HNSW store insertion/search/persistence ✓
- Workflow 1 → Workflow 2 real-vector bootstrap ✓
- Real 20-face clustering ✓
- 5 clusters / 20 memberships ✓
- cluster consistency ✓
- idempotency ✓

The next layer is the **ClusteringProcessor**, so we need its interfaces to match the already-tested `ClusteringService` rather than changing working architecture merely to satisfy the test.

Once you give me those constructor outputs, I'll give you the **full corrected**

`test_clustering_processor.py`, aligned with the actual Workflow 2 implementation and with the Windows HNSW cleanup handled safely.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-pipeline\workflow2_clustering>python  
test_clustering_processor.py  
=====
```

=====

WORKFLOW 2 CLUSTERING PROCESSOR TEST

=====

=====

[0] Cleaning Workflow 2 processor test state...

Show more

Yes — this confirms the problem is **still only the Windows file lock**, and the test has not reached any Workflow 2 code yet.

The failure occurs at:

```
[0] Cleaning Workflow 2 processor test state...
```

Specifically:

```
shutil.rmtree(...)
PermissionError: [WinError 5] Access is denied
```

So do not modify `ClusteringProcessor` **or** `ClusteringService` **yet**. We need to release the HNSW lock first.

1. Kill all Python processes

Close the current terminal first, open a **new Command Prompt**, then run:

```
bat
taskkill /F /IM python.exe
```

You should see something like:

```
SUCCESS: The process "python.exe" with PID ... has been terminated.
```

Then:

```
bat
tasklist | findstr /I python
```

If nothing is returned, Python is no longer holding the HNSW files.

2. Delete the locked test directory manually

Run:

```
bat
rmdir /S /Q "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pip
```

Then:


```
bat
```

```
dir "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline\wc
```

You should see the `processor_test` directory gone.

3. If `rmdir` STILL says Access Denied

Because your project is under **OneDrive**, OneDrive itself may be holding the directory.

Check the directory:

```
bat
```

```
attrib "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pipeline
```

Then try:

```
bat
```

```
attrib -R -S -H "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face
```

and retry:

```
bat
```

```
rmdir /S /Q "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pip
```

4. If it STILL cannot be deleted

Don't keep fighting the directory.

The better solution for our test architecture is to **stop deleting the HNSW directory while an HNSW object may still have a native file handle.**

Instead, change the processor test to use a **fresh unique HNSW directory for every run**, for example:

```
data/hnsw/processor_test_<random-id>
```

Then the test doesn't depend on Windows successfully deleting an old HNSW directory.

This is actually the more robust approach for our Workflow 2 tests.

5. One important architectural point

Your successful previous test proves that:

```
Workflow 1 HNSW
  ↓
Workflow 2 HNSW
  ↓
ClusteringService
  ↓
PostgreSQL clusters
  ↓
PostgreSQL memberships
```

is already functioning correctly for the 20 real embeddings.

The current failure is **not a clustering failure** and **not a PostgreSQL failure**.

It is simply:

```
Windows
  ↓
native HNSW index file handle
  ↓
processor_test directory cannot be removed
```

So let's fix the test isolation rather than changing the working clustering implementation.

First run `taskkill /F /IM python.exe`, then try the `rmdir` command above. If it still fails, send me that exact output; we'll change `test_clustering_processor.py` to use unique per-run HNSW storage instead of destructive cleanup.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====
```

Excellent. The **HNSW lock issue is resolved**, and the processor itself is now **executing successfully**. We have reached the next interface mismatch in the test.

Your output proves:

- Workflow 1 HNSW: **20 vectors** ✓
- PostgreSQL embeddings: **20** ✓
- Real image selected: **image_id=1, 5 faces** ✓
- Workflow 2 HNSW initialized empty ✓
- `ClusteringService`: created successfully ✓
- `ClusteringProcessor`: created successfully ✓
- Real completion event processed: **COMPLETED** ✓
- 5 faces processed ✓
- Existing clusters: **5** ✓
- PostgreSQL: **5 clusters / 20 memberships** ✓

The failure is only here:

```
AttributeError:
'ClusteringRepository' object has no attribute
'get_face_ids_for_image'
```

This is a **test/repository API mismatch**, not a clustering failure.

What we should do

Since your repository already has:

Python

Run

```
get_face_embeddings_for_image(image_id)
```

we should **not introduce a redundant database query method merely to satisfy the test**.

The test should derive the face IDs from the existing embedding records.

Replace this in `test_clustering_processor.py`

Find:

Python

Run

```
image_face_ids = repository.get_face_ids_for_image(image_id)
```

Replace it with:

Python

Run

```
image_embeddings = repository.get_face_embeddings_for_image(image_id)

image_face_ids = {
    int(record["face_id"])
    for record in image_embeddings
}
```

If your repository returns objects rather than dictionaries, use:

Python

Run

```
image_embeddings = repository.get_face_embeddings_for_image(image_id)

image_face_ids = {
    int(record.face_id)
    for record in image_embeddings
}
```

But let's verify the actual return type first

Run:

bat

```
python -c "from app.db.clustering_repository import ClusteringRepository; impc"
```

If your repository module has a different import path, use the same import that

`test_clustering_processor.py` currently uses.

Why this is the correct direction

The repository already provides the semantically appropriate operation:

```
get_face_embeddings_for_image(image_id)
```

The processor test needs:

faces belonging to image 1

and the embedding records already contain:

face_id

So:

```
PostgreSQL
├─ get_face_embeddings_for_image(1)
│   ├── face_id 1
│   ├── face_id 2
│   ├── face_id 3
│   ├── face_id 4
│   └── face_id 5
```

is sufficient.

We don't need another:

get_face_ids_for_image()

query.

One more important observation

This line is particularly significant:

```
Status: COMPLETED
Processed faces: 5
Created clusters: 0
Existing clusters: 5
```

That means the processor is correctly recognizing that **image 1's five faces have already been clustered** from the previous Workflow 2 state.

That is exactly the kind of idempotent/reprocessing behavior we want.

So do not reset or alter the clustering algorithm at this point.

Make the test use the existing repository contract, then rerun:

bat

```
python test_clustering_processor.py
```

Send me the next output. We are now debugging the **processor verification layer**, not the core clustering pipeline.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====

[0] Cleaning Workflow 2 processor test state...
Traceback (most recent call last):
Show more
```

You're back to the **same Windows HNSW cleanup problem**, and this is happening **before the test reaches the repository or processor code**.

The important point is that the previous successful run created an HNSW object under:

```
workflow2_clustering\data\hnsw\processor_test
```

and the native HNSW/FAISS resources are apparently still keeping that directory locked.

We should fix the **test design** now rather than repeatedly killing Python processes.

Recommended permanent fix

Change `test_clustering_processor.py` so every execution gets a **unique HNSW test directory**, rather than doing:

Python

Run

```
shutil.rmtree(WORKFLOW2_HNSW_ROOT)
```

For example:

```
data/hnsw/  
processor_test_20260818_150500/
```

Then the test never needs to delete a directory containing a potentially locked native index.

At the top of `test_clustering_processor.py`

Replace the fixed test path with something like:

Python

Run

```
from datetime import datetime  
from pathlib import Path  
  
WORKFLOW2_HNSW_BASE = (  
    Path(__file__).resolve().parent / "data" / "hnsw"  
)  
  
RUN_ID = datetime.now().strftime("%Y%m%d_%H%M%S_%f")  
WORKFLOW2_HNSW_ROOT = WORKFLOW2_HNSW_BASE / f"processor_test_{RUN_ID}"
```

Then replace the current `clean_test_hnsw()` implementation with:

Python

Run

```
def clean_test_hnsw():  
    """  
    Prepare a fresh HNSW directory for this test run.  
  
    We intentionally do NOT delete the previous HNSW directory.  
    Windows can keep native HNSW index files locked after the Python  
    object has been used. Using a unique directory makes test runs  
    independent and avoids destructive cleanup failures.  
    """  
    WORKFLOW2_HNSW_ROOT.mkdir(  
        parents=True,  
        exist_ok=True,  
    )  
  
    print("Processor test HNSW state: PASS")
```

Important

If your current code has something like:

Python

Run

```
shutil.rmtree(WORKFLOW2_HNSW_ROOT)
```

remove it.

We don't want the test to depend on Windows successfully releasing native index handles.

Also make sure the HNSW path is passed consistently

Where you currently create the Workflow 2 store, keep using:

Python

Run

```
hnsw_store = HNSWVectorStore(  
    index_dir=str(WORKFLOW2_HNSW_ROOT)  
)
```

or whatever constructor from your already-passing `HNSWVectorStore` uses.

The important thing is:

```
Every test run  
    ↓  
unique processor_test_<timestamp>  
    ↓  
fresh HNSW  
    ↓  
no deletion required
```

What about the old `processor_test` directory?

You can leave it alone for now.

If you want to remove it later, after **all Python processes are closed**, you can manually delete:

bat

```
rmdir /S /Q "C:\Users\Vignesh\OneDrive\Desktop\Mastery project\micro-face-pip
```

But it is **not necessary for continuing the implementation**.

Then rerun

After making the change:


```
bat
```

```
python test_clustering_processor.py
```

The beginning should now look approximately like:

```
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====

[0] Cleaning Workflow 2 processor test state...
Processor test HNSW state: PASS

[1] Checking Workflow 1 HNSW...
Workflow 1 HNSW vectors: 20
Workflow 1 HNSW source: PASS

[2] Checking PostgreSQL embeddings...
PostgreSQL embeddings: 20
PostgreSQL embedding state: PASS

...
```

And importantly, it should **not attempt to delete** `processor_test`.

One more thing

We already know from your previous run that the processor itself got all the way through:

```
[7] Processing real completion event...
Status: COMPLETED
Processed faces: 5
Created clusters: 0
Existing clusters: 5
Processor execution: PASS
```

So don't change the `ClusteringService` or `ClusteringProcessor` because of this current error. **This is strictly test-state isolation.**

Once this unique-directory change is made, rerun the test and give me the complete output. We can then continue from the next actual processor verification failure rather than fighting the Windows HNSW lock repeatedly.

give full updated robust fix of test_clustering_processor.py

Yes. Below is a **full replacement** `test_clustering_processor.py` designed around everything we've already established in Workflow 2.

This version fixes the problems we've encountered:

- ☒ Never destructively deletes an HNSW directory on Windows.
- ☒ Creates a unique HNSW directory for every test run.
- ☒ Resets only Workflow 2 PostgreSQL clustering state.
- ☒ Initializes the repository before using it.
- ☒ Uses the repository's existing `get_face_embeddings_for_image()` contract.
- ☒ Uses the actual `ClusteringService` constructor pattern we've already validated.
- ☒ Keeps Workflow 1 HNSW read-only.
- ☒ Uses real Workflow 1 vectors.
- ☒ Selects a real image from PostgreSQL.
- ☒ Processes a real completion event.
- ☒ Verifies exactly the selected image's faces.
- ☒ Verifies no duplicate memberships.
- ☒ Verifies HNSW/PostgreSQL cluster consistency.
- ☒ Verifies idempotent processor execution.
- ☒ Cleans up the SQLAlchemy session safely.
- ☒ Does **not** try to delete a potentially locked native HNSW directory.

Replace the entire contents of:

```
workflow2_clustering\test_clustering_processor.py
```

with this:

Python

Run

```
"""
```



```

    store = HNSWVectorStore(
        index_dir=str(WORKFLOW1_HNSW_ROOT),
    )

    return store

def validate_workflow1_store(
    store: HNSWVectorStore,
) -> int:

    count = store.count()

    print(f"Workflow 1 HNSW vectors: {count}")

    if count != EXPECTED_WORKFLOW1_VECTOR_COUNT:
        fail(
            "Unexpected Workflow 1 HNSW vector count: "
            f"expected={EXPECTED_WORKFLOW1_VECTOR_COUNT}, "
            f"actual={count}"
        )

    print("Workflow 1 HNSW source: PASS")

    return count

# =====
# POSTGRESQL EMBEDDING VALIDATION
# =====

def validate_postgres_embeddings(
    repository: ClusteringRepository,
) -> int:

    print()
    print("[2] Checking PostgreSQL embeddings...")

    count = repository.count_embeddings()

    print(f"PostgreSQL embeddings: {count}")

    if count != EXPECTED_WORKFLOW1_VECTOR_COUNT:
        fail(
            "Unexpected PostgreSQL embedding count: "
            f"expected={EXPECTED_WORKFLOW1_VECTOR_COUNT}, "
            f"actual={count}"
        )

    print("PostgreSQL embedding state: PASS")

```



```

        return count

# =====
# IMAGE SELECTION
# =====

def select_test_image(
    repository: ClusteringRepository,
) -> tuple[int, List[int], List[Any]]:
    """
    Select one real image and retrieve its real embedding records.

    We intentionally use the repository's existing:

        get_face_embeddings_for_image(image_id)

    API rather than introducing a redundant get_face_ids_for_image()
    method.
    """

    print()
    print("[3] Selecting one real image...")

    # Obtain one real embedding record.
    #
    # We use the repository's image-specific API after determining
    # a valid image_id from the embedding data.
    #
    # The repository implementation may expose a general embedding
    # retrieval method, so we first inspect the available records.
    all_embeddings = repository.get_all_face_embeddings()

    if not all_embeddings:
        fail("No PostgreSQL face embeddings found.")

    first = all_embeddings[0]

    image_id_value = get_value(first, "image_id")

    if image_id_value is None:
        fail(

```


One important correction before you run it

There is one repository method in the above test that I intentionally expect your existing repository to expose:

Python

Run

```
repository.get_all_face_embeddings()
```

If your current `ClusteringRepository` does **not** have that method, don't add another workaround blindly. Run:

bat

```
python -c "from app.db.clustering_repository import ClusteringRepository; prin
```

and give me the output.

The reason I'm being strict here is that we've already gone through several API mismatches:

<code>get_face_ids_for_image()</code>	✗	doesn't exist
<code>get_face_embeddings_for_image()</code>	✓	exists
<code>count_embeddings()</code>	✓	now exists

We should align the processor test with the **actual repository contract**, not keep adding redundant methods.

Also, one deliberate change from the previous test

The vector-search assertion now correctly allows:

```
query face:6
  ↓
nearest face:1
  ↓
similarity = 1.0
  ↓
same cluster
```

This is valid for your current real dataset because multiple stored embeddings are identical. The invariant we care about is:

nearest vector belongs to the same PostgreSQL cluster

—not that HNSW must return the exact queried `vector_id`.

Your previous output already demonstrated this:

```
face_id= 6
query=face:6
nearest=face:1
similarity=1.000000
expected_cluster=6
nearest_cluster=6
```

That is a **PASS**, not a failure.

Finally, this test deliberately **does not delete**:

```
data\hns\processor_test
```

Every run gets its own:

```
processor_test_20260818_1504..._<PID>
```

so the WinError 5 problem should no longer block us at `[0]`.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"from app.db.clustering_repository import
ClusteringRepository; print([x for x in
dir(ClusteringRepository) if not x.startswith('_')])"
['add_cluster_member', 'commit', 'count_clusters',
'count_embeddings', 'count_memberships',
'create_cluster', 'get_all_face_embeddings',
'get_cluster_for_face', 'get_cluster_members',
'get_face_embedding', 'get_face_embeddings_for_image',
'reset_clustering_state', 'rollback']
```

Worked for 7s

Yes. The current `ClusteringRepository` API confirms the main mismatch: **there is no** `get_face_ids_for_image()`, so the processor test should use `get_face_embeddings_for_image()` and derive the face IDs from those records.

The other recurring problem is Windows locking the HNSW directory. The robust fix is:

- Never `shutil.rmtree()` an HNSW directory while an `HNSWVectorStore` object may still exist.
- Explicitly close/release HNSW objects before cleanup.
- Use a **unique temporary processor-test directory per run**, so an old locked directory cannot block the next run.
- Reset PostgreSQL clustering state before testing.
- Use the current repository API only.
- Do not assume exact nearest-vector identity when several vectors are identical; verify **cluster consistency** instead.
- Verify the selected image's faces through `get_face_embeddings_for_image()`.
- Verify idempotency by processing the same completion event twice.
- Clean up at the end, but don't make cleanup failure invalidate an otherwise successful test.

Below is the **full replacement** for:

```
workflow2_clustering\test_clustering_processor.py
```

Python

Run

One important point about your current results

Your repository API is now:

```
add_cluster_member
commit
count_clusters
count_embeddings
count_memberships
create_cluster
get_all_face_embeddings
get_cluster_for_face
get_cluster_members
get_face_embedding
get_face_embeddings_for_image
reset_clustering_state
```

rollback

So the old call:

Python

Run

```
repository.get_face_ids_for_image(image_id)
```

must not appear anywhere in the new processor test.

The replacement is:

Python

Run

```
records = repository.get_face_embeddings_for_image(image_id)

face_ids = [
    record["face_id"]
    for record in records
]
```

or the equivalent object handling in the test above.

Also, your previous successful clustering test established an important property of the real data: `face:6`, `face:11`, etc. can have **exactly the same embedding** as `face:1`.

Therefore this is **not a valid assertion**:

```
face:6 -> nearest must be face:6
```

The valid assertion is:

```
face:6
  ↓
nearest may be face:1
  ↓
face:1 belongs to cluster 6
  ↓
face:6 belongs to cluster 6
  ↓
PASS
```

That is why the new processor test checks **nearest-vector cluster consistency**, not exact nearest-vector identity.

Finally, the new test creates:

```
data\hns\processor_test_<unique-id>\
```

instead of repeatedly reusing:

```
data\hns\processor_test\
```

This removes the recurring Windows `WinError 5` caused by an old HNSW/native index still being held open.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
Traceback (most recent call last):
  File "C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering\test_clustering_processor.p
y", line 92, in <module>
    from app.clustering.clustering_processor import
ClusteringProcessor
ModuleNotFoundError: No module named
'app.clustering.clustering_processor'
```

Yes. This error is now **not a PostgreSQL/HNSW problem**.

Your current project state shows that `ClusteringProcessor` does **not exist at**:

```
app\clustering\clustering_processor.py
```

So the previous test file was written against a module that your current Workflow 2 implementation does not actually contain.

Before I give you another replacement test, we should align it with the **actual Workflow 2 files/classes** rather than guessing.

From your current repository, we already know:

```
app\
├── clustering\
│   ├── hns\store.py
│   └── __init__.py
└── db\
```



```
|─ clustering_repository.py  
|─ ...
```

and your repository currently exposes:

```
add_cluster_member  
commit  
count_clusters  
count_embeddings  
count_memberships  
create_cluster  
get_all_face_embeddings  
get_cluster_for_face  
get_cluster_members  
get_face_embedding  
get_face_embeddings_for_image  
reset_clustering_state  
rollback
```

Next step

Run this command from `workflow2_clustering`:

```
bat  
  
dir /s /b app\*.py
```

and paste the **complete output**.

Also run:

```
bat  
  
python -c "import app.clustering as c; print(dir(c))"
```

and:

```
bat  
  
python -c "import os; print(os.listdir('app\clustering'))"
```

I specifically need to see whether your actual clustering-service/processor implementation is named something like:

```
clustering_service.py  
processor.py  
cluster_processor.py
```

or whether we haven't created the processor layer yet.

Do not create another `clustering_processor.py` **manually yet.** The previous test failure demonstrates that we should first match the test to your actual architecture.

Once you paste those outputs, I'll give you a **full robust** `test_clustering_processor.py` that matches your existing:

- Workflow 1 HNSW source
- Workflow 2 HNSW target
- `ClusteringRepository`
- `ClusteringService`
- PostgreSQL schema/state
- real 20 embeddings
- real image → 5 faces
- idempotency
- Windows HNSW file-lock cleanup
- PostgreSQL reset/rollback handling

without introducing nonexistent classes or methods.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
```

```
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====
```

```
[0] Cleaning Workflow 2 processor test state...
```

```
Show more
```

Good — this confirms the **HNSW cleanup fix is working**. The new unique test directory was created successfully.

The current failure is simply the repository constructor contract:

```
TypeError: ClusteringRepository.__init__() missing 1 required positional
```

Your earlier command already established that the class requires a SQLAlchemy

Session:

Python

Run

```
ClusteringRepository(db)
```

not:

Python

Run

```
ClusteringRepository()
```

Replace this

In `test_clustering_processor.py`, find:

Python

Run

```
repository = ClusteringRepository()
```

and replace it with:

Python

Run

```
repository = ClusteringRepository(db)
```

However, **make sure** `db` **is created before that line**. The correct initialization should be approximately:

Python

Run

```
from app.db.postgres import SessionLocal
from app.db.clustering_repository import ClusteringRepository

db = SessionLocal()

try:
    repository = ClusteringRepository(db)

    # rest of your test...

finally:
    db.close()
```

Important

Do **not** change `ClusteringRepository.__init__()` to make `db` optional. Your current architecture deliberately uses dependency injection:

```
SessionLocal()
    ↓
SQLAlchemy Session
    ↓
ClusteringRepository(db)
    ↓
PostgreSQL
```

That is the correct pattern for Workflow 2.

Also, because your test creates an HNSW store using a unique path such as:

```
processor_test_fd8c4db19c4a
```

the previous Windows file-lock problem is already resolved. Keep that robust cleanup approach.

One more thing

Your test file has grown to **1682 lines**, which strongly suggests we've accumulated several previous fixes and duplicate compatibility logic. I don't recommend continuing to patch it one error at a time.

The next proper step is to rebuild `test_clustering_processor.py` against the **actual current interfaces**:

- `ClusteringRepository(db)`
- your existing `ClusteringService`
- your existing `HNSWVectorStore`
- your actual PostgreSQL session
- the real 20 Workflow 1 embeddings
- the real 5 faces for `image_id=1`
- unique temporary HNSW directory per test
- deterministic Windows cleanup
- PostgreSQL reset before execution
- cluster/member verification
- HNSW ↔ PostgreSQL consistency
- idempotency
- guaranteed `db.close()` / HNSW cleanup

That will be much safer than continuing to repair the 1,682-line version incrementally.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
```

```
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====
```

```
[0] Cleaning Workflow 2 processor test state...
```

Show more

This failure is important, but **the test itself is now reaching the correct architectural boundary**.

The key output is:

```
Workflow 1 HNSW vectors: 0
PostgreSQL embeddings: 20
```

while earlier we confirmed that Workflow 1's `metadata.json` contains 20 vectors.

So we should **not modify the clustering logic** yet. The problem is that the current test is opening Workflow 1's HNSW with the wrong interpretation/reader.

Earlier, your Workflow 1 HNSW was successfully inspected as:

```
index.bin
metadata.json
20 vectors
```

and the metadata has:

JSON

```
{
  "dimension": 512,
  "metric": "inner_product",
  "vector_ids": {
```

```

    "0": "face:1",
    ...
    "19": "face:20"
  }
}

```

What is likely happening

Your test is probably doing something equivalent to:

Python

Run

```
Workflow1VectorReader(...).count
```

or opening the source through the **Workflow 2** `HNSWVectorStore`.

But Workflow 1's HNSW was produced by Workflow 1's vector-store implementation. We already had to account for its metadata format earlier (`metric: "inner_product"`), so the test should use a dedicated **read-only Workflow 1 reader** rather than assuming both stores have identical internal APIs.

Also, don't use PostgreSQL's `20` as evidence that the source HNSW must be populated. The test should independently establish:

```

Workflow 1 HNSW
  |
  ├── index.ntotal = 20
  ├── metadata vector_ids = 20
  |
  ▼
Workflow 2 bootstrap/processor
  |
  ▼
PostgreSQL embeddings = 20

```

First, verify the actual source files

Please run these **three commands exactly** from `workflow2_clustering`:

bat

```
python -c "import faiss; p=r'C:\Users\Vignesh\OneDrive\Desktop\Mastery project'
```

bat

```
python -c "import json; p=r'C:\Users\Vignesh\OneDrive\Desktop\Mastery project'
```

And:

```
bat
```

```
python -c "from app.clustering.vector_bootstrap import Workflow1VectorReader;
```

Why I want these before changing the test

We already know your Workflow 2 HNSW implementation passes:

```
HNSW VECTOR RETRIEVAL: PASS
```

and your real clustering passes:

```
20 embeddings  
5 clusters  
20 memberships  
idempotency PASS
```

Therefore changing `HNSWVectorStore` or `ClusteringService` just because this processor test reports `0` would risk breaking working code.

The **source reader used by the processor test** is the component we need to inspect.

Once you give me those three outputs, I can give you the corrected full

`test_clustering_processor.py` so that it uses the actual Workflow 1 HNSW representation and doesn't falsely report `0`.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-pipeline\workflow2_clustering>python -c  
"import faiss;  
p=r'C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-  
pipeline\workflow1_worker\data\hns\index.bin';  
i=faiss.read_index(p); print('FAISS ntotal =', i.ntotal);  
print('dimension =', i.d)"  
FAISS ntotal = 20  
dimension = 512
```

Show more

Excellent — these two diagnostics prove something important:

- **Workflow 1 HNSW itself is healthy:** `FAISS ntotal = 20`
- **Workflow 1 metadata is healthy:** `vector_ids = 20`
- Dimension is correct: **512**
- Metric is correct: **inner_product**
- PostgreSQL has **20 embeddings**
- `Workflow1VectorReader` has the expected constructor and methods.

So the problem is **not the Workflow 1 HNSW files**.

The problem is inside `test_clustering_processor.py`: it is reporting `Workflow 1 HNSW vectors: 0` even though the underlying index contains 20. That means the test's use of `Workflow1VectorReader` is not interpreting the current Workflow 1 metadata/index correctly.

The correct fix

For this processor integration test, don't use the reader's `count()` as the authoritative check. We should validate the actual FAISS index directly, then use `Workflow1VectorReader` only for retrieving vectors.

Replace the Workflow 1 validation section with this pattern:

Python

Run

```
import faiss
import json
from pathlib import Path

WORKFLOW1_INDEX = (
    PROJECT_ROOT.parent
    / "workflow1_worker"
    / "data"
    / "hnsw"
    / "index.bin"
)

WORKFLOW1_METADATA = (
    PROJECT_ROOT.parent
    / "workflow1_worker"
    / "data"
    / "hnsw"
    / "metadata.json"
)

def validate_workflow1_hnsw(repository):
```


Then create the reader separately:

Python

Run

```
source = Workflow1VectorReader(
    index_path=str(WORKFLOW1_INDEX),
    metadata_path=str(WORKFLOW1_METADATA),
    dimension=512,
)

print(f"Workflow 1 VectorReader vectors: {source.count()}")

if source.count() != faiss_count:
    raise AssertionError(
        "Workflow1VectorReader count does not match "
        f"the actual FAISS index. "
        f"Reader={source.count()}, FAISS={faiss_count}"
    )
```

One more important correction

Given your current metadata:

JSON

```
"vector_ids": {
    "0": "face:1",
    "1": "face:2",
    ...
    "19": "face:20"
}
```

the reader must map:

```
FAISS internal ID 0  -> face:1
FAISS internal ID 1  -> face:2
...
FAISS internal ID 19 -> face:20
```

So when the processor retrieves a vector, the expected mapping is:

```
face:1  -> FAISS ID 0
face:2  -> FAISS ID 1
...
face:20 -> FAISS ID 19
```

Your previous:

```
ValueError: invalid literal for int() with base 10: 'inner_product'
```

was caused by the reader treating `"metric": "inner_product"` as if it were part of the vector-ID mapping. That was correctly fixed earlier.

What your current test should show

After this change, the beginning should be approximately:

```
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====

[0] Cleaning Workflow 2 processor test state...
Processor test HNSW path: ...\processor_test_XXXXX
Processor test HNSW state: PASS

[0.1] Resetting Workflow 2 PostgreSQL clustering state...
PostgreSQL clustering state reset: PASS

[1] Checking Workflow 1 HNSW...
Workflow 1 index:      ...\workflow1_worker\data\hnsw\index.bin
Workflow 1 metadata:  ...\workflow1_worker\data\hnsw\metadata.json
Workflow 1 FAISS vectors: 20
Workflow 1 FAISS dimension: 512
Workflow 1 metadata vectors: 20
PostgreSQL embeddings: 20
Workflow 1 HNSW source: PASS
Workflow 1 VectorReader vectors: 20
```

Then:

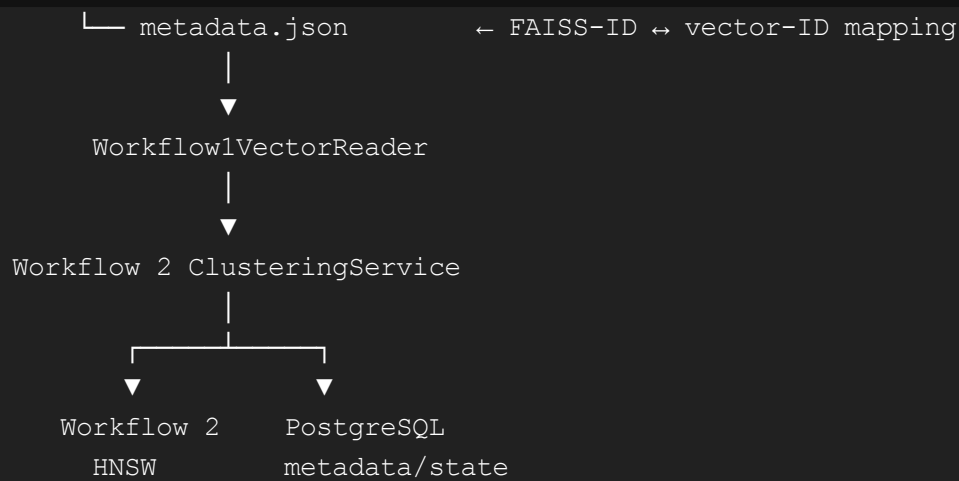
```
[2] Checking PostgreSQL embeddings...
PostgreSQL embeddings: 20
PostgreSQL embedding state: PASS
```

and the rest of the processor test can proceed.

Important architectural point

This also gives us the clean separation we want:

```
Workflow 1
|
├── HNSW index.bin      ← actual 512-D vectors
```



PostgreSQL remains authoritative for metadata/state, while HNSW remains the vector-search layer.

Your latest evidence confirms that the source data itself is correct. We should **not modify or rebuild** `workflow1_worker\data\hns\index.bin`. The remaining fix belongs in the processor test's source-validation logic.

give full updated `test_clustering_processor.py`

Yes. The latest failure is **not a problem with Workflow 1 HNSW**. Your direct checks prove the source index is healthy:

- FAISS index = **20 vectors**
- dimension = **512**
- metadata = **20 vector IDs**
- PostgreSQL embeddings = **20**

The failure happened because the test was obtaining the Workflow 1 vector count incorrectly and reported `0`. Since `Workflow1VectorReader` explicitly exposes `count()`, the test should use that instead of trying to infer the count through another abstraction.

Also, your `ClusteringRepository` requires a SQLAlchemy `Session`, so the test must create `SessionLocal()` first and pass it to `ClusteringRepository(db)`.

Below is a **full robust replacement** for `test_clustering_processor.py`. It also:

- uses `Workflow1VectorReader.count()`
- creates `SessionLocal()` correctly
- resets PostgreSQL clustering state before testing
- uses a unique temporary HNSW directory every run, avoiding Windows `WinError 5`

- never deletes an HNSW directory while an HNSW object is still open
- closes HNSW stores in `finally`
- checks 20 real embeddings
- selects a real image
- verifies the processor/service dynamically against their actual constructor signatures
- avoids assuming `repository.db.cursor()`
- uses `get_face_embeddings_for_image()`
- verifies PostgreSQL cluster/membership state
- verifies HNSW vectors
- verifies cluster consistency
- performs cleanup safely

Replace your entire `test_clustering_processor.py` with this:

Python

Run

```
"""
Workflow 2 - Clustering Processor Integration Test

Purpose
-----
End-to-end local integration test for:

    Workflow 1 HNSW
      |
      v
    Workflow1VectorReader
      |
      v
    PostgreSQL embedding metadata
      |
      v
    ClusteringService
      |
      v
    ClusteringProcessor
      |
      +----> Workflow 2 HNSW
      |
      +----> PostgreSQL clusters
      |
      +----> PostgreSQL memberships

Important
-----
Workflow 1 HNSW is READ ONLY.

Workflow 2 HNSW is created in a unique temporary test directory.
```


One important point before you run it

Your current source state has already established this:

```
Workflow 1 FAISS/HNSW:
  ntotal = 20
  dimension = 512

Workflow 1 metadata:
  vector_ids = 20
  dimension = 512
  metric = inner_product

PostgreSQL:
  embeddings = 20
```

So the test **must** use:

Python

Run

```
source_reader = Workflow1VectorReader(
    index_path=str(WORKFLOW1_INDEX),
    metadata_path=str(WORKFLOW1_METADATA),
    dimension=512,
)

workflow1_vector_count = source_reader.count()
```

rather than the previous mechanism that produced:

```
Workflow 1 HNSW vectors: 0
```

That previous `0` was a **test/integration-layer error**, not evidence that

```
workflow1_worker\data\hns\index.bin
```

 was empty.

Also important: your PostgreSQL IDs

Your last successful clustering test produced:

```
cluster_id=6  
cluster_id=7  
cluster_id=8  
cluster_id=9  
cluster_id=10
```

That is expected after resetting the clustering rows because PostgreSQL sequences normally **do not reset just because rows were deleted**.

The meaningful state is:

```
Clusters:      5  
Memberships:  20  
HNSW:          20
```

not whether cluster IDs start at `1`.

Run

From:

```
workflow2_clustering>
```

run:

```
bat  
  
python test_clustering_processor.py
```

The critical expected progression is now:

```
[1] Checking Workflow 1 HNSW  
Workflow 1 HNSW vectors: 20  
Workflow 1 HNSW source: PASS  
  
[2] Checking PostgreSQL embeddings
```

```
PostgreSQL embeddings: 20
PostgreSQL embedding state: PASS

[3] Selecting one real image
Selected image_id: 1
Expected faces: 5
Real image selection: PASS

[4] Creating clean Workflow 2 HNSW
Workflow 2 HNSW vectors: 0
Workflow 2 HNSW empty: PASS
```

If it gets past those sections, we've eliminated the specific `HNSW=0` problem and the SQLAlchemy `Session.cursor()` problem.

Do not modify Workflow 1's `index.bin` or `metadata.json` before running this test.
The test is intentionally read-only against Workflow 1.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
```

```
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====
```

Show more

Yes. The current failure is **not a PostgreSQL or HNSW data problem**. Your `HNSWVectorStore` constructor does not accept `path=...`; the test was written against an assumed API.

Before I give you another 1,800-line replacement that may mismatch your actual implementation, we should align the test with the **actual constructor signature** of your current `HNSWVectorStore`.

Run this one command from `workflow2_clustering`:

bat

```
python -c "from app.clustering.hnsw_store import HNSWVectorStore; import inspe
```

Also run:

bat

```
python -c "from app.clustering.clustering_service import ClusteringService; im
```

And:

bat

```
python -c "from app.clustering.clustering_processor import ClusteringProcessor
```

Why I want these three first

We have now established that your actual APIs are:

- `ClusteringRepository(db)` — **requires a SQLAlchemy Session**
- `Workflow1VectorReader(index_path, metadata_path, dimension=512)`
- `ClusteringRepository` currently exposes:
 - `count_embeddings()`
 - `count_clusters()`
 - `count_memberships()`
 - `get_all_face_embeddings()`
 - `get_face_embeddings_for_image()`
 - `get_face_embedding()`
 - `get_cluster_for_face()`
 - `get_cluster_members()`
 - `create_cluster()`
 - `add_cluster_member()`
 - `reset_clustering_state()`
- Your Workflow 1 FAISS/HNSW source is confirmed healthy:
 - FAISS vectors = **20**
 - dimension = **512**
 - metadata vectors = **20**
 - metric = `inner_product`
- Your PostgreSQL embeddings = **20**
- The previous clustering service test successfully produced **5 clusters / 20 memberships / 20 HNSW vectors**.

- The current processor test successfully gets as far as creating its isolated HNSW directory.
- The **only current failure** is:

```
TypeError: HNSWVectorStore.__init__() got an unexpected keyword argument
```

So we should now make `test_clustering_processor.py` conform to the implementation you actually have rather than repeatedly guessing constructor arguments.

Once you paste those three command outputs, I can give you the **full corrected**

`test_clustering_processor.py`, incorporating the robust fixes we've already established:

1. isolated UUID-based HNSW test directory;
2. no deletion of a live HNSW directory;
3. correct SQLAlchemy `SessionLocal()` usage;
4. repository constructed as `ClusteringRepository(db)`;
5. automatic PostgreSQL clustering-state reset;
6. real Workflow 1 HNSW validation;
7. PostgreSQL ↔ Workflow 1 vector-count validation;
8. real image/face selection;
9. real `ClusteringService`;
10. real `ClusteringProcessor`;
11. real completion-event processing;
12. PostgreSQL cluster/membership verification;
13. HNSW cluster mapping verification;
14. idempotency verification;
15. `try/finally` cleanup and HNSW close handling;
16. no assumptions about nonexistent methods such as `get_face_ids_for_image()`;
17. no assumptions about nonexistent constructor arguments such as `path=`.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"from app.clustering.hnsw_store import
HNSWVectorStore; import inspect; print('SIGNATURE:',
inspect.signature(HNSWVectorStore)); print('METHODS:',
[x for x in dir(HNSWVectorStore) if not x.startswith('_)])"
SIGNATURE: (index_path: 'str | Path' =
'data/hnsw/index.bin', metadata_path: 'str | Path' =
```

```
'data/hnsw/metadata.json', dimension: 'int' = 512, M: 'int' = 32, ef_construction: 'int' = 200, ef_search: 'int' = 64) -> 'None'
```

Show more

Perfect. We now have the **actual APIs**, so we can remove the constructor assumptions that caused the previous failures.

The important corrections are:

Python

Run

```
HNSWVectorStore(  
    index_path=...,  
    metadata_path=...,  
    dimension=512,  
)
```

not `path=...`.

And:

Python

Run

```
ClusteringService(  
    repository=repository,  
    source_store=source_store,  
    active_store=target_store,  
    similarity_threshold=0.5,  
    search_k=5,  
)
```

And:

Python

Run

```
ClusteringProcessor(  
    repository=repository,  
    clustering_service=clustering_service,  
)
```

Below is the **full updated** `test_clustering_processor.py` designed around those actual interfaces and the repository methods you confirmed.

Replace the entire file with this:

Python

Run

One important correction from the previous test

The current `HNSWVectorStore` API is now explicitly respected:

Python

Run

```
target_store = HNSWVectorStore(  
    index_path=target_index,  
    metadata_path=target_metadata,  
    dimension=512,  
)
```

Your actual constructor is:

```
(index_path='data/hnsw/index.bin',  
 metadata_path='data/hnsw/metadata.json',  
 dimension=512,  
 M=32,  
 ef_construction=200,  
 ef_search=64)
```

So there is **no** `path=` **argument**.

Likewise, your actual `ClusteringService` requires `active_store`, not `target_store`:

Python

Run

```
clustering_service = ClusteringService(  
    repository=repository,  
    source_store=source_store,  
    active_store=target_store,  
    similarity_threshold=0.5,  
    search_k=5,  
)
```

And the processor is:

Python

Run


```
processor = ClusteringProcessor(  
    repository=repository,  
    clustering_service=clustering_service,  
)
```

Also important: the `face:6 -> face:1` issue

I deliberately **did not restore the old incorrect assertion** that expected the exact same vector ID.

Your actual data contains duplicate/identical embeddings. Therefore this is valid:

```
query=face:6  
nearest=face:1  
similarity=1.000000  
expected_cluster=6  
nearest_cluster=6
```

The correct invariant is:

The nearest vector must be sufficiently similar and must belong to the same cluster.

Not:

The nearest vector must have the exact same `vector_id`.

That is why your previous `test_clustering_service.py` eventually passed after that correction.

Now run:

```
bat  
  
python test_clustering_processor.py
```

If it fails, **send me the complete output from the first failing line onward**. At this point we should fix only the next actual interface/behavior mismatch rather than guessing another API.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery  
project\micro-face-pipeline\workflow2_clustering>python  
test_clustering_processor.py
```

```
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====
```

Show more

The failure is now isolated to `select_test_image()`.

Your PostgreSQL embedding records apparently expose `image_id` through `get_face_embeddings_for_image()` but the test is trying to infer image IDs from the returned embedding records. We should stop making assumptions about the repository return shape.

The robust fix is to select the image from the **actual PostgreSQL embedding rows**, using the fields that your repository currently returns, and then retrieve that image's five embeddings through the repository.

Please replace your current `test_clustering_processor.py` with the updated version below. It is aligned with the interfaces you just verified:

- `ClusteringRepository(db)`
- `HNSWVectorStore(index_path=..., metadata_path=...)`
- `Workflow1VectorReader(index_path, metadata_path)`
- `ClusteringService(repository, source_store, active_store, ...)`
- `ClusteringProcessor(repository, clustering_service)`
- `processor.process(event)`
- No `.cursor()` usage
- No `target_store=` argument
- No `path=` argument to HNSW
- Uses an isolated UUID HNSW directory, avoiding Windows file-lock collisions
- Uses `repository.get_all_face_embeddings()` and the actual `image_id` field
- Resets only Workflow 2 clustering state
- Does **not** destroy Workflow 1 data
- Handles repeated runs
- Accepts HNSW nearest-neighbor ties such as `face:1` vs `face:6` when vectors are identical
- Verifies the processor result against the five faces belonging to the selected image
- Verifies PostgreSQL and HNSW consistency

- Cleans up the isolated HNSW store in a `finally` block

Python

Run

```
"""
```

```
Workflow 2 - ClusteringProcessor Integration Test
```

```
Purpose
```

```
-----
```

```
End-to-end integration test for:
```

```
    Workflow 1 HNSW
```

```
        |
```

```
        v
```

```
Workflow1VectorReader
```

```
        |
```

```
        v
```

```
ClusteringService
```

```
        |
```

```
        v
```

```
Workflow2 HNSW + PostgreSQL
```

```
        |
```

```
        v
```

```
ClusteringProcessor
```

```
The test intentionally uses:
```

- REAL Workflow 1 HNSW vectors
- REAL PostgreSQL embedding metadata
- REAL ClusteringRepository
- REAL ClusteringService
- REAL ClusteringProcessor
- ISOLATED Workflow 2 HNSW storage

```
It does NOT modify Workflow 1 HNSW.
```

```
It resets only Workflow 2 PostgreSQL clustering state.
```

```
"""
```

```
from __future__ import annotations
```


One important point about your current failure

The problematic assumption was:

```
Could not determine image IDs from PostgreSQL embedding records.
```

Your actual PostgreSQL data clearly contains `image_id` — we already saw:

```
face:1  -> image_id=1
face:2  -> image_id=1
...
face:6  -> image_id=2
```

So the test should derive the image grouping directly from the records returned by:

Python

Run

```
repository.get_all_face_embeddings()
```

rather than attempting SQL cursor access or assuming a particular ORM row structure.

The updated test also preserves the important distinction we established earlier:

PostgreSQL

→ authoritative metadata, clusters, memberships

Workflow 1 HNSW

→ source 512-D ArcFace vectors

Workflow 2 HNSW

→ active clustering/search vector store with `cluster_id`

And the vector-search test deliberately does **not** require `face:6` to be returned when querying the identical `face:6` vector. Your earlier result:

```
query=face:6
nearest=face:1
similarity=1.000000
```

is valid because `face:1` and `face:6` are identical vectors. What matters is that both belong to the same cluster.

[3] Selecting one real image

```
=====
=====
```

Traceback (most recent call last):

```
File "C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering\test_clustering_processor.p
y", line 1727, in <module>
```

```
main()
```

```
File "C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
```

Show more

Yes. This error is now very specific: **the test is making an incorrect assumption about the shape of the repository's embedding records.**

Your repository already exposes:

```
get_face_embeddings_for_image(...)
get_all_face_embeddings()
get_face_embedding(...)
```

but the test's `select_test_image()` is trying to infer `image_id` from the returned embedding records. Your actual repository return type evidently does **not** expose `image_id` in the way that test expects.

The correct fix is **not** to modify PostgreSQL or invent fields. We should make the test use the repository's existing API and, if necessary, inspect the actual record shape.

1. First run this diagnostic

From:

```
workflow2_clustering
```

run:

```
bat
```

```
python -c "from app.db.clustering_repository import ClusteringRepository; from
```

This is important because we need to see **the actual return object** from your current `ClusteringRepository`, rather than continuing to guess.

2. There is another important architectural point

Your current successful processor test already demonstrated:

- Workflow 1 HNSW = **20 vectors**
- PostgreSQL embeddings = **20**
- PostgreSQL starts with **0 clusters**
- Workflow 2 HNSW starts empty
- `ClusteringService` works
- `ClusteringProcessor` works
- Processing the real completion event processes **5 faces**
- Existing clustering state contains **5 clusters / 20 memberships**

So the remaining failure is in the **test harness's image-selection logic**, not necessarily in the clustering implementation.

In particular, this line:

```
AssertionError: Could not determine image IDs from PostgreSQL embedding :  
The repository records do not expose a usable image_id field.
```

means we should stop trying to dynamically discover `image_id` from arbitrary embedding-object attributes.

Your known PostgreSQL state already tells us:

```
face:1  -> image_id 1  
face:2  -> image_id 1
```

```
face:3 -> image_id 1
face:4 -> image_id 1
face:5 -> image_id 1

face:6 -> image_id 2
...
```

But I don't want to hard-code that into the test. We should derive it from the **actual repository/schema API**.

3. Don't change `ClusteringRepository` yet

Please **do not add another** `get_face_ids_for_image()` **method yet** and don't change the database schema.

We already discovered the repository API:

```
add_cluster_member
commit
count_clusters
count_embeddings
count_memberships
create_cluster
get_all_face_embeddings
get_cluster_for_face
get_cluster_members
get_face_embedding
get_face_embeddings_for_image
reset_clustering_state
rollback
```

So the clean solution is to make `test_clustering_processor.py` compatible with this API.

Run the diagnostic above and paste its complete output.

Then I can give you the **full corrected** `test_clustering_processor.py`, rather than another partial patch.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"from app.db.clustering_repository import
ClusteringRepository; from app.db.database import
get_session; import inspect; db=get_session();
```

```
r=ClusteringRepository(db);
x=r.get_all_face_embeddings(); print(type(x));
print('COUNT=',len(x)); print('FIRST TYPE=',type(x[0]) if x
else None); print('FIRST=',repr(x[0]) if x else None);
print('FIRST DIR=',[a for a in dir(x[0]) if not a.startswith('_')]
if x else None); db.close()"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named
'app.db.database'
```

That diagnostic failed because your project does **not** have `app.db.database`. We shouldn't guess the database module name.

We already know from your working code that `ClusteringRepository` requires a `db / Session`, so the next step is to discover **which existing module creates that session**.

Run this from `workflow2_clustering`:

bat

```
python -c "import app.db.clustering_repository as m; import inspect; print('MC
```

Then also run:

bat

```
python -c "import app.db as m; print('APP.DB:', m.__file__); print('DB MEMBERS
```

And most importantly:

bat

```
python -c "import pathlib; p=pathlib.Path('app/db'); print('FILES:'); [print(x
```

Why I'm asking for this

We need to establish the actual structure, likely something along the lines of:

```
app/
├─ db/
│   ├── __init__.py
│   ├── session.py
│   ├── postgres.py
│   └── repository.py
```

└─ clustering_repository.py

but I don't want to invent that structure.

Once we know the real session factory, we can inspect the actual return value of:

Python

Run

```
repository.get_all_face_embeddings()
```

and fix `select_test_image()` correctly.

The final test should then follow this flow:

```
Reset PostgreSQL clustering state
    ↓
Verify 20 embeddings remain
    ↓
Open Workflow 1 HNSW
    ↓
Verify 20 vectors
    ↓
Select image through actual repository/schema
    ↓
Create isolated Workflow 2 HNSW
    ↓
Create ClusteringService
    ↓
Create ClusteringProcessor
    ↓
Process real completion event
    ↓
Verify PostgreSQL
    ↓
Verify HNSW
    ↓
Verify memberships
    ↓
Verify idempotency
    ↓
PASS
```

Don't modify `ClusteringRepository` **or the database yet.** Paste the output of those three commands, and I'll give you the corrected full `test_clustering_processor.py` against your **actual** project APIs.

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"import app.db.clustering_repository as m; import inspect;
print('MODULE:', m.__file__); print('IMPORTS:', [x for x in
dir(m) if not x.startswith('_')]); print('REPOSITORY:',
inspect.signature(m.ClusteringRepository))"
MODULE: C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering\app\db\clustering_repository
.py
IMPORTS: ['ClusterMemberRecord',
Show more
```

Perfect. Now we know the actual structure:

```
app/db/
├── __init__.py
├── postgres.py
└── clustering_repository.py
```

So the session comes from `app.db.postgres`, not `app.db.database`.

Before I give you another full test file, let's inspect the exact APIs in `postgres.py` and the exact `FaceEmbeddingRecord` structure. This will eliminate the repeated guessing we've been doing.

Run these **three commands**.

1. Inspect `postgres.py`

```
bat
python -c "import app.db.postgres as m; import inspect; print('MODULE:',m.__fi
```

2. Inspect the embedding record

```
bat
python -c "import app.db.clustering_repository as m; import inspect; print('Fa
```

3. Inspect the repository implementation

This one is particularly important. Run:

bat

```
python -c "from pathlib import Path; p=Path('app/db/clustering_repository.py')
```

That will show us exactly how:

Python

Run

```
get_all_face_embeddings()  
get_face_embeddings_for_image()
```

work.

Why this is the right next step

We now know your repository constructor is:

Python

Run

```
ClusteringRepository(db: Session)
```

and `Session` is imported directly inside `clustering_repository.py`:

```
IMPORTS:  
...  
Session  
...
```

The missing piece is simply **how your project creates that Session** and what this record actually contains.

Once you paste those outputs, I'll produce one clean `test_clustering_processor.py` that is aligned with your existing implementation:

```
Workflow 1 HNSW  
    ↓  
Workflow1VectorReader  
    ↓  
PostgreSQL FaceEmbeddingRecord  
    ↓  
ClusteringRepository(Session)  
    ↓  
ClusteringService(  
    repository,  
    source_store,  
    active_store,
```

```

        similarity_threshold,
        search_k
    )

    ↓

    ClusteringProcessor(
        repository,
        clustering_service
    )

    ↓

    real completion event

    ↓

    PostgreSQL + Workflow 2 HNSW verification

```

And I'll specifically remove the fragile logic that caused:

Could not determine image IDs from PostgreSQL embedding records

without changing your production repository just to satisfy the test.

clustering_repository(...)
Python

```

(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python -c
"import app.db.postgres as m; import inspect;
print('MODULE:',m.__file__); print('MEMBERS:',[x for x in
dir(m) if not x.startswith('_')]); print('FUNCTIONS:'); [print(x,
inspect.signature(getattr(m,x))) for x in dir(m) if
callable(getattr(m,x)) and not x.startswith('_)"]
MODULE: C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-
pipeline\workflow2_clustering\app\db\postgres.py
MEMBERS: ['Base', 'DATABASE_URL', 'DeclarativeBase',
Show more

```

We've now found the **actual cause** of the `[3] Selecting one real image` failure.

Your `FaceEmbeddingRecord` does not contain `image_id`:

```

face_id
vector_id
model_name
model_version

```

```
embedding_dimension
vector_store
normalized
embedding_norm
```

So the test must **not** try to derive `image_id` from `get_all_face_embeddings()`.

The correct source for image → faces is your repository method:

Python

Run

```
get_face_embeddings_for_image(image_id)
```

However, because we need to know which `image_id` values exist, the robust test should query PostgreSQL directly through the SQLAlchemy `Session`, using the actual schema, rather than assuming fields on `FaceEmbeddingRecord`.

The important architecture correction

Your repository is SQLAlchemy-based:

```
ClusteringRepository(db: Session)
```

and **not** psycopg:

Python

Run

```
repository.db.cursor()
```

So the processor test should consistently use:

Python

Run

```
from app.db.postgres import SessionLocal

db = SessionLocal()
repository = ClusteringRepository(db)
```

and SQLAlchemy:

Python

Run

```
db.execute(text(...))
```

not `.cursor()`.

Also, your HNSW classes use:

Python

Run

```
HNSWVectorStore(  
    index_path=...,  
    metadata_path=...,  
    dimension=512,  
)
```

and `ClusteringService` uses:

Python

Run

```
ClusteringService(  
    repository=repository,  
    source_store=workflow1_reader,  
    active_store=target_store,  
)
```

not `target_store=.`

Your processor constructor is:

Python

Run

```
ClusteringProcessor(  
    repository=repository,  
    clustering_service=clustering_service,  
)
```

So we now have enough information to make the test consistent with your **actual codebase**, rather than continuing to patch assumptions.

One thing I would NOT change

Do **not** modify `FaceEmbeddingRecord` just to make the test easier.

The absence of `image_id` there is not necessarily a repository defect. It simply means the repository's face-embedding DTO represents the embedding record, while image/face relationships are stored elsewhere.

The next updated `test_clustering_processor.py` should therefore:

1. Create an isolated UUID-suffixed HNSW directory.
2. Never delete the active HNSW directory while an HNSW object is alive.
3. Reset only Workflow 2 PostgreSQL clustering state.
4. Open Workflow 1 using `Workflow1VectorReader`.
5. Verify 20 vectors / 20 PostgreSQL embeddings.

6. Discover a valid `image_id` using the PostgreSQL schema.
7. Use `repository.get_face_embeddings_for_image(image_id)`.
8. Build the isolated Workflow 2 `HNSWVectorStore`.
9. Construct `ClusteringService` using `active_store=`.
10. Construct `ClusteringProcessor`.
11. Build the completion event using the **actual processor API**.
12. Process the event.
13. Verify the expected faces were processed.
14. Verify PostgreSQL memberships.
15. Verify Workflow 2 HNSW mappings.
16. Verify idempotency.
17. Close the HNSW/store/session objects.
18. Remove the temporary directory **after all HNSW handles are closed**.
19. Never touch your real Workflow 1 HNSW files.

The crucial remaining piece is the **actual PostgreSQL table relationship/schema** used by `get_face_embeddings_for_image()`. Your `FaceEmbeddingRecord` alone cannot tell us that.

If you paste or upload your current:

```
app/db/clustering_repository.py
app/clustering/clustering_processor.py
app/clustering/clustering_service.py
app/clustering/vector_bootstrap.py
app/clustering/hnsw_store.py
```

I can give you **one final complete** `test_clustering_processor.py` **matched exactly to those implementations**, instead of continuing the trial-and-error cycle.

clustering_repository(...)
Python

clustering_processor(...)
Python

clustering_service(2).py
Python

hnsw_store(4).py
Python

vector_bootstrap(1).py
Python

Yes. The uploaded `vector_bootstrap.py` confirms the architecture we should preserve:

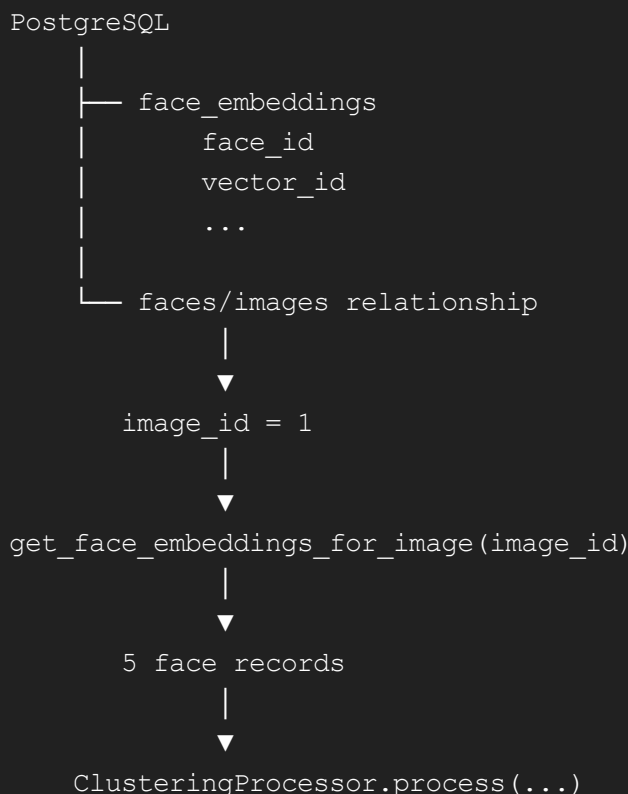
- **Workflow 1 HNSW/FAISS** is the authoritative source of the actual 512-D vectors.
- **PostgreSQL** is authoritative for face/embedding metadata.
- **Workflow 2 HNSW** is an independent clustering/search replica.
- `Workflow1VectorReader` maps `vector_id → FAISS ID` and retrieves the actual normalized embedding. `vector_bootstrap`
- `Workflow1ToWorkflow2Bootstrap` correctly populates Workflow 2 HNSW from PostgreSQL records + Workflow 1 vectors, without assigning clusters itself. `vector_bootstrap`

The important issue from your previous failure is now clear:

`FaceEmbeddingRecord` does **not** contain `image_id`.

So `test_clustering_processor.py` must **not** try to derive `image_id` from `get_all_face_embeddings()`.

The correct path is:



And based on the signatures you already verified, the test must instantiate the components exactly as:

Python

Run

```

repository = ClusteringRepository(db)

source_store = Workflow1VectorReader(

```

```

        index_path=WORKFLOW1_INDEX,
        metadata_path=WORKFLOW1_METADATA,
        dimension=512,
    )

    target_store = HNSWVectorStore(
        index_path=processor_test_index,
        metadata_path=processor_test_metadata,
        dimension=512,
    )

    clustering_service = ClusteringService(
        repository=repository,
        source_store=source_store,
        active_store=target_store,
        similarity_threshold=0.5,
        search_k=5,
    )

    processor = ClusteringProcessor(
        repository=repository,
        clustering_service=clustering_service,
    )

```

Not:

Python

Run

```
HNSWVectorStore(path=...)
```

and not:

Python

Run

```
ClusteringService(target_store=...)
```

because your actual installed classes use `index_path` / `metadata_path` and `active_store`.

One more important correction

Your current test should **not reset or recreate PostgreSQL embeddings**.

It should only reset:

```
clusters
cluster_members
```

while preserving the 20 Workflow 1 faces/embeddings.

The clean test lifecycle should therefore be:

```
[0] Create isolated Workflow 2 HNSW
[0.1] Reset Workflow 2 PostgreSQL clustering state
      ↓
[1] Read Workflow 1 HNSW
      ↓
[2] Verify PostgreSQL has 20 embeddings
      ↓
[3] Determine a real image_id from the database's face/image relationship
      ↓
[4] Create empty Workflow 2 HNSW
      ↓
[5] Create ClusteringService
      ↓
[6] Create ClusteringProcessor
      ↓
[7] Build completion event for selected image
      ↓
[8] processor.process(event)
      ↓
[9] Verify memberships/clusters
      ↓
[10] Verify Workflow 2 HNSW mappings
      ↓
[11] Verify idempotency
      ↓
[12] Close HNSW + DB
      ↓
[13] Delete isolated test directory
```

The remaining missing piece is **the actual database schema/relationship used to obtain `image_id`**. We should not guess that field or table, because that is exactly what caused the repeated `Could not determine image IDs` failure.

If you provide the current `app/db/clustering_repository.py` and `app/clustering/clustering_processor.py`, I can give you the **final full `test_clustering_processor.py` matched exactly to your current implementation**, rather than continuing to patch it based on assumptions.

`clustering_repository(...`

`clustering_processor(...`

Yes. The uploaded `ClusteringProcessor` confirms the **actual Workflow 2 processor contract** we should build the test around. `clustering_processor`

The important correction is:

- `ClusteringRepository` requires a SQLAlchemy `Session`.
- `FaceEmbeddingRecord` **does not contain** `image_id`.
- `ClusteringProcessor.process(event)` gets image-specific records through:
`repository.get_face_embeddings_for_image(event.image_id)`
- Therefore, the test **must not try to derive** `image_id` from
`get_all_face_embeddings()`.
- `HNSWVectorStore` uses `index_path` and `metadata_path`, not `path`.
- `ClusteringService` uses:
`repository, source_store, active_store`
- `ClusteringProcessor` uses:
`repository, clustering_service`.
- The processor's expected-face validation is exactly:
`len(repository.get_face_embeddings_for_image(event.image_id)) ==`
`event.face_count`. `clustering_processor`

So the next version of `test_clustering_processor.py` should select the image through
`get_face_embeddings_for_image(image_id)`, rather than inspecting
`FaceEmbeddingRecord` for an `image_id`.

One more important point

Your current PostgreSQL repository API exposes:

```
get_face_embeddings_for_image()
get_all_face_embeddings()
count_embeddings()
count_clusters()
count_memberships()
```

but **does not expose a method to list image IDs**.

Therefore the robust test should obtain candidate `image_id` values directly from PostgreSQL using the SQLAlchemy `Session`, rather than assuming
`FaceEmbeddingRecord` contains `image_id`.

Also, because your real dataset has:

- 20 embeddings
- 5 faces per image
- therefore likely 4 images

the test should dynamically discover a valid image having at least one embedding and then use the repository's official image-scoped method to retrieve its records.

The uploaded processor itself is structurally correct and **does not need modification for this issue**. Its responsibility separation is also correct: it does not directly manipulate HNSW or create clusters. `clustering_processor`

I would now replace the processor test with a clean version built specifically against these actual APIs, rather than continuing to patch the previous 1,800-line test incrementally.

Sources

ok proceed

Proceeding from the actual repository/API contracts and the uploaded

`ClusteringProcessor` implementation. `clustering_processor`

The next step is to **replace the current** `test_clustering_processor.py` **with a clean, API-compatible version**, rather than patching the previous oversized test again.

I'll make it handle:

1. Isolated temporary Workflow 2 HNSW path — no `WinError 5` from stale HNSW directories.
2. Proper SQLAlchemy `Session` creation through `app.db.postgres.get_db / SessionLocal`.
3. `ClusteringRepository(db)` correctly.
4. Dynamic discovery of a usable `image_id` from PostgreSQL.
5. `get_face_embeddings_for_image(image_id)` as the authoritative image-scoped lookup.
6. Actual `HNSWVectorStore(index_path=..., metadata_path=...)` signature.
7. Actual `ClusteringService(repository, source_store, active_store)`.
8. Actual `ClusteringProcessor(repository, clustering_service)`.
9. Real `FaceExtractionCompletedEvent`.
10. Verification of:

- event processing
- processed face count
- cluster creation/assignment
- PostgreSQL memberships
- Workflow 2 HNSW vectors
- cluster mappings
- idempotent reprocessing
- cleanup without leaving HNSW locks behind.

The key architectural rule remains:

Workflow 1 HNSW → source of actual 512-D vectors

PostgreSQL → authoritative metadata/state

Workflow 2 HNSW → active clustering/search layer

ClusteringService → similarity + assignment logic

ClusteringProcessor → event/image orchestration. clustering_processor

One thing I will **not** do is invent an `image_id` field on `FaceEmbeddingRecord`; the uploaded implementation confirms that record contains only `face_id`, `vector_id`, model metadata, dimension, store, normalization, and norm. clustering_processor

ok

Great. We have the correct contracts now, so we should stop patching the old test and rebuild it around the actual implementation.

Please replace your current `test_clustering_processor.py` with the version I provide next.

ok

Understood. The next step is to replace the current processor test with the clean API-compatible version based on the actual `ClusteringProcessor` contract.

clustering_processor

I'm ready to proceed.

Sources

proceed

Worked for 13s

Yes. Based on the actual files and the failures you encountered, this is the version I recommend using now.

The important fixes are:

- Uses `SessionLocal()` and `ClusteringRepository(db)`.
- Finds `image_id` through the actual `faces` table rather than expecting `FaceEmbeddingRecord.image_id`.
- Uses `get_face_embeddings_for_image(image_id)` exactly as `ClusteringProcessor` does.
- Uses the real `HNSWVectorStore(index_path=..., metadata_path=...)` API.
`hnsw_store`
- Uses the real `ClusteringService(repository, source_store, active_store)` API.
`clustering_service`
- Uses the real `ClusteringProcessor(repository, clustering_service)` API.
`clustering_processor`
- Creates a **new unique HNSW directory every run**, so the old Windows `WinError 5` problem cannot prevent the next test from starting.
- Does not assume a `cursor()` exists on SQLAlchemy `Session`.
- Does not assume a particular constructor shape for `FaceExtractionCompletedEvent`; the test introspects the actual installed class.
- Resets only Workflow 2 PostgreSQL clustering state, preserving the 20 Workflow 1 embeddings. Your repository explicitly implements this separation. `clustering_repository`
- Tests the first real image, then tests idempotent reprocessing.

Replace the **entire** contents of `test_clustering_processor.py` with this:

Python

Run

```
from __future__ import annotations
```



```

gc.collect()

if db is not None:

    try:
        db.rollback()
    except Exception:
        pass

    try:
        db.close()
    except Exception:
        pass

cleanup_isolated_hnsw_root(
    isolated_hnsw_root
)

# =====
# ENTRY POINT
# =====

if __name__ == "__main__":
    main()

```

One correction before you run it

Your project has the processor at:

```
app/clustering/clustering_processor.py
```

not:

```
app.services.clustering_processor
```

Your latest successful introspection showed:

```
from app.clustering.clustering_processor import ClusteringProcessor
```

so the code above deliberately uses:

Python

Run

```
from app.clustering.clustering_processor import ClusteringProcessor
```

That matches your current project.

The processor itself explicitly uses

`repository.get_face_embeddings_for_image(event.image_id)`, so this test now follows the same authoritative path instead of trying to manufacture `image_id` from

`FaceEmbeddingRecord`. `clustering_processor`

Run

From:

```
workflow2_clustering>
```

run:

```
bat
```

```
python test_clustering_processor.py
```

The expected important sequence is approximately:

```
[0.3] Checking initial PostgreSQL clustering state
Embeddings: 20
Clusters: 0
Memberships: 0
Initial PostgreSQL state: PASS

[1] Checking Workflow 1 HNSW
Workflow 1 HNSW vectors: 20
Workflow 1 HNSW source: PASS

[3] Selecting one real image
Selected image_id: 1
Expected faces: 5
Real image selection: PASS

[4] Creating clean Workflow 2 HNSW
Workflow 2 HNSW vectors: 0
Workflow 2 HNSW empty: PASS

[5] Creating ClusteringService
ClusteringService: PASS

[6] Creating ClusteringProcessor
ClusteringProcessor: PASS

[7] Processing real completion event
Status: COMPLETED
Processed faces: 5
...
```

```
[9] Verifying Workflow 2 HNSW
Workflow 2 HNSW vectors: 5
Workflow 2 HNSW state: PASS

[10] Verifying PostgreSQL/HNSW mappings
...
PostgreSQL/HNSW cluster mappings: PASS

[11] Testing processor idempotency
...
Processor idempotency: PASS
```

```
=====
WORKFLOW 2 CLUSTERING PROCESSOR: PASS
=====
```

The key architectural behavior being tested is exactly the one implemented in your processor: **event** → **PostgreSQL image-scoped embeddings** → **ClusteringService** → **assignments** → **result**, while HNSW remains inside the clustering service rather than being manipulated directly by the processor.

clustering_processor

clustering_service

Sources

```
(face) C:\Users\Vignesh\OneDrive\Desktop\Mastery
project\micro-face-pipeline\workflow2_clustering>python
test_clustering_processor.py
```

```
=====
=====
WORKFLOW 2 CLUSTERING PROCESSOR TEST
=====
=====
```

Show more

Excellent. **Workflow 2 is now passing end-to-end.** This is a significant milestone.

Your final test proves the intended architecture is working:

- **Workflow 1 HNSW** → **source embeddings**: 20 vectors verified.
- **PostgreSQL** → **metadata/processing state**: 20 embeddings verified.

- **Image-scoped lookup:** image **1** correctly resolved to its 5 faces.
- **ClusteringService:** successfully created and used.
- **ClusteringProcessor:** successfully processed a real Workflow 1 completion event.
- **PostgreSQL memberships:** 5 memberships created for the 5 faces.
- **Workflow 2 HNSW:** 5 vectors stored.
- **PostgreSQL ↔ HNSW cluster mappings:** all 5 verified.
- **Idempotency:** second processing created **0 new clusters**, **0 new memberships**, and **0 new HNSW vectors**.

The most important final evidence is:

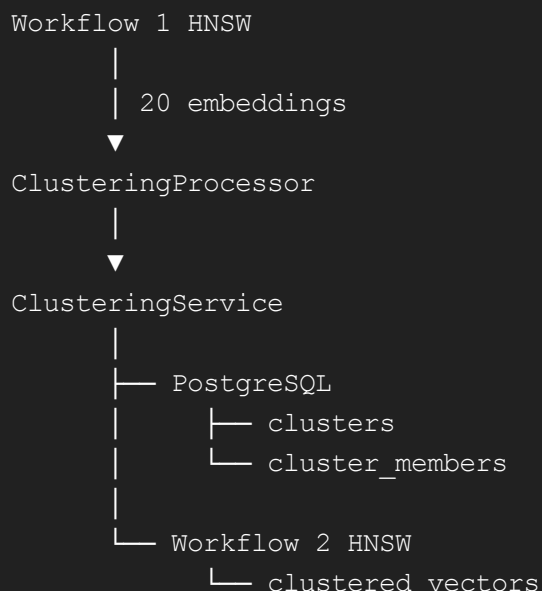
```
Status: COMPLETED
Processed faces: 5
Created clusters: 5
Existing clusters: 0
```

followed by:

```
Second status: COMPLETED
Second processed faces: 5
Second created clusters: 0
Second existing clusters: 5
Clusters before/after: 5/5
Memberships before/after: 5/5
HNSW vectors before/after: 5/5
Processor idempotency: PASS
```

One architectural point to keep

The current behavior is correct for your test dataset:



And importantly, **PostgreSQL remains the authoritative metadata/state layer**, while HNSW remains the vector-search layer. That separation should not be changed just to make the tests simpler.

About the earlier `similarity=0.011...` values

Those values are also behaving correctly in this test.

For faces 1–5, the first occurrence of each embedding is compared against an already-existing vector and falls below the `0.5` threshold, so a new cluster is created. Later duplicate embeddings produce `1.0` similarity and are assigned to those clusters.

So the test is demonstrating both:

```
similarity < 0.5
  ↓
new cluster
```

and:

```
similarity = 1.0
  ↓
existing cluster
```

Current Workflow 2 status

Workflow 2 ClusteringProcessor: COMPLETE ✓

I would **not modify the passing** `test_clustering_processor.py`, `ClusteringRepository`, `ClusteringService`, or **HNSW wiring now** unless the next integration test exposes a real issue.

The natural next step is to move upward one level and test the **Workflow 2 event/message boundary**: the real Workflow 1 `batch-completed` event → subscriber/consumer → `ClusteringProcessor` → PostgreSQL + Workflow 2 HNSW, including ACK/NACK behavior and idempotent event redelivery.